LAPORAN PRAKTIKUM

Komunikasi dan Jaringan Komputer

RIP, Analisis Server, dan Default Gateway



Dosen Pengampu

Dr. Ferry Astika Saputra, ST, M.Sc

NIP. 197708232001121002

Disusun Oleh

Aprilia Dwi Cristyana

NRP. 1226800011

PROGRAM STUDI S2 TEKNIK INFORMATIKA

DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

BAB 1 Menerapkan dan Menganalisa RIP (Dynamic Routing)

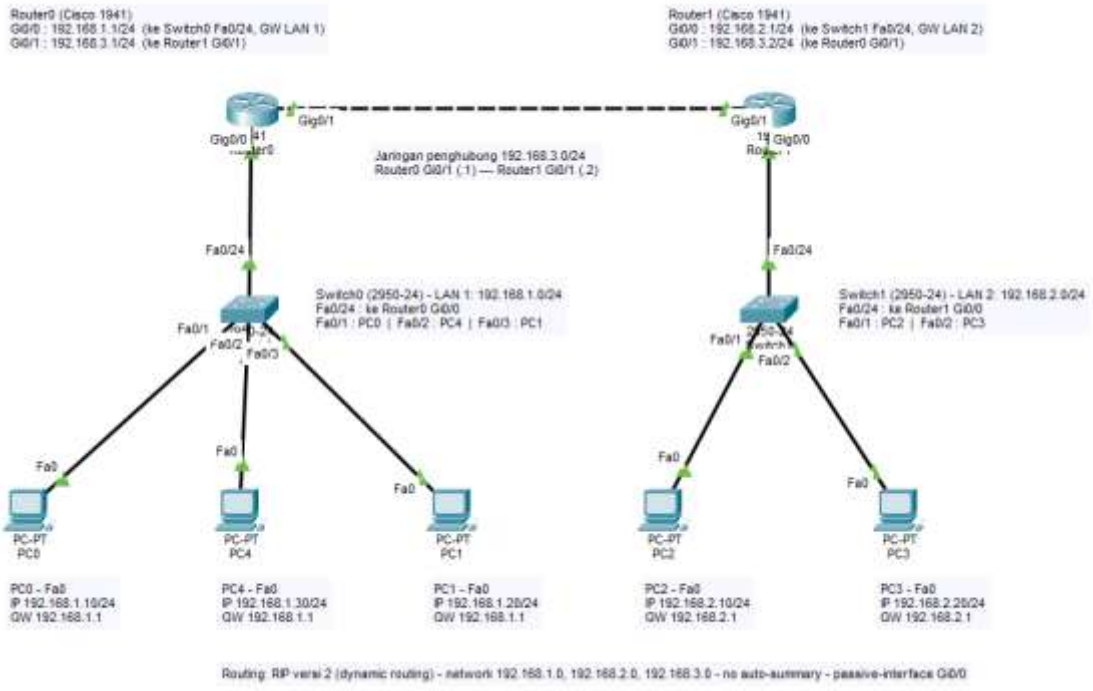
1.1 Tujuan

Menerapkan RIP versi 2 sebagai pengganti rute statis pada topologi dua router, menguji konektivitas antar-LAN, dan menganalisis perbedaan pembentukan serta pembaruan rute.

1.2 Topologi dan pengalamatan

Topologi tetap sama seperti tugas static routing: dua router terhubung lewat jaringan 192.168.3.0/24, masing-masing melayani satu LAN lewat switch. File: `RIP_Dynamic_Routing_v2.pkt`.

Perangkat	Interface	Alamat IP	Default gateway
Router0 (1941)	Gi0/0 (LAN 1)	192.168.1.1/24	-
Router0 (1941)	Gi0/1 (penghubung)	192.168.3.1/24	-
Router1 (1941)	Gi0/0 (LAN 2)	192.168.2.1/24	-
Router1 (1941)	Gi0/1 (penghubung)	192.168.3.2/24	-
PC0 / PC4 / PC1	Fa0	192.168.1.10 / .30 / .20	192.168.1.1
PC2 / PC3	Fa0	192.168.2.10 / .20	192.168.2.1



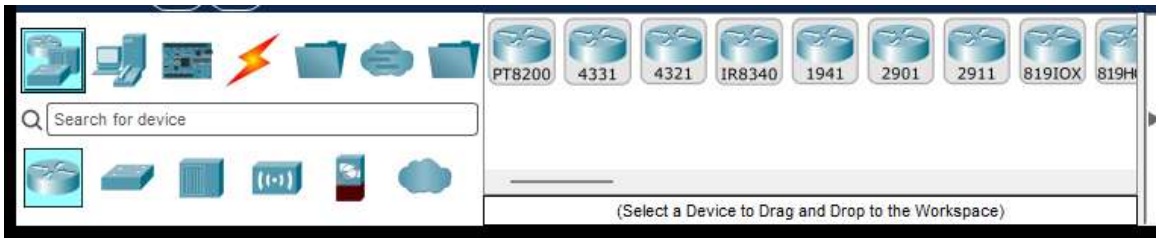
Gambar 1.1 Topologi RIP

Topologi terdiri atas dua LAN dan satu jaringan penghubung antar-router. Router0 melayani LAN 192.168.1.0/24 melalui Gi0/0 dengan alamat 192.168.1.1, sedangkan Router1 melayani LAN 192.168.2.0/24 melalui Gi0/0 dengan alamat 192.168.2.1. Kedua router terhubung melalui Gi0/1 pada jaringan 192.168.3.0/24, menggunakan alamat 192.168.3.1 dan 192.168.3.2. PC0, PC4, dan PC1 berada di LAN Router0, sedangkan PC2 dan PC3 berada di LAN Router1. Setiap PC menggunakan alamat interface router pada LAN-nya sebagai default gateway. Label interface dan port pada gambar menunjukkan jalur fisik paket dari PC, melalui switch, menuju router dan LAN tujuan.

1.3 Proses pembuatan topologi

Pembuatan topologi dilakukan melalui tujuh langkah, mulai dari penempatan router, switch, dan PC, penyambungan kabel, pengisian alamat IP PC dan router, hingga pengaktifan RIP.

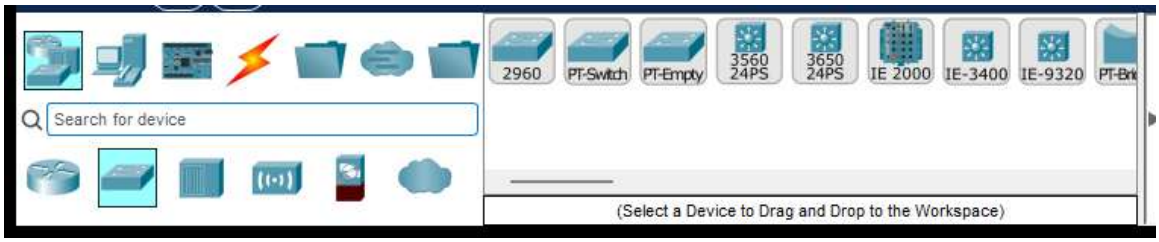
Langkah 1: Menaruh router. Klik kategori Network Devices, pilih Routers, lalu seret Cisco 1941 ke workspace sebanyak dua kali (Router0 dan Router1).



Gambar 1.2 Palet Network Devices > Routers di Cisco Packet Tracer

Subkategori Routers pada bagian kiri bawah menampilkan pilihan model router. Model Cisco 1941 dipilih untuk menyediakan dua interface GigabitEthernet yang diperlukan pada setiap router dalam topologi ini.

Langkah 2: Menaruh switch. Pilih sub-kategori Switches, lalu seret switch ke workspace dua kali (Switch0 dan Switch1). Model 2950-24 dapat dicari lewat kotak Search for device bila tidak tampil di daftar.



Gambar 1.3 Palet Network Devices > Switches

Switch menghubungkan PC dan router dalam satu LAN. Untuk meneruskan frame pada Layer 2, switch tidak memerlukan alamat IP manajemen. Alamat IP hanya diperlukan apabila switch akan dikelola melalui jaringan.

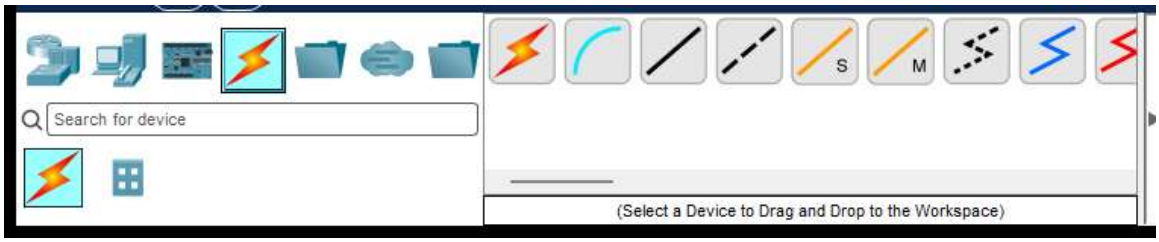
Langkah 3: Menaruh PC. Klik kategori End Devices, pilih PC, lalu seret lima PC: PC0, PC4 dan PC1 di LAN 1, PC2 dan PC3 di LAN 2.



Gambar 1.4 Palet End Devices > PC

Perangkat pengguna akhir seperti PC, laptop dan server ada di kategori ini.

Langkah 4: Menyambungkan kabel. Klik kategori Connections, pilih Copper Straight-Through, klik perangkat asal lalu pilih port-nya, kemudian klik perangkat tujuan dan pilih port-nya.

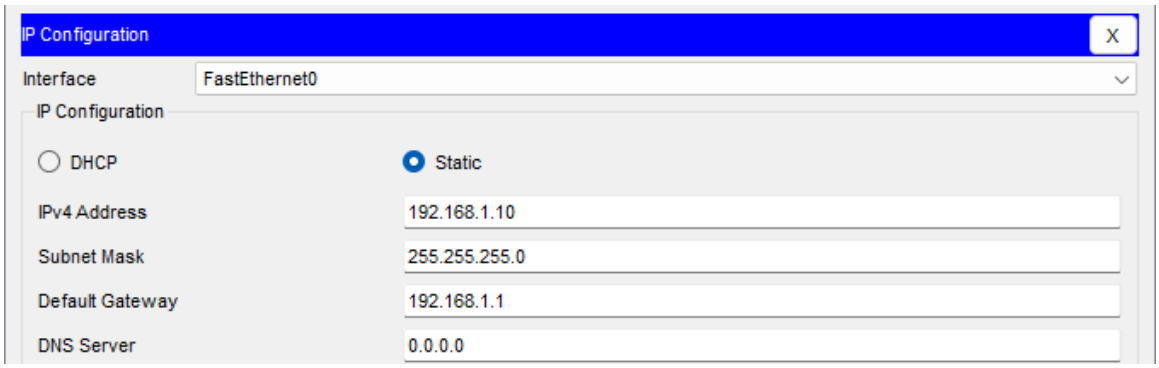


Gambar 1.5 Palet Connections (jenis kabel)

Kabel straight-through dipakai untuk PC ke switch dan switch ke router. Untuk router ke router dapat memakai copper cross-over, atau tipe Automatic yang memilih kabel sendiri.

Kabel	Dari	Ke
1	PC0 Fa0	Switch0 Fa0/1
2	PC4 Fa0	Switch0 Fa0/2
3	PC1 Fa0	Switch0 Fa0/3
4	Switch0 Fa0/24	Router0 Gi0/0
5	Router0 Gi0/1	Router1 Gi0/1
6	Router1 Gi0/0	Switch1 Fa0/24
7	PC2 Fa0	Switch1 Fa0/1
8	PC3 Fa0	Switch1 Fa0/2

Langkah 5: Mengisi IP di PC. Klik PC, buka tab Desktop > IP Configuration, pilih Static, lalu isi IPv4 Address, Subnet Mask dan Default Gateway sesuai tabel pengalamatan.



Gambar 1.6 IP Configuration PC0

PC0 diberi IP 192.168.1.10, subnet mask 255.255.255.0 dan default gateway 192.168.1.1, yaitu IP Gi0/0 Router0. Langkah yang sama diulang untuk PC lain dengan IP masing-masing.

Langkah 6: Mengisi IP di router. Klik router, buka tab CLI, lalu ketik:

```
Router> enable
Router# configure terminal
Router(config)# hostname Router0
Router0(config)# interface GigabitEthernet0/0
Router0(config-if)# ip address 192.168.1.1 255.255.255.0
Router0(config-if)# no shutdown
Router0(config-if)# interface GigabitEthernet0/1
Router0(config-if)# ip address 192.168.3.1 255.255.255.0
Router0(config-if)# no shutdown
Router0(config-if)# end
```

Router1 dikonfigurasi dengan cara yang sama: Gi0/0 = 192.168.2.1, Gi0/1 = 192.168.3.2.

```
Router0>enable
Router0#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.3.1     YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
Router0#
```

```
Router1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.2.1     YES manual up                    up
GigabitEthernet0/1     192.168.3.2     YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
```

Gambar 1.7 Hasil show ip interface brief di Router0 & Router1

Kedua interface Gigabit sudah memiliki IP dan berstatus up/up, artinya kabel tersambung dan interface aktif.

Langkah 7: Mengaktifkan RIP. Masukkan konfigurasi router rip, lalu simpan dengan `write memory`.

```
!
interface GigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.3.1 255.255.255.0
 duplex auto
 speed auto
!
interface Vlan1
 no ip address
 shutdown
!
router rip
 version 2
 passive-interface GigabitEthernet0/0
 network 192.168.1.0
 network 192.168.3.0
 no auto-summary
!
```

Gambar 1.8 Running-config Router0: interface dan router rip

Terlihat IP kedua interface dan blok router rip: version 2, passive-interface Gi0/0, network 192.168.1.0 dan 192.168.3.0, serta no auto-summary.

```
!
interface GigabitEthernet0/0
 ip address 192.168.2.1 255.255.255.0
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.3.2 255.255.255.0
 duplex auto
 speed auto
!
interface Vlan1
 no ip address
 shutdown
!
router rip
 version 2
 passive-interface GigabitEthernet0/0
 network 192.168.2.0
 network 192.168.3.0
 no auto-summary
!
```

Gambar 1.9 Running-config Router1: interface dan router rip

Router1 mengaktifkan RIP pada jaringan 192.168.2.0 dan 192.168.3.0. Pertukaran informasi rute antar-router berlangsung melalui jaringan penghubung 192.168.3.0/24. Jaringan LAN tetap diiklankan kepada router tetangga meskipun interface menuju PC dibuat pasif.

1.4 Konfigurasi: dari route statis ke RIP v2

Route statis dihapus, lalu tiap router cukup mengumumkan jaringan yang terhubung langsung dengannya.

Sebelumnya (static routing):

```
Router0(config)# ip route 192.168.2.0 255.255.255.0 192.168.3.2
Router1(config)# ip route 192.168.1.0 255.255.255.0 192.168.3.1
```

Sesudahnya (RIP v2):

```
Router0(config)# no ip route 192.168.2.0 255.255.255.0 192.168.3.2
Router0(config)# router rip
Router0(config-router)# version 2
Router0(config-router)# no auto-summary
Router0(config-router)# network 192.168.1.0
Router0(config-router)# network 192.168.3.0
Router0(config-router)# passive-interface GigabitEthernet0/0

Router1(config)# no ip route 192.168.1.0 255.255.255.0 192.168.3.1
Router1(config)# router rip
Router1(config-router)# version 2
Router1(config-router)# no auto-summary
Router1(config-router)# network 192.168.2.0
Router1(config-router)# network 192.168.3.0
Router1(config-router)# passive-interface GigabitEthernet0/0
```

Perintah	Fungsi
`router rip`	Mengaktifkan proses RIP
`version 2`	Memakai RIPv2: membawa subnet mask, update dikirim multicast 224.0.0.9
`no auto-summary`	Mencegah rute diringkas ke batas kelas jaringan
`network`	Mengumumkan jaringan yang terhubung langsung ke router tetangga
`passive-interface Gi0/0`	Tidak mengirim update RIP ke arah LAN PC, tapi jaringannya tetap diumumkan

Rute statis ke tujuan yang sama dihapus agar hasil pengujian benar-benar menggunakan RIP. Secara default, administrative distance rute statis adalah 1, lebih kecil daripada RIP yang bernilai 120. Jika kedua rute menuju prefiks yang sama tersedia dan valid, rute statis lebih diutamakan.

1.5 Hasil pengujian RIP

Kedua router mempelajari LAN seberang secara otomatis dengan kode R, dan semua PC antar-LAN dapat berkomunikasi.

Pengujian dilakukan pada topologi Gambar 1.1. Ringkasan hasilnya:

Uji	Hasil
`show ip route` Router0	`R 192.168.2.0/24 [120/1] via 192.168.3.2`
`show ip route` Router1	`R 192.168.1.0/24 [120/1] via 192.168.3.1`
`show ip protocols` Router0	RIP v2, update tiap 30 s, invalid 180 s, hold down 180 s, flush 240 s, passive Gi0/0, AD 120
PC0 ping 192.168.2.10 (PC2)	3/4 berhasil, TTL=126 (1 timeout awal karena ARP)
PC0 ping 192.168.2.20 (PC3)	3/4 berhasil, TTL=126 (1 timeout awal karena ARP)
PC0 `tracert 192.168.2.20`	192.168.1.1, 192.168.3.2, 192.168.2.20
Router1 Gi0/0 di-`shutdown`	Router0 langsung menandai `192.168.2.0/24 is possibly down`
Router1 Gi0/0 di-`no shutdown`	Rute kembali sendiri: `R 192.168.2.0/24 [120/1] via`

	192.168.3.2

Notasi [120/1] menunjukkan administrative distance sebesar 120 dan metrik RIP sebesar 1 hop. Artinya, jaringan tujuan dipelajari melalui satu router tetangga. Nilai hop count digunakan RIP untuk memilih rute, sedangkan administrative distance menentukan prioritas sumber rute yang berbeda.

Tabel routing Router0 dan Router1

```
Router0>enable
Router0#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
R    192.168.2.0/24 [120/1] via 192.168.3.2, 00:00:07, GigabitEthernet0/1
     192.168.3.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.3.0/24 is directly connected, GigabitEthernet0/1
L       192.168.3.1/32 is directly connected, GigabitEthernet0/1

Router0#
```

Gambar 1.10 Tabel routing Router0 (show ip route)

Jaringan 192.168.1.0/24 dan 192.168.3.0/24 bertanda C karena terhubung langsung. Jaringan 192.168.2.0/24 bertanda R, artinya dipelajari dari RIP lewat 192.168.3.2 (Router1) dengan metrik 1 hop.

```
Gateway of last resort is not set

R    192.168.1.0/24 [120/1] via 192.168.3.1, 00:00:21, GigabitEthernet0/1
     192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.2.0/24 is directly connected, GigabitEthernet0/0
L       192.168.2.1/32 is directly connected, GigabitEthernet0/0
     192.168.3.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.3.0/24 is directly connected, GigabitEthernet0/1
L       192.168.3.2/32 is directly connected, GigabitEthernet0/1

Router1#
```

Gambar 1.11 Tabel routing Router1 (show ip route)

Sebaliknya, Router1 mempelajari 192.168.1.0/24 dari Router0 (via 192.168.3.1). Tidak ada satu pun route statis (S) di kedua router.

Parameter RIP di Router0

```
Router0#show ip protocols
Routing Protocol is "rip"
Sending updates every 30 seconds, next due in 18 seconds
Invalid after 180 seconds, hold down 180, flushed after 240
Outgoing update filter list for all interfaces is not set
Incoming update filter list for all interfaces is not set
Redistributing: rip
Default version control: send version 2, receive 2
  Interface             Send  Recv  Triggered RIP  Key-chain
  GigabitEthernet0/1    22
Automatic network summarization is not in effect
Maximum path: 4
Routing for Networks:
           192.168.1.0
           192.168.3.0
Passive Interface(s):
           GigabitEthernet0/0
Routing Information Sources:
           Gateway         Distance      Last Update
           192.168.3.2          120      00:00:10
Distance: (default is 120)
Router0#
```

Gambar 1.12 Parameter RIP di Router0 (show ip protocols)

RIP mengirim update tiap 30 detik, rute dianggap invalid setelah 180 detik tanpa update dan dihapus setelah 240 detik. Gi0/0 berstatus passive sehingga update tidak dikirim ke LAN PC. Sumber informasi rute adalah 192.168.3.2 dengan distance 120.

Uji perubahan jaringan: link LAN 2 dimatikan lalu dinyalakan

```
Router0#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
R    192.168.2.0/24 is possibly down, routing via 192.168.3.2, GigabitEthernet0/1
     192.168.3.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.3.0/24 is directly connected, GigabitEthernet0/1
L       192.168.3.1/32 is directly connected, GigabitEthernet0/1

Router0#
```

Gambar 1.13 Rute 192.168.2.0/24 ditandai possibly down setelah Gi0/0 Router1 dimatikan

Setelah Gi0/0 Router1 dinonaktifkan, Router0 menampilkan rute LAN 2 sebagai possibly down. Hasil ini menunjukkan bahwa informasi ketidaktersediaan jaringan diteruskan melalui RIP dan memengaruhi status rute pada router tetangga. Mekanisme triggered update memungkinkan pemberitahuan perubahan tanpa harus menunggu seluruh interval pembaruan berkala.

```
Router0#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
R    192.168.2.0/24 [120/1] via 192.168.3.2, 00:00:17, GigabitEthernet0/1
     192.168.3.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.3.0/24 is directly connected, GigabitEthernet0/1
L       192.168.3.1/32 is directly connected, GigabitEthernet0/1

Router0#
```

Gambar 1.14 Rute kembali otomatis setelah Gi0/0 Router1 dinyalakan

Setelah Gi0/0 Router1 diaktifkan kembali dengan no shutdown, rute R 192.168.2.0/24 [120/1] melalui 192.168.3.2 muncul kembali pada Router0. RIP mempelajari kembali keterjangkauan jaringan tersebut melalui pertukaran informasi rute. Rute statis tidak mempunyai mekanisme pertukaran informasi seperti ini, sehingga gangguan pada LAN jauh tidak otomatis diketahui hanya dari konfigurasi ip route.

Ping dan tracert dari PC0

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping -n 4 192.168.2.10

Pinging 192.168.2.10 with 32 bytes of data:

Request timed out.
Reply from 192.168.2.10: bytes=32 time=1ms TTL=126
Reply from 192.168.2.10: bytes=32 time<1ms TTL=126
Reply from 192.168.2.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.2.10:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>ping -n 4 192.168.2.20

Pinging 192.168.2.20 with 32 bytes of data:

Request timed out.
Reply from 192.168.2.20: bytes=32 time<1ms TTL=126
Reply from 192.168.2.20: bytes=32 time=1ms TTL=126
Reply from 192.168.2.20: bytes=32 time=1ms TTL=126

Ping statistics for 192.168.2.20:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\>tracert 192.168.2.20

Tracing route to 192.168.2.20 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.1.1
  2    0 ms      0 ms      0 ms      192.168.3.2
  3    0 ms      0 ms      1 ms      192.168.2.20

Trace complete.

C:\>
```

Gambar 1.15 Ping dan tracert dari PC0 ke LAN 2

Ping dari PC0 menuju PC2 dan PC3 menghasilkan balasan dengan TTL 126. Pada simulasi ini, nilai tersebut konsisten dengan lintasan melalui dua router. Timeout pada percobaan awal sesuai dengan kemungkinan proses resolusi ARP, sehingga pengujian perlu diulang setelah tabel ARP terisi. Tracert menuju PC3 memperlihatkan urutan 192.168.1.1, 192.168.3.2, kemudian alamat PC tujuan.

Pengujian tambahan menggunakan PDU ICMP pada mode simulasi

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num
	Successful	PC0	Router0	ICMP		0.000	N	0
	Successful	PC2	Router1	ICMP		0.000	N	1
	Successful	PC0	Router0	ICMP		0.000	N	10
	Successful	PC0	Router1	ICMP		0.000	N	11
	Successful	PC0	Router1	ICMP		0.000	N	12
	Successful	PC4	Router1	ICMP		0.000	N	7 (edit)
	Successful	PC1	Router1	ICMP		0.000	N	8 (edit)
	Successful	PC3	Router0	ICMP		0.000	N	9 (edit)

Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit
	Successful	PC0	Router1	ICMP		0.000	N	2	(edit)
	Successful	PC0	Router1	ICMP		0.000	N	3	(edit)
	Successful	PC0	PC2	ICMP		0.000	N	4	(edit)
	Successful	PC0	Router1	ICMP		0.000	N	5	(edit)
	Successful	PC2	Router0	ICMP		0.000	N	6	(edit)

Gambar di atas memperlihatkan daftar PDU ICMP dengan status Successful untuk beberapa pasangan sumber dan tujuan, antara lain PC0 ke Router0, PC2 ke Router1, PC0 ke Router1, PC0 ke PC2, PC4 ke Router1, serta PC3 ke Router0. Pengujian ke router pada LAN yang sama memeriksa akses lokal, sedangkan pengujian ke router atau PC di LAN seberang memeriksa penerusan paket antarjaringan. Keberhasilan pada pasangan yang ditampilkan mendukung hasil ping dan tracert bahwa pengalamatan, default gateway, dan rute RIP telah bekerja. Kolom Time(sec) bernilai 0.000 pada daftar tersebut tidak digunakan sebagai pengukuran latensi jaringan. Bukti pembentukan rute secara dinamis tetap diperoleh dari kode R pada show ip route dan perubahan status rute saat interface dinonaktifkan lalu diaktifkan kembali.

1.6 Analisa perbedaan static routing dan RIP

Pada kondisi normal, rute statis dan RIP sama-sama memungkinkan komunikasi antar-LAN apabila pengalamatannya benar. Perbedaannya terletak pada sumber informasi rute. Rute statis ditentukan administrator, sedangkan RIP mempelajari jaringan melalui pertukaran informasi dengan router tetangga dan memperbaruinya ketika terjadi perubahan keterjangkauan.

Aspek	Static routing (tugas sebelumnya)	RIP v2 (praktikum ini)
Cara rute dibuat	Admin menulis `ip route` per jaringan tujuan	Router bertukar informasi rute secara otomatis
Kode di tabel routing	S	R
Administrative distance	1	120
Metrik	Tidak menggunakan metrik hop count RIP	Hop count, maksimal 15 (16 = tidak terjangkau)
Yang ditulis admin	Jaringan tujuan + next hop	Hanya jaringan yang terhubung langsung (`network`)
Saat link putus	Tidak menerima pemberitahuan gangguan LAN jauh. Rute dapat hilang jika interface atau next hop lokal tidak lagi dapat digunakan	Memperbarui status rute berdasarkan informasi RIP dan timer. Pada uji ini muncul possibly down
Saat link pulih	Konfigurasi tetap ada, pemakaian rute bergantung pada keterjangkauan next hop	Rute LAN 2 dipelajari kembali pada pengujian subbab 1.5
Menambah jaringan baru	Semua router harus diberi `ip route` baru	Cukup `network` di router yang memiliki jaringan itu
Beban jaringan	Tidak ada lalu lintas routing	Update tiap 30 detik ke 224.0.0.9
Keamanan	Tidak bertukar update rute. Keamanan tetap bergantung pada kontrol akses dan konfigurasi	Passive interface mengurangi pengiriman update ke LAN PC, tetapi bukan pengganti autentikasi dan kontrol akses
Cocok untuk	Jaringan kecil dan jarang berubah	Jaringan yang berkembang atau punya jalur cadangan

Pada topologi kecil dengan dua router, rute statis relatif sederhana dan tidak menambah trafik pembaruan protokol routing. RIP mengurangi kebutuhan mengubah rute tujuan secara manual ketika topologi berubah, tetapi memiliki batas metrik 15 hop dan memerlukan waktu konvergensi. Oleh karena itu, RIP sesuai sebagai sarana mempelajari routing dinamis, bukan pilihan universal untuk semua ukuran jaringan.

1.7 Kesimpulan BAB 1

1. RIP v2 berhasil menggantikan routing statis: Router0 dan Router1 saling mempelajari LAN seberang (kode R, metrik 1 hop) tanpa `ip route` manual.
2. Komunikasi antar-LAN tetap berjalan seperti saat static routing (ping TTL=126, tracert lewat 192.168.1.1 dan 192.168.3.2).
3. Perbedaan terlihat ketika LAN 2 dinonaktifkan dan diaktifkan kembali. RIP memperbarui status rute berdasarkan informasi tetangga, sedangkan rute statis tidak bertukar informasi keterjangkauan jaringan jauh.
4. Route statis harus dihapus saat beralih ke RIP, karena AD-nya (1) lebih diutamakan daripada RIP (120).

BAB 2 Analisis Server Iteratif dan Server Berbasis Poll

Bab ini membahas pengujian s1_server_iteratif.c dan s2_server_poll.c pada Ubuntu WSL. Pengujian dilakukan dalam dua bagian. Bagian manual memakai dua klien Netcat, perekaman loopback melalui tcpdump, dan analisis berkas capture pada Wireshark Windows. Bagian lanjutan memakai klien penguji Python (uji_klien.py) dan skrip otomatis (run_uji.sh) untuk sebelas skenario tambahan yang direkam ke berkas capture baru. Hasilnya menunjukkan bahwa S1 menunda pelayanan klien berikutnya sampai klien aktif keluar, sedangkan S2 dapat melayani koneksi lain ketika salah satu klien tetap terhubung dan diam. Eksperimen lanjutan juga memperlihatkan flow control TCP (TCP ZeroWindow) ketika S1 berhenti membaca, antrean backlog yang penuh pada 17 koneksi sehingga 8 klien gagal tersambung ke S1, framing baris dan batas 32 klien pada S2, serta kelemahan send() blocking pada S2 yang membekukan seluruh ruang chat selama lebih dari 5 detik ketika satu klien tidak membaca.

Pembahasan bab ini mencakup prosedur yang dapat diulang, bukti terminal dan paket, alur source code, pemetaan baris kode dengan bukti eksperimen, keterbatasan implementasi, lampiran kode lengkap kedua program, serta lampiran program penguji dan skrip otomatis.

2.1 Tujuan dan ruang lingkup

Tujuan pengujian adalah menjelaskan hubungan antara rancangan loop server dan pelayanan banyak koneksi. Bukti diperoleh dengan mengamati apakah klien B mendapat respons ketika klien A masih terhubung, lalu mencocokkannya dengan accept(), recv(), send(), dan poll() pada source code.

S1 adalah aplikasi echo: byte yang diterima dikirim kembali kepada pengirim. S2 adalah aplikasi chat: baris pertama menjadi nama pengguna, sedangkan pesan selanjutnya disiarkan kepada pengguna terdaftar lain. Karena fungsi aplikasinya berbeda, jumlah paket dan waktu respons keduanya tidak diperlakukan sebagai benchmark yang setara.

2.2 Dasar teori dan rancangan percobaan

2.2.1 TCP dan koneksi loopback

Kedua program membuat socket IPv4 bertipe SOCK_STREAM sehingga menggunakan TCP. Server melakukan socket(), setsockopt(), bind(), dan listen() pada 127.0.0.1 port 5000. Alamat loopback membuat komunikasi tetap berada dalam lingkungan jaringan lokal system. Wi-Fi tidak diperlukan setelah alat pengujian tersedia.

TCP membentuk koneksi melalui SYN, SYN ACK, dan ACK. Setiap koneksi dibedakan oleh pasangan alamat dan port sumber serta tujuan. Pada loopback, alamat kedua sisi sama, tetapi port sementara klien berbeda. TCP merupakan aliran byte: batas send(), segmen TCP, dan recv() tidak selalu berkorespondensi satu banding satu.

ACK menyatakan kemajuan penerimaan byte TCP pada sisi penerima. ACK tidak membuktikan bahwa aplikasi sudah membaca, mengolah, atau membalas pesan. Hal ini menjadi penting pada S1: koneksi B dapat terbentuk dan pesannya diakui kernel ketika aplikasi masih sibuk menunggu A.

2.2.2 Model iteratif dan multiplexing

Server iteratif menyelesaikan pelayanan satu klien sebelum berpindah ke klien berikutnya. Pada S1, loop recv() berlangsung hingga klien menutup koneksi atau terjadi kegagalan. Klien yang terhubung tetapi diam dapat menahan alur program di recv().

S2 menggunakan multiplexing kesiapan I/O. poll() memantau listener dan semua socket klien dalam satu proses dan satu thread. Saat ada socket siap, program memproses kejadiannya lalu kembali memantau. Banyak koneksi dapat dilayani bergantian tanpa membuat thread untuk setiap klien.

2.2.3 Skenario dan bukti

Skenario	Tindakan	Bukti yang diamati
S1 dua klien	A tetap terhubung. B mengirim. A kemudian keluar	Echo B tertunda hingga A keluar
S2 chat	Aprilia dan cristyana saling mengirim pesan	Pesan diteruskan melalui koneksi penerima
S2 klien diam	Aprilia tetap terhubung. cristyana mengirim /who	Server membalas saat Aprilia masih aktif
S2 validasi	Koneksi tambahan mencoba nama Aprilia	Nama ditolak dan koneksi ditutup
S2 keluar	Aprilia mengirim /quit lalu cristyana /who	Notifikasi keluar dan daftar pengguna berubah
Lanjutan s1-dua	Klien otomatis A dan B. ss merekam antrean kernel	Recv-Q listener 1 dan Recv-Q B 12 byte
Lanjutan s1-pecah	Satu baris dikirim dalam 3 segmen ke S1	S1 memantulkan 3 potongan terpisah
Lanjutan s1-banjir	A diam, B mengirim blok 64 KB selama 5 detik	TCP Window Full, ZeroWindow, probe
Lanjutan s1-backlog	A menahan S1, 25 koneksi baru serentak	17 antre, 8 klien mengulang SYN
Lanjutan s2-chat	Klien diam tanpa nama dan tiga pengguna chat	Broadcast 1 ke N-1, klien diam tidak menghambat
Lanjutan s2-nama	Nama berisi spasi, nama 25 karakter, nama ganda	Pesan !!! lalu FIN dari server
Lanjutan s2-pecah dan s2-gabung	1 baris dalam 3 segmen. 3 baris dalam 1 segmen	rakit_baris menggabung dan memecah
Lanjutan s2-panjang	2100 byte tanpa newline	Baris terlalu panjang lalu RST
Lanjutan s2-penuh	33 koneksi serentak	Koneksi ke-33 menerima server penuh
Lanjutan s2-lambat	Satu pengguna tidak pernah membaca saat chat ramai	Seluruh server membeku karena send blocking

Baris skenario berawalan Lanjutan adalah eksperimen tambahan yang direkam ulang secara otomatis. Skenario manual menjawab pertanyaan utama, yaitu apakah klien kedua dilayani ketika klien pertama masih aktif. Skenario lanjutan menguji perilaku di bawah beban, batas buffer, batas jumlah klien, dan kondisi kegagalan yang tidak mudah dibuat dengan mengetik manual di Netcat.

2.2.4 Antrean backlog, flow control, dan framing

Three-way handshake TCP ditangani oleh kernel. Koneksi yang telah terbentuk dapat menunggu pada antrean accept sampai aplikasi memanggil accept(). Parameter listen(ls, 16) meminta ukuran backlog kepada sistem operasi. Pada eksperimen Linux dalam laporan ini, ss menunjukkan 17 koneksi mengantre dengan nilai backlog 16. Hasil tersebut merupakan perilaku pada lingkungan uji, bukan batas yang berlaku sama pada seluruh sistem operasi. Ketika kapasitas antrean tidak lagi mencukupi, sebagian upaya koneksi tidak memperoleh respons dan klien mengulang SYN.

Data TCP yang telah diterima kernel tetapi belum dibaca aplikasi menempati receive buffer. Ketika ruang penerimaan habis, penerima dapat mengiklankan window nol atau TCP ZeroWindow sehingga pengirim menunda pengiriman data baru. Pengirim dapat mengirim zero-window probe untuk memeriksa apakah ruang sudah tersedia. Pada capture tertentu, probe tersebut diberi label Keep-Alive oleh Wireshark, sehingga interpretasinya harus dikaitkan dengan window dan urutan paket. Sebaliknya, ketika peer tidak membaca dan buffer kirim penuh, send() pada socket blocking dapat menunggu.

TCP adalah aliran byte. Satu send() dapat tiba sebagai beberapa recv(), dan beberapa send() dapat tiba dalam satu recv(). Protokol berbasis baris seperti S2 karena itu memerlukan buffer per klien dan pemisah baris (framing). S1 tidak memiliki framing karena hanya memantulkan byte apa adanya.

2.3 Persiapan lingkungan

Komponen	Konfigurasi
Host	Windows
Lingkungan eksekusi	Ubuntu pada WSL
Bahasa dan compiler	C dan GCC
Klien	Netcat OpenBSD melalui perintah nc
Perekaman	tcpdump pada interface lo
Analisis	Wireshark Windows membaca pcap
Folder Windows	D:\S2\Jaringan\ANALISIS SERVER
Folder Ubuntu	/mnt/d/S2/Jaringan/ANALISIS SERVER
Endpoint server	127.0.0.1 TCP port 5000

Empat tab digunakan agar proses tidak tertukar: tab 1 SERVER, tab 2 REKAM, tab 3 KLIEN A, dan tab 4 KLIEN B. Semua tab memakai distro Ubuntu yang sama. Tab baru yang masih berupa PowerShell perlu masuk ke Ubuntu dengan perintah berikut. Jika prompt sudah berupa pengguna@host dengan tanda $, perintah wsl tidak perlu dijalankan lagi.

```
wsl -d Ubuntu
```

Pada terminal Ubuntu, buka folder sumber dan siapkan alat. Password sudo adalah password pengguna Ubuntu. karakter password tidak terlihat ketika diketik.

```
cd "/mnt/d/S2/Jaringan/ANALISIS SERVER"
sudo apt update
sudo apt install build-essential netcat-openbsd tcpdump
```

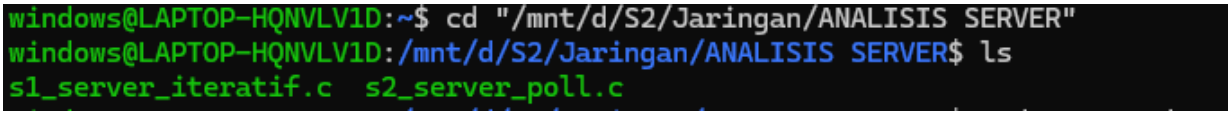
Saat koneksi repositori sempat menunggu, pengunduhan dicoba melalui IPv4 dengan opsi berikut. Opsi ini merupakan langkah pemulihan koneksi pada percobaan, bukan persyaratan protokol server.

```
sudo apt -o Acquire::ForceIPv4=true update
sudo apt -o Acquire::ForceIPv4=true install build-essential netcat-openbsd tcpdump
```

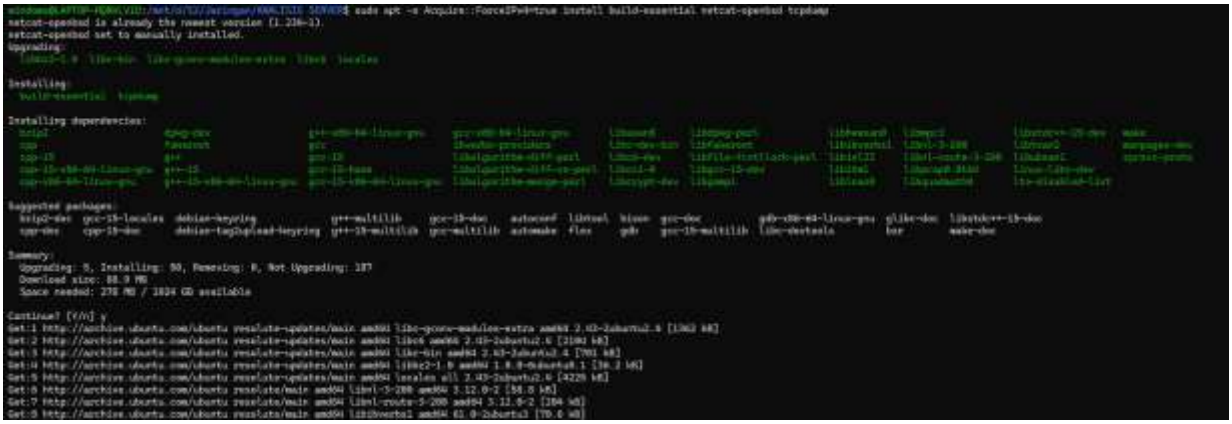
Kompilasi dilakukan satu kali pada folder sumber. -Wall dan -Wextra mengaktifkan banyak peringatan compiler, -O2 mengaktifkan optimasi, dan -o menentukan nama executable.

```
gcc -Wall -Wextra -O2 -o s1_server s1_server_iteratif.c
gcc -Wall -Wextra -O2 -o s2_server s2_server_poll.c
```

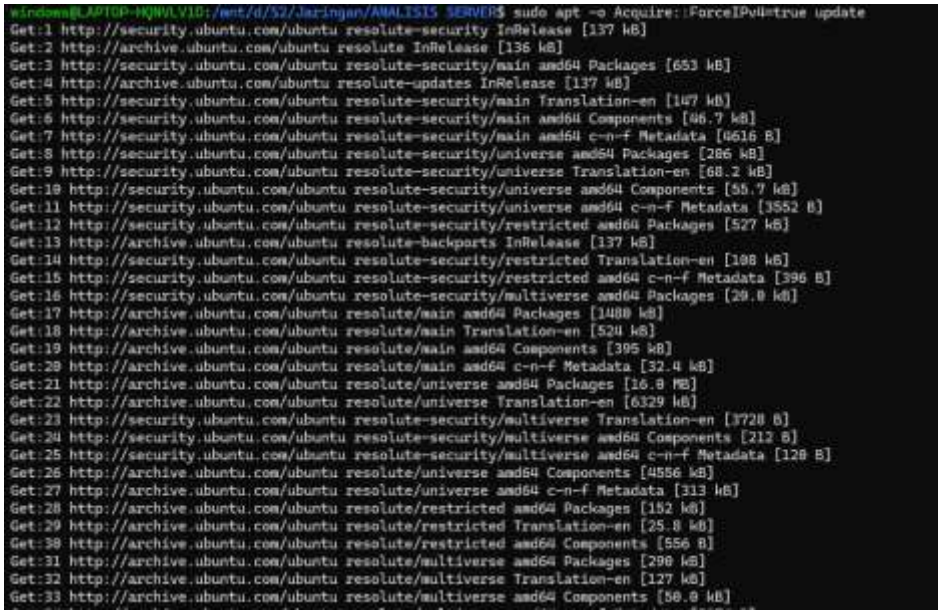
Prompt yang kembali tanpa pesan error menunjukkan kompilasi selesai. Instalasi serta kompilasi tidak perlu diulang ketika hanya mengulang percobaan. Hentikan S1 sebelum menjalankan S2 pada port yang sama agar tidak terjadi kegagalan bind karena alamat sedang dipakai.



Gambar 2.1 Pemeriksaan lokasi source code pada folder percobaan



Gambar 2.2 Persiapan instalasi compiler dan alat perekaman



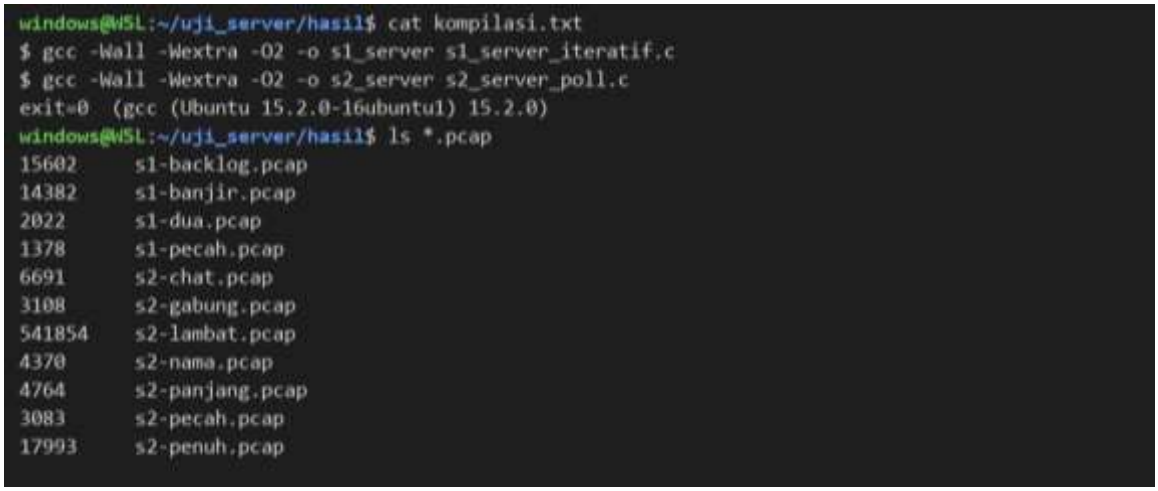
Gambar 2.3 Proses pembaruan daftar paket Ubuntu sebelum instalasi

2.3.1 Perangkat eksperimen lanjutan

Eksperimen lanjutan dijalankan dengan dua berkas baru di folder uji_server. uji_klien.py adalah klien penguji yang hanya memakai pustaka standar Python dan berisi sebelas skenario (Lampiran C). run_uji.sh mengompilasi kedua server, menjalankan tcpdump pada interface lo, menjalankan server sebagai pengguna biasa, menjalankan klien penguji, mengambil keluaran ss pada saat kritis, lalu menghentikan server dan tcpdump (Lampiran D). Setiap skenario menghasilkan berkas .pcap, log server, dan log klien.

```
wsl -d Ubuntu -u root -- bash "/mnt/d/S2/Jaringan/ANALISIS SERVER/uji_server/run_uji.sh"
wsl -d Ubuntu -u root -- bash ".../uji_server/run_uji.sh" s1-banjir s2-lambat
```

Server dan klien dijalankan melalui runuser sebagai pengguna windows. Kompilasi ulang dengan -Wall dan -Wextra tidak menghasilkan peringatan pada pengujian ini. Pada s1-banjir dan s2-lambat, tcpdump menggunakan snaplen 200 byte untuk membatasi ukuran capture. Header TCP tetap dapat diperiksa, tetapi isi payload yang melebihi batas perekaman tidak tersedia seluruhnya pada berkas tersebut.



Gambar 2.4 Kompilasi ulang tanpa warning dan daftar berkas capture baru

Ringkasan berkas capture baru:

Berkas	Server	Paket	Koneksi	Inti pengamatan
s1-dua.pcap	S1	23	2	Echo B tertunda sampai A keluar
s1-pecah.pcap	S1	16	1	Tiga potongan dipantulkan terpisah
s1-banjir.pcap	S1	102	2	Window Full, ZeroWindow, probe
s1-backlog.pcap	S1	181	26	18 SYN,ACK, 48 SYN diulang
s2-chat.pcap	S2	74	4	Broadcast dan klien diam
s2-nama.pcap	S2	48	4	Tiga penolakan nama
s2-pecah.pcap	S2	34	2	3 segmen menjadi 1 baris
s2-gabung.pcap	S2	34	2	1 segmen menjadi 3 pesan
s2-panjang.pcap	S2	29	2	Baris 2100 byte ditolak
s2-penuh.pcap	S2	200	33	Koneksi ke-33 ditolak
s2-lambat.pcap	S2	3658	4	Server membeku 5,2 detik

Wireshark dibuka dengan kolom tambahan agar bukti mudah dibaca: No., Time (relatif), Src Port, Dst Port, Flags (tcp.flags.str), Len (tcp.len), Win (tcp.window_size), Payload (teks) dari data.text, dan Info. Opsi Edit > Preferences > Protocols > Data > Show data as text diaktifkan sehingga payload ASCII tampil langsung. Filter tampilan yang dipakai:

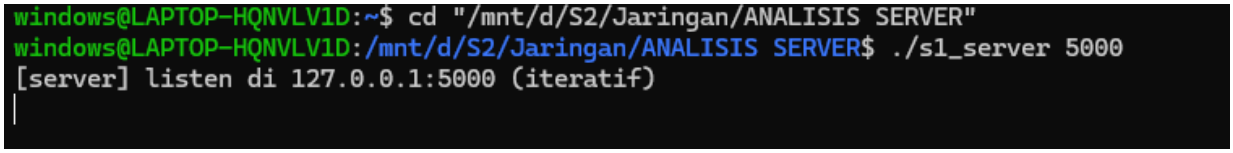
Filter tampilan	Kegunaan
tcp.port == 5000	Semua trafik server
tcp.len > 0	Hanya segmen yang membawa data aplikasi
tcp.flags.syn==1 && tcp.flags.ack==1	SYN,ACK: koneksi yang diterima kernel
tcp.analysis.retransmission	SYN yang diulang karena antrean penuh
tcp.analysis.flags	Penanda analisis: ZeroWindow, Window Full, Keep-Alive, dan lainnya
tcp.analysis.zero_window	Pihak yang menyuruh lawan berhenti mengirim
tcp.flags.fin==1 || tcp.flags.reset==1	Pihak yang menutup koneksi dan caranya
tcp.port == <port klien>	Mengisolasi satu koneksi

2.4 Pengujian dan analisis S1

2.4.1 Prosedur percobaan

Pada tab SERVER, jalankan program berikut dan biarkan tetap aktif. Pesan listen menandakan server siap menerima koneksi. terminal yang tidak kembali ke prompt sedang menjalankan proses server.

```
./s1_server 5000
```



Gambar 2.5 S1 mendengarkan pada 127 0 0 1 port 5000

Pada tab REKAM, jalankan perintah berikut sebelum menghubungkan klien. -i lo memilih loopback. -nn menonaktifkan resolusi nama dan nama layanan. -s 0 memakai snapshot length maksimum yang didukung. -w menulis paket ke file. Filter tcp port 5000 membatasi capture pada trafik pengujian. Tunggu tulisan listening on lo.

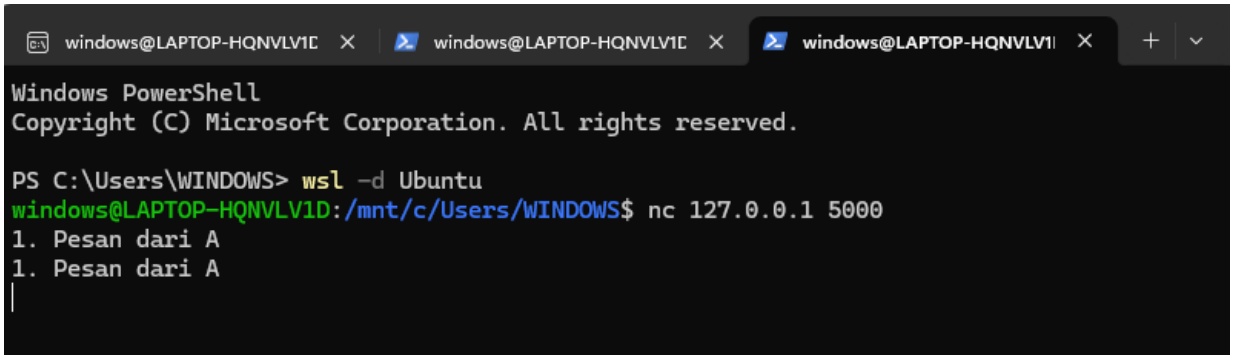
```
sudo tcpdump -i lo -nn -s 0 -w "/mnt/d/S2/Jaringan/ANALISIS SERVER/s1_iteratif.pcap" "tcp port 5000"
```



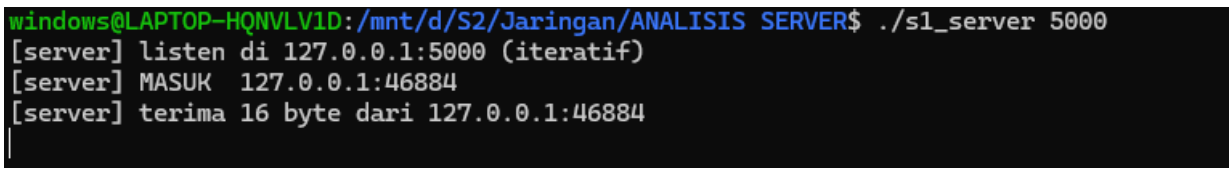
Gambar 2.6 Perekaman trafik S1 melalui tcpdump

Pada tab KLIEN A, jalankan nc 127.0.0.1 5000. Kirim teks “1. Pesan dari A” lalu Enter. Teks pada terminal muncul dua kali, yaitu ketikan lokal dan echo dari server. Biarkan A terhubung setelah menerima echo. Capture aktual menunjukkan payload A berukuran 16 byte termasuk newline.

```
nc 127.0.0.1 5000
```

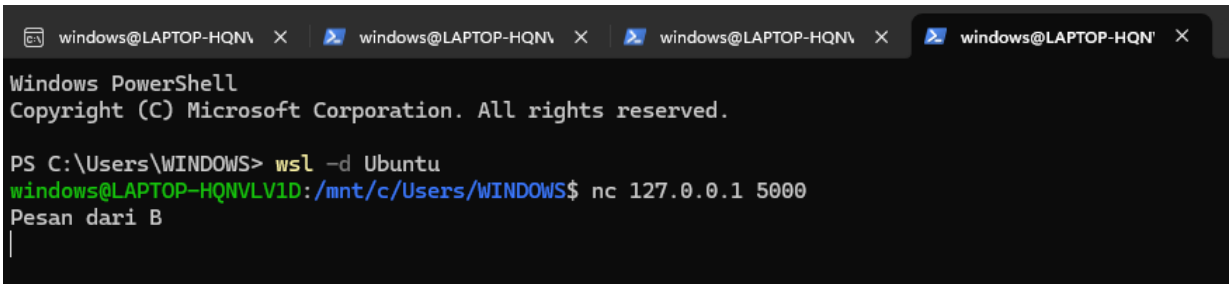


Gambar 2.7 Klien A memperoleh echo dan tetap terhubung

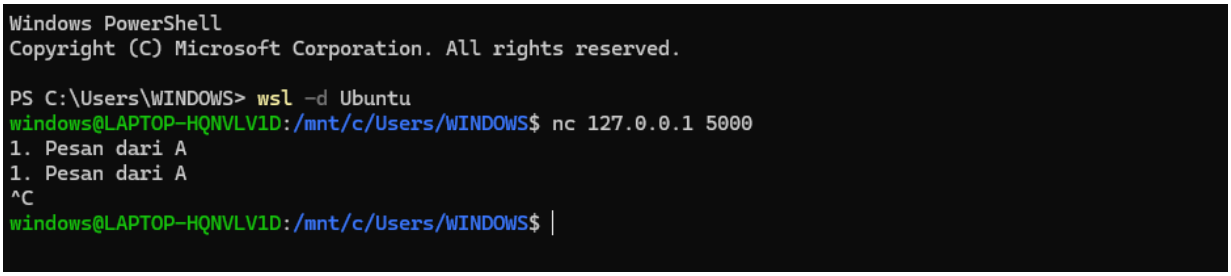


Gambar 2.8 Log server ketika menerima koneksi dan data dari A

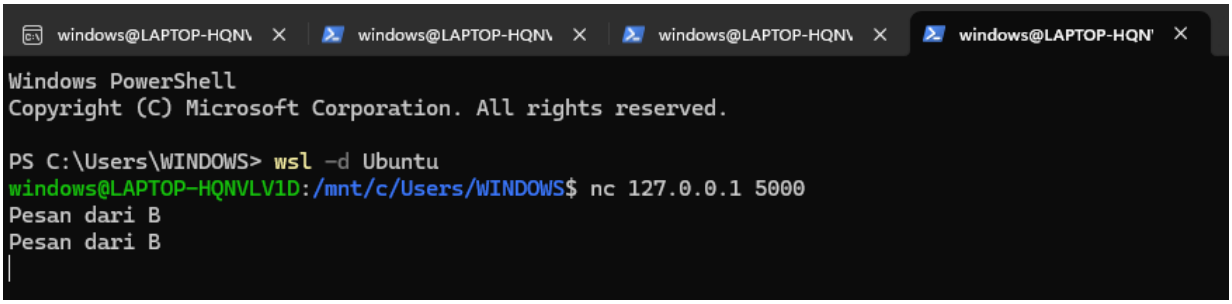
Pada tab KLIEN B, jalankan perintah nc yang sama dan kirim “Pesan dari B” lalu Enter. Jangan tutup A dahulu. B menampilkan ketikan lokal, tetapi belum menerima echo. Setelah mendokumentasikan keadaan tersebut, tekan Ctrl+C pada tab A. Kembali ke B dan amati echo yang baru muncul. Jeda ini bergantung pada berapa lama A dibiarkan terhubung.



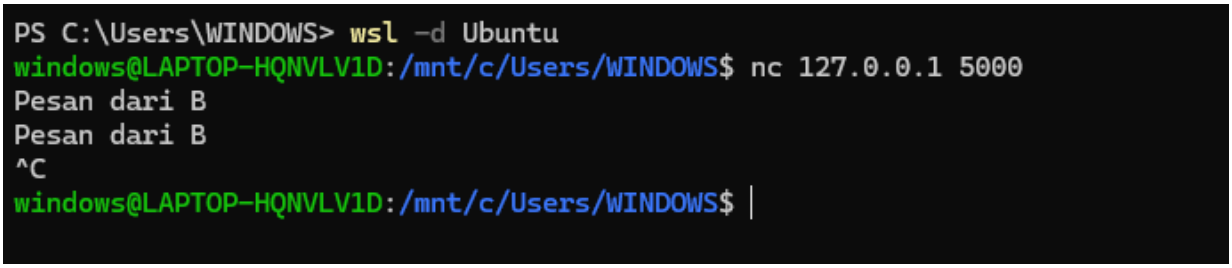
Gambar 2.9 Klien B belum mendapat echo ketika A masih terhubung



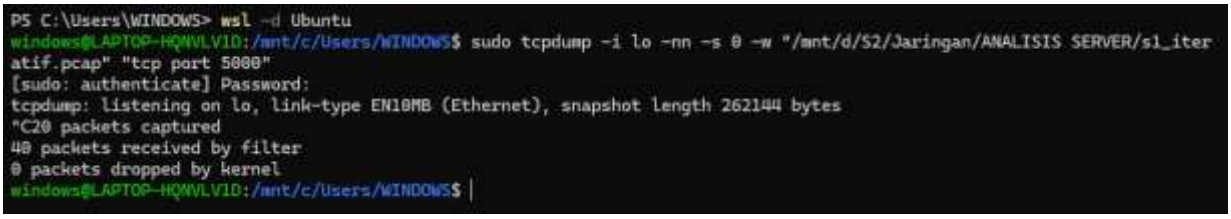
Gambar 2.10 Klien A diputus untuk memberi giliran pelayanan kepada B



Gambar 2.11 Klien B mendapat echo setelah A diputus

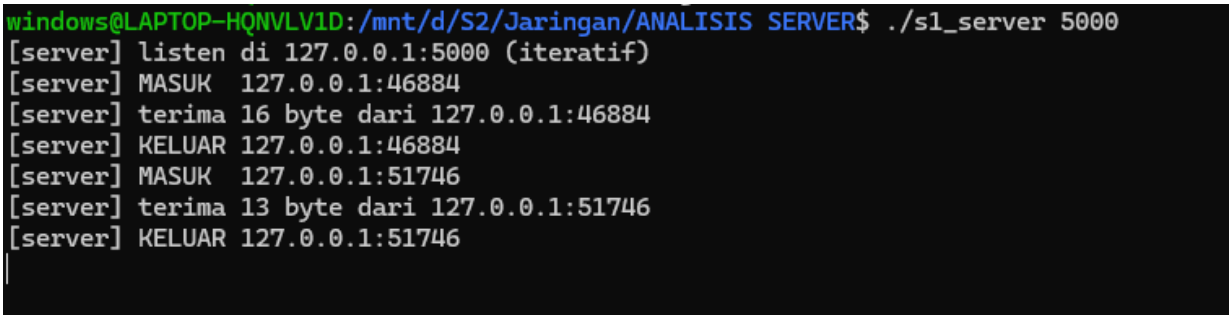


Gambar 2.12 Klien B dihentikan setelah echo diterima



Gambar 2.13 Perekaman S1 dihentikan dan statistik capture ditampilkan

Setelah echo B diterima, tekan Ctrl+C pada B, lalu Ctrl+C pada tab REKAM untuk menutup file capture. Simpan screenshot log SERVER dan hentikan server dengan Ctrl+C. Menutup capture setelah klien keluar memungkinkan paket penutupan koneksi ikut terekam.



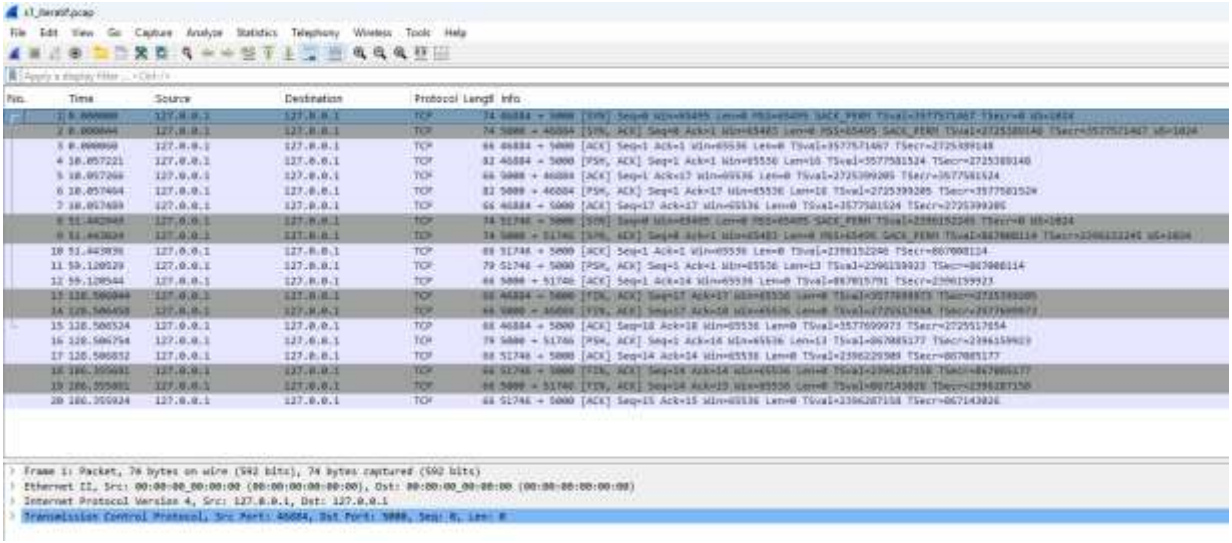
Gambar 2.14 Log S1 menunjukkan A keluar sebelum B diterima aplikasi

2.4.2 Membuka dan membaca capture

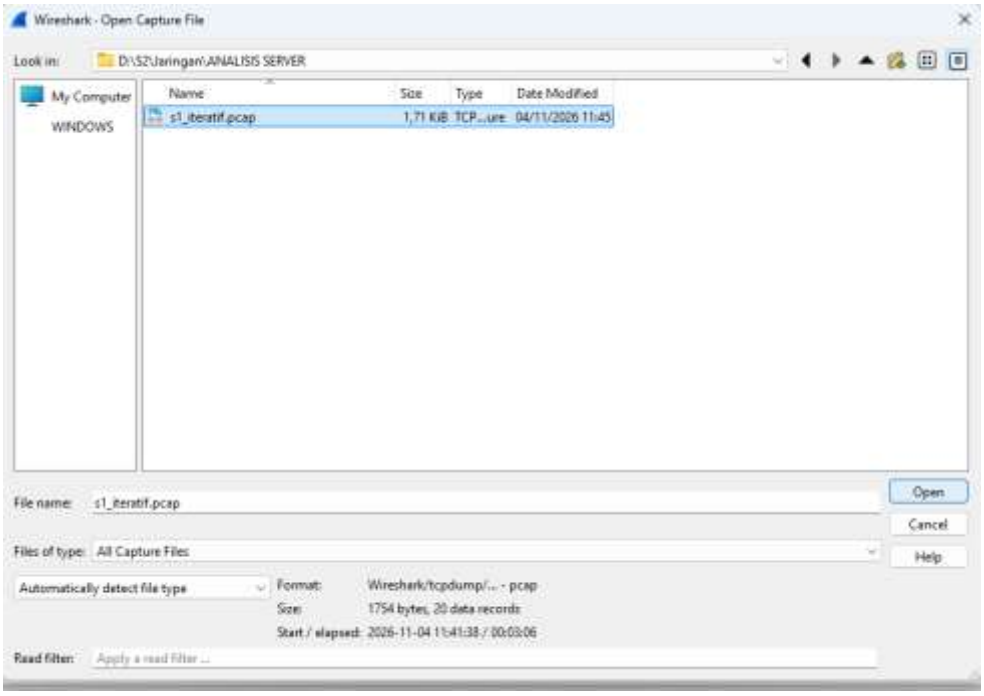
Pada Wireshark Windows pilih File kemudian Open dan buka s1_iteratif.pcap dari folder percobaan. Tidak perlu memulai live capture di Wireshark karena perekaman sudah dilakukan tcpdump di Ubuntu. Masukkan display filter tcp.port == 5000 lalu Enter. Klik kanan paket dan pilih Follow kemudian TCP Stream untuk membaca percakapan satu koneksi.

```
tcp.port == 5000
tcp.port == 5000 && tcp.len > 0
```

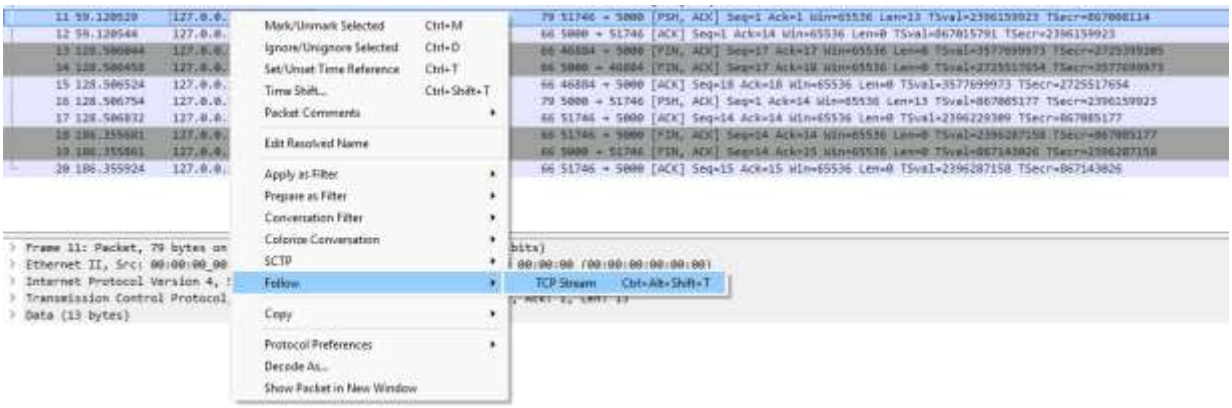
Filter kedua hanya menampilkan segmen yang memuat payload. Port server adalah 5000, sedangkan port A adalah 46884 dan port B adalah 51746. Kolom Length menyatakan panjang frame capture. TCP Len menyatakan panjang data TCP, sehingga kedua angka tidak boleh disamakan.



Gambar 2.15 Daftar paket capture S1



Gambar 2.16 Pemilihan file capture S1 melalui dialog Open Wireshark



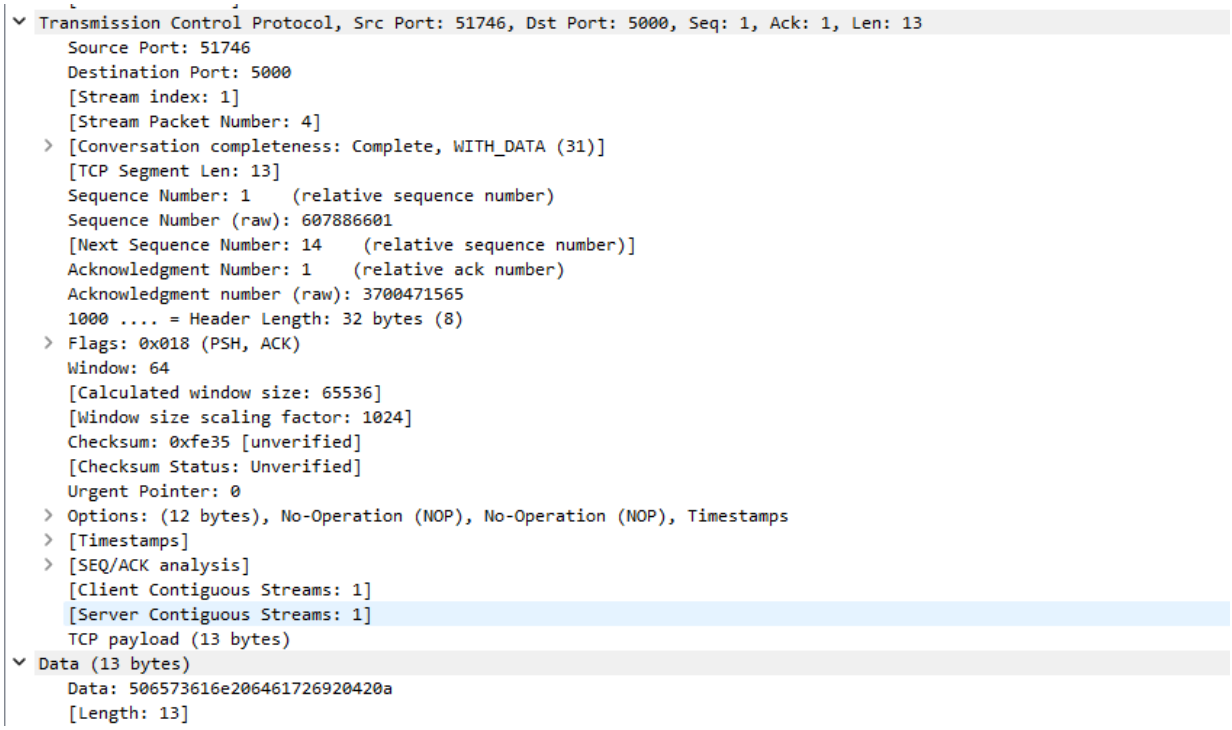
Gambar 2.17 Menu Follow TCP Stream untuk membuka percakapan B

Paket	Waktu detik	Kejadian
1 sampai 3	0,000000 sampai 0,000602	Handshake koneksi A
4	10,057221	A mengirim 16 byte
6	10,057464	Server mengirim echo 16 byte kepada A
8 sampai 10	51,442949 sampai 51,443036	Handshake koneksi B
11	59,120529	B mengirim 13 byte
12	59,120544	ACK TCP terhadap data B
13 sampai 15	128,506044 sampai 128,506524	Penutupan koneksi A
16	128,506754	Server mengirim echo 13 byte kepada B
18 sampai 20	186,355681 sampai 186,355924	Penutupan koneksi B

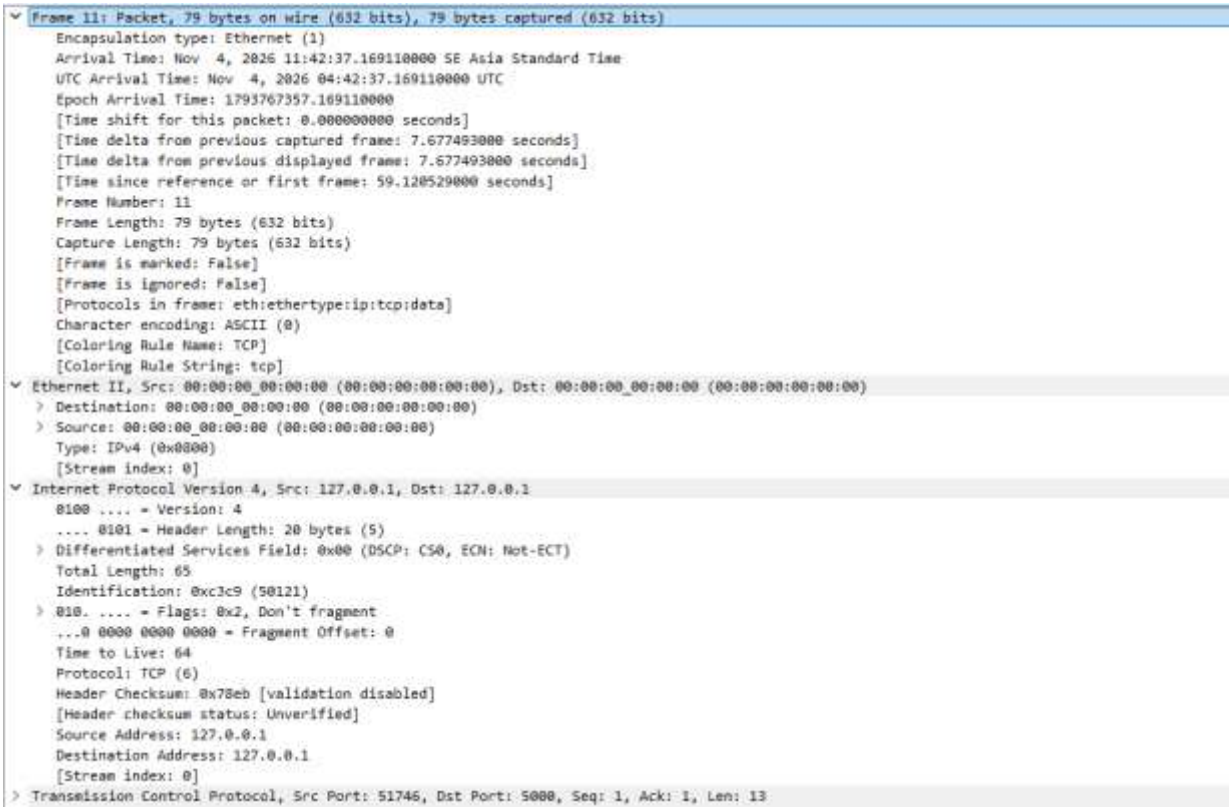
2.4.3 Detail pesan dan echo B

Field	Paket 11	Paket 16
Arah port	51746 ke 5000	5000 ke 51746
TCP stream	1	1
Sequence relatif	1	1
ACK relatif	1	14
Next Sequence	14	14
Flag	PSH ACK	PSH ACK
Payload	Pesan dari B diikuti newline	Pesan dari B diikuti newline
TCP payload	13 byte	13 byte
Panjang frame	79 byte	79 byte

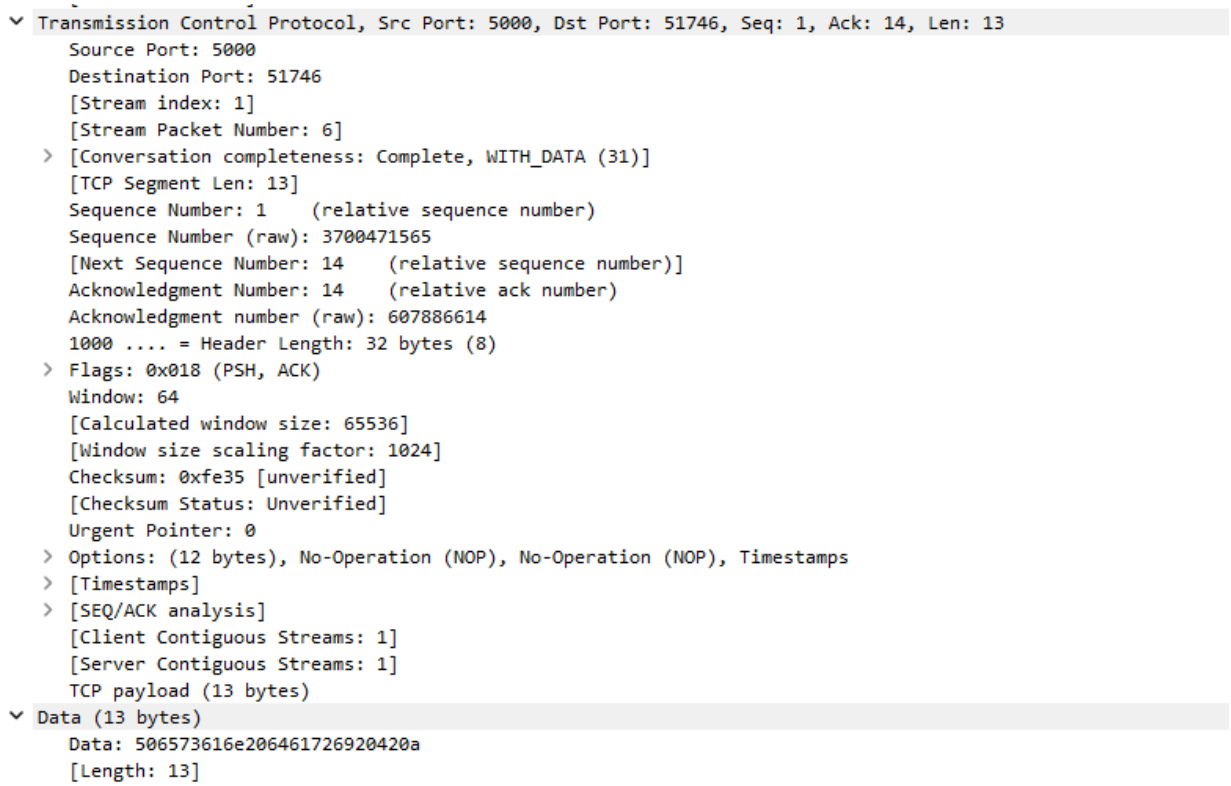
Payload heksadesimal 506573616e206461726920420a menyatakan “Pesan dari B” diikuti 0a atau newline. Next Sequence bernilai 14 karena data dimulai dari sequence relatif 1 dan membawa 13 byte. Nomor sequence pada dua arah adalah ruang penomoran terpisah. kesamaan angka 1 pada kedua arah bukan berarti satu penomoran global.



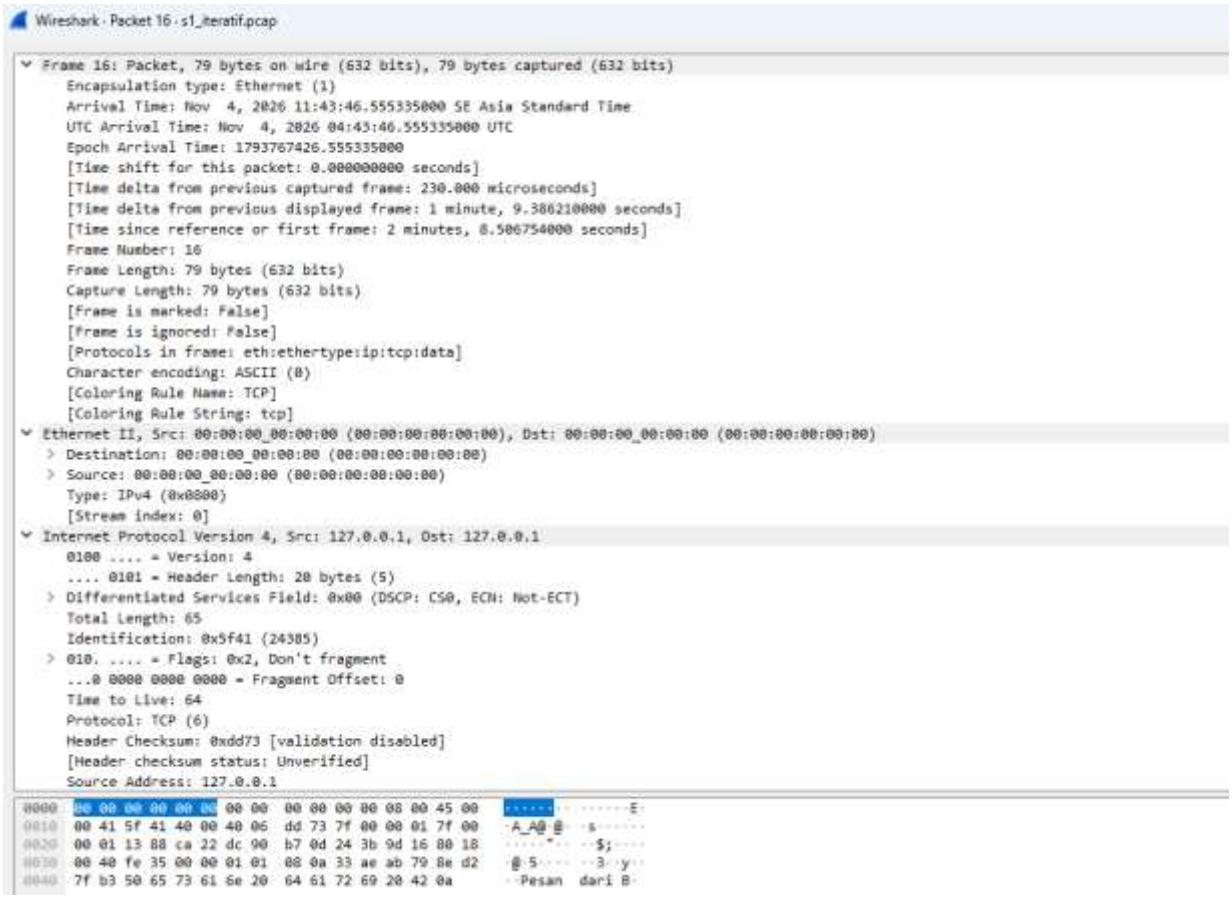
Gambar 2.18 Detail TCP paket 11 dari B kepada server



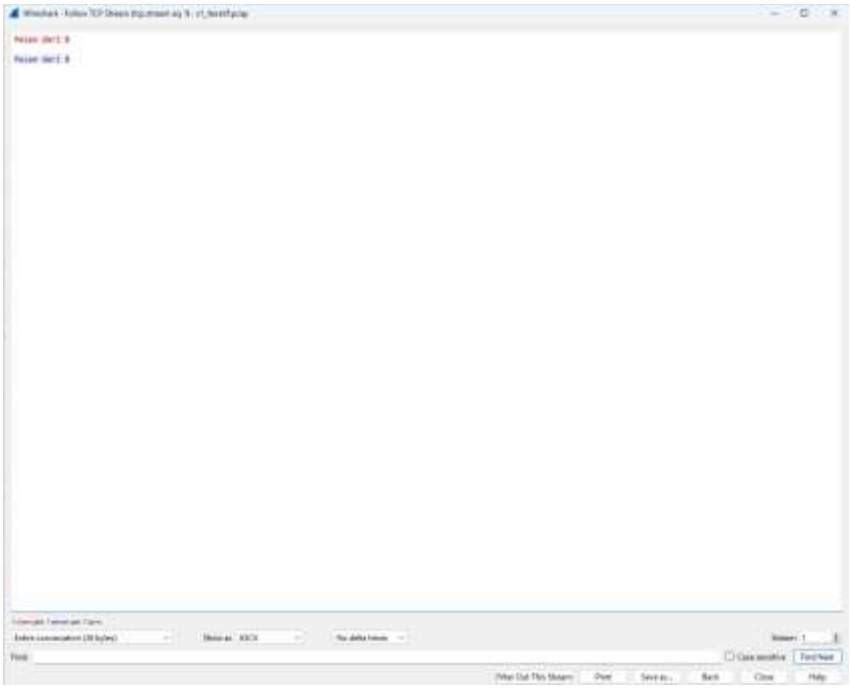
Gambar 2.19 Detail frame dan IPv4 pada paket 11 sebagai konteks pesan B



Gambar 2.20 Detail TCP paket 16 yang membawa echo kepada B



Gambar 2.21 Detail frame dan timestamp paket 16 untuk menghitung waktu tunggu



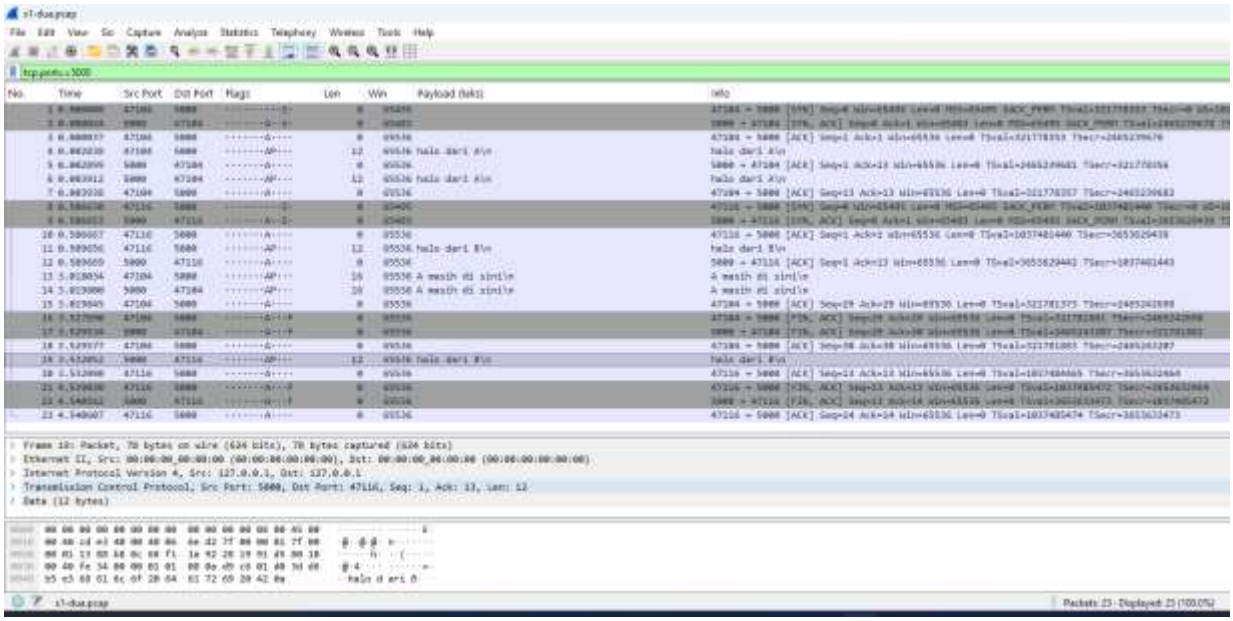
Gambar 2.22 Follow TCP Stream koneksi B menunjukkan isi pesan dan echo

Selisih pesan dan echo B dihitung dari timestamp dua paket pada capture yang sama: 128,506754 − 59,120529 = 69,386225 detik. Sebagai pembanding deskriptif, echo A pada paket 4 dan 6 berjarak 0,000243 detik atau 0,243 ms. Lamanya B menunggu terutama dipengaruhi tindakan pengguna mempertahankan koneksi A. angka 69 detik bukan waktu CPU yang dibutuhkan server untuk menyalin pesan.

Paket 12 membuktikan kernel telah mengakui data B, tetapi echo aplikasinya baru muncul pada paket 16 setelah penutupan A. Ini konsisten dengan loop S1 yang belum kembali ke accept() saat A masih dilayani. Handshake B tetap bisa selesai karena kernel mengelola antrean koneksi sebelum aplikasi menerima socket tersebut.

2.4.4 Eksperimen lanjutan S1 dua klien dan antrean kernel

Skenario s1-dua mengulang percobaan manual secara otomatis dengan jeda yang terkendali. Klien A (port 47104) tersambung dan mengirim halo dari A. Setengah detik kemudian klien B (port 47116) tersambung dan mengirim halo dari B. A tetap terhubung, mengirim satu baris lagi pada detik ke-3, lalu menutup koneksi.



Gambar 2.23 s1-dua.pcap filter tcp.port==5000, data B frame 11 baru di-echo pada frame 19 setelah FIN A

Frame	Waktu (s)	Arah	Flag	Len	Keterangan
1-3	0,000000	A dan S	SYN, SYN ACK, ACK	0	Handshake A
4 ke 6	0,002838 ke 0,003912	A ke S ke A	PSH ACK	12	Echo A 1,07 ms
8-10	0,506630	B dan S	SYN, SYN ACK, ACK	0	Handshake B sukses oleh kernel
11	0,509656	B ke S	PSH ACK	12	Data B tiba
12	0,509669	S ke B	ACK	0	Hanya ACK kernel, tanpa echo
13-14	3,018034	A dan S	PSH ACK	16	A masih dilayani
16-18	3,527890	A dan S	FIN	0	A menutup koneksi
19	3,532052	S ke B	PSH ACK	12	Echo B baru keluar
21-23	4,539030	B dan S	FIN	0	B menutup koneksi

```
windows@WSL:~/uji_server/hasil$ cat s1-dua.server.log
[server] listen di 127.0.0.1:5000 (iteratif)
[server] MASUK  127.0.0.1:47104
[server] terima 12 byte dari 127.0.0.1:47104
[server] terima 16 byte dari 127.0.0.1:47104
[server] KELUAR 127.0.0.1:47104
[server] MASUK  127.0.0.1:47116
[server] terima 12 byte dari 127.0.0.1:47116
[server] KELUAR 127.0.0.1:47116
windows@WSL:~/uji_server/hasil$ cat s1-dua.ss.txt     # diambil saat B menunggu
# ss -ltn  (baris LISTEN: Recv-Q = koneksi antre belum di-accept)
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 1      16         127.0.0.1:5000      0.0.0.0:*

# ss -tn  (Recv-Q = byte belum dibaca aplikasi, Send-Q = byte belum di-ACK peer)
State Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB 0      0          127.0.0.1:5000    127.0.0.1:47104
ESTAB 12     0          127.0.0.1:5000    127.0.0.1:47116
windows@WSL:~/uji_server/hasil$ cat s1-dua.klien.log
  0.025s [A] terhubung dari port lokal 47104
  0.027s [A] >> 'halo dari A\n'
  0.530s [A] << halo dari A
  0.532s [B] terhubung dari port lokal 47116
  0.533s [B] >> 'halo dari B\n'
  3.038s [B] << (tidak ada balasan)
  3.041s [A] >> 'A masih di sini\n'
  3.546s [A] << A masih di sini
  3.551s [A] menutup koneksi (FIN)
  4.559s [B] << halo dari B
  4.562s [B] echo B diterima +4.028s setelah dikirim
```

Gambar 2.24 Log server, keluaran ss saat B menunggu, dan log klien s1-dua

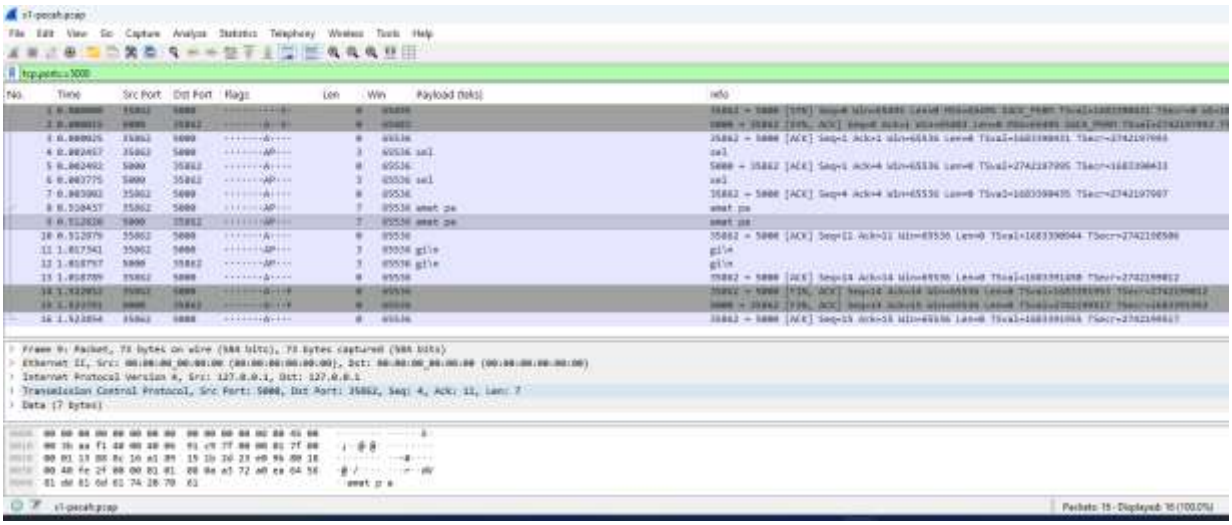
Data B tiba pada frame 11, sedangkan echo baru dikirim pada frame 19. Selisih waktunya adalah 3,532052 - 0,509656 = 3,022396 detik. Echo A pada percobaan yang sama berjarak sekitar 1,07 ms. Echo B muncul sekitar 4,2 ms setelah FIN dari A pada frame 16, sehingga penundaan B konsisten dengan alur S1 yang belum menerima dan membaca koneksi B selama sesi A berlangsung.

Keluaran ss yang diambil saat B menunggu menunjukkan baris LISTEN dengan Recv-Q 1, artinya satu koneksi (B) sudah ESTABLISHED tetapi belum diambil oleh accept(), dan socket B memiliki Recv-Q 12, yaitu 12 byte halo dari B yang sudah di-ACK kernel tetapi belum dibaca aplikasi.

Log server menegaskan urutan: MASUK B baru tercetak setelah KELUAR A. Hasil otomatis ini konsisten dengan percobaan manual pada subbab 2.4.2 dan 2.4.3 yang mencatat penundaan 69,386 detik. perbedaan angka hanya berasal dari berapa lama A dibiarkan terhubung.

2.4.5 Eksperimen lanjutan S1 pesan terpecah tanpa framing

Skenario s1-pecah mengirim satu kalimat selamat pagi dalam tiga send() terpisah dengan jeda 0,5 detik: sel, amat pa, dan gi diikuti newline. Skenario yang sama kemudian dikirim ke S2 (subbab 2.5.8) sebagai pembanding.

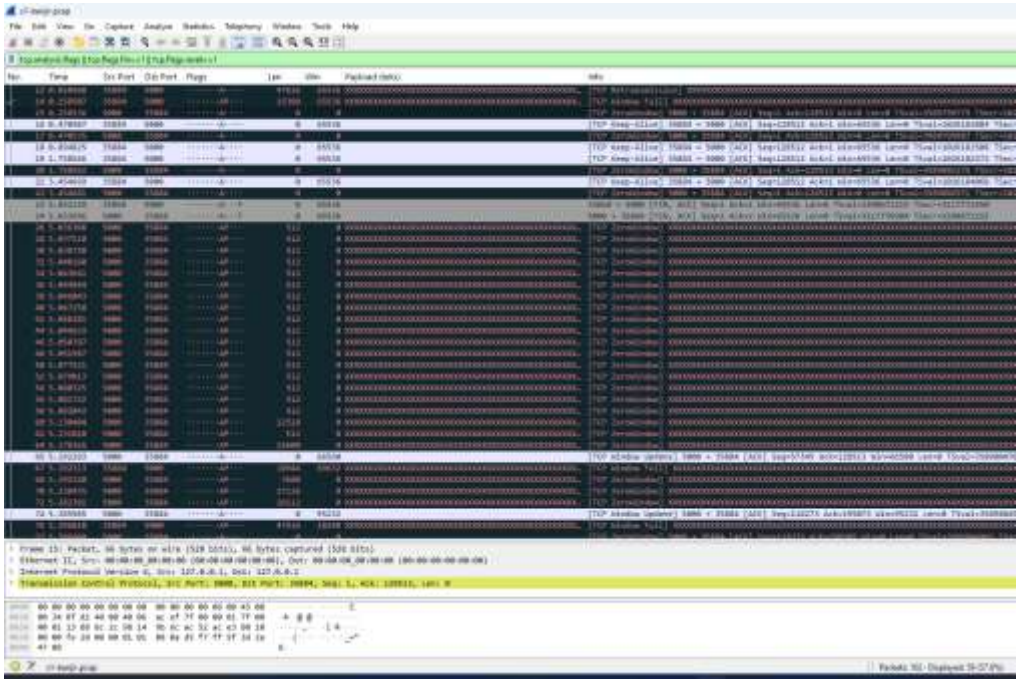


Gambar 2.25 s1-pecah.pcap, S1 memantulkan tiga potongan sebagai tiga segmen terpisah (frame 6, 9, 12)

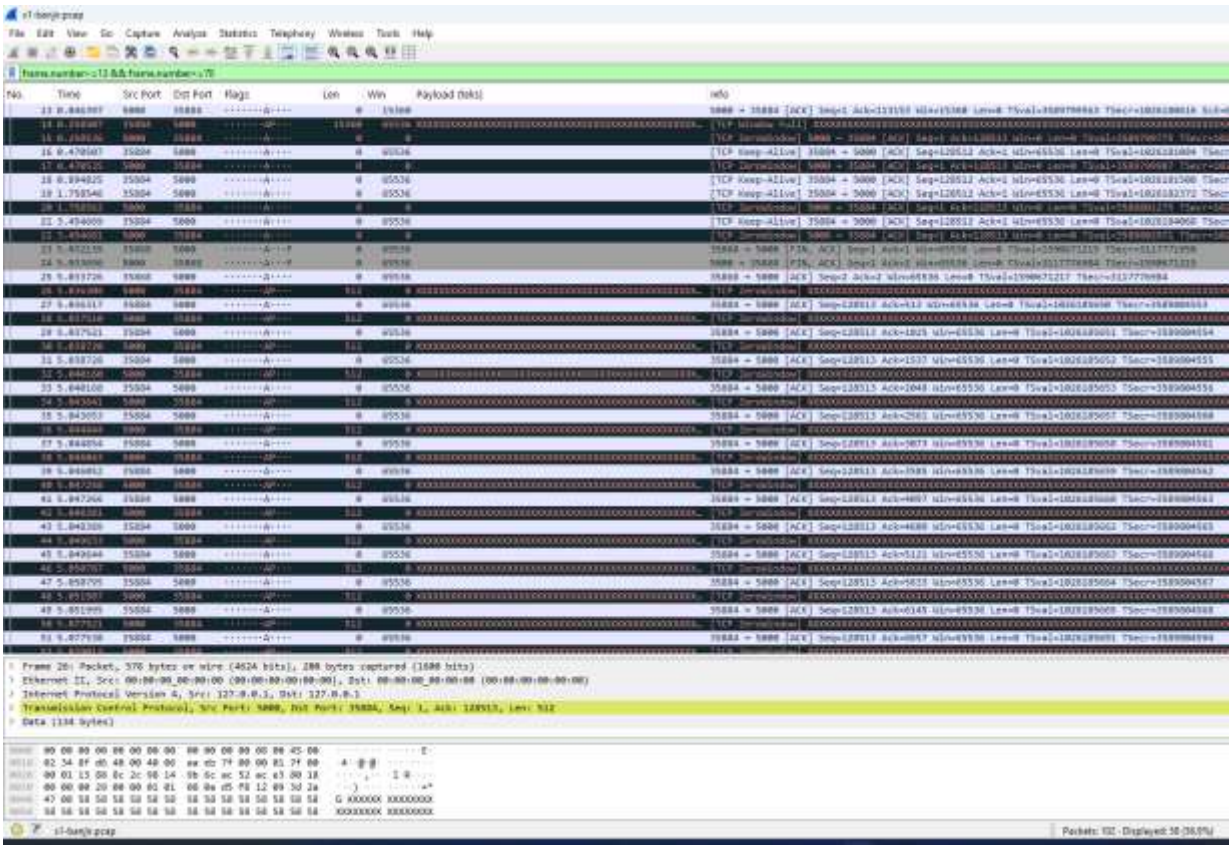
Pada percobaan ini, tiga potongan langsung dipantulkan melalui pasangan frame 4 dan 6, 8 dan 9, serta 11 dan 12. Log server mencatat penerimaan 3, 7, dan 3 byte. Hasil ini sesuai dengan S1 yang mengirim kembali byte dari setiap recv() tanpa menunggu newline. Namun, batas segmen yang terlihat tidak menjamin bahwa setiap send() selalu menjadi satu segmen atau satu recv(), karena TCP tetap merupakan aliran byte.

2.4.6 Eksperimen lanjutan S1 banjir data dan flow control

Skenario s1-banjir menguji apa yang terjadi pada klien kedua yang terus mengirim data ketika S1 masih terjebak melayani klien pertama. A (port 35868) tersambung lalu diam selama 5 detik. B (port 35884) mengirim blok 64 KB secara non-blocking tanpa henti, kemudian A menutup koneksi dan B menutup satu detik kemudian.



Gambar 2.26 s1-banjir.pcap filter tcp.analysis.flags || FIN || RST, Window Full pada frame 14 diikuti ZeroWindow dan probe



Gambar 2.27 s1-banjir.pcap frame 13-70, setelah FIN A (frame 23) S1 mulai membaca B dan echo keluar per 512 byte

Frame	Waktu (s)	Arah	Kejadian
7-11	0,0026	B ke S	B mengirim sekitar 113 KB dalam beberapa segmen besar
14	0,258507	B ke S	TCP Window Full, Len 15360: receive buffer server untuk B penuh
15	0,258536	S ke B	TCP ZeroWindow, Win 0, Ack 128513
16, 18, 19, 21	0,47, 0,89, 1,76, 3,45	B ke S	Zero-window probe (ditandai Keep-Alive) dengan jeda makin panjang
17, 20, 22	0,47, 1,76, 3,45	S ke B	Kernel server tetap menjawab Win 0
23-25	5,032139	A dan S	A menutup koneksi
26-58	5,036 sampai 5,083	S ke B	S1 akhirnya membaca B, echo 512 byte per segmen
65	5,192293	S ke B	TCP Window Update, Win 66560: B boleh mengirim lagi
66-100	5,19 sampai 6,0	B dan S	Siklus Window Full, ZeroWindow, Window Update berulang
102	6,04	B ke S	RST: B menutup socket yang masih berisi echo belum dibaca

```
windows@WSL:~/uji_server/hasil$ cat s1-banjir.klien.log
  0.025s [A] terhubung dari port lokal 35868
  0.027s [B] terhubung dari port lokal 35884
  5.053s [B] berhasil menitip 2720256 byte ke kernel sebelum macet (lihat TCP ZeroWindow)
  5.056s [A] menutup koneksi
windows@WSL:~/uji_server/hasil$ cat s1-banjir.ss.txt
# ss -ltn  (baris LISTEN: Recv-Q = koneksi antre belum di-accept)
State  Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 1      16         127.0.0.1:5000      0.0.0.0:*

# ss -tn  (Recv-Q = byte belum dibaca aplikasi, Send-Q = byte belum di-ACK peer)
State Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB 128512 0          127.0.0.1:5000    127.0.0.1:35884
ESTAB 0      0          127.0.0.1:5000    127.0.0.1:35868
windows@WSL:~/uji_server/hasil$ grep -c 'terima 512' s1-banjir.server.log
1606
windows@WSL:~/uji_server/hasil$ head -3 s1-banjir.server.log; tail -3 s1-banjir.server.log
[server] listen di 127.0.0.1:5000 (iteratif)
[server] MASUK  127.0.0.1:35868
[server] KELUAR 127.0.0.1:35868
send: Connection reset by peer
[server] KELUAR 127.0.0.1:35884
```

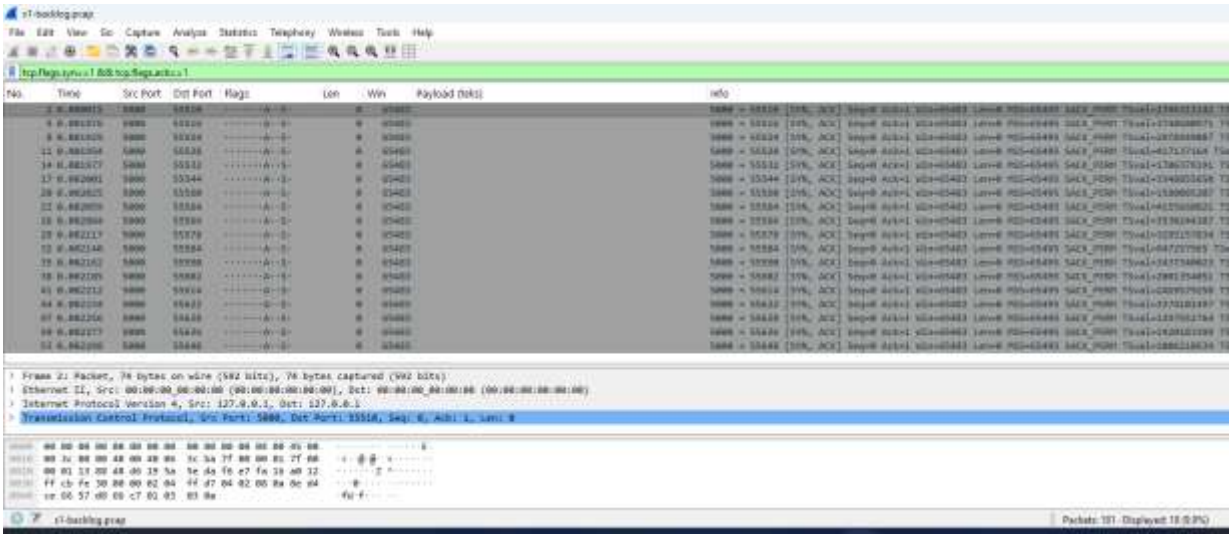
Gambar 2.28 Log klien, ss, dan log server s1-banjir

Dari sekitar detik 0,258 hingga 5,036, kemajuan transfer pada koneksi B tertahan selama sekitar 4,78 detik. Pada saat itu S1 masih menunggu data A sehingga belum membaca data B. Keluaran ss menunjukkan Recv-Q B sebesar 128512 byte dan satu koneksi menunggu pada antrean listener. Capture menampilkan ZeroWindow, Window Full, dan probe dengan jeda yang meningkat. Keadaan ini menunjukkan flow control akibat data yang belum dibaca aplikasi, bukan bukti bahwa koneksi fisik mengalami gangguan.

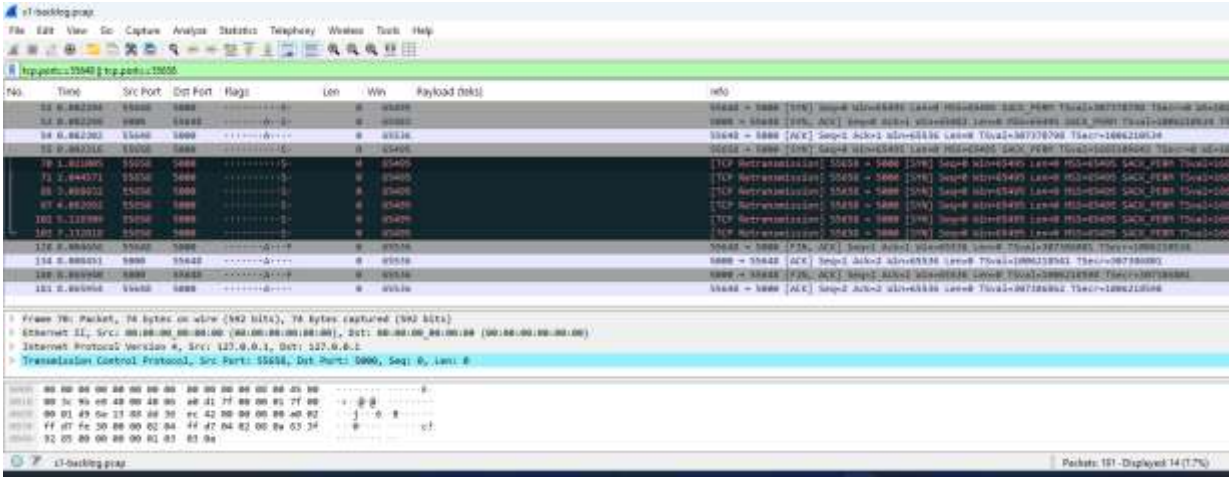
Buffer buf[512] membatasi satu recv() S1 paling banyak 512 byte. Pada capture ini, echo awal terlihat dalam segmen 512 byte dan log memuat 1606 penerimaan sebesar 512 byte. Jumlah 2.720.256 byte yang dilaporkan klien menyatakan data yang berhasil diterima operasi pengiriman klien, bukan seluruhnya telah diproses server. Ketika B menutup koneksi dengan RST, send() S1 pada pengujian ini gagal dengan pesan Connection reset by peer dan program melanjutkan loop pelayanan. Ukuran segmen besar pada loopback dapat dipengaruhi MTU serta mekanisme offloading, sehingga tidak dapat disamakan dengan ukuran frame pada jaringan Ethernet fisik.

2.4.7 Eksperimen lanjutan S1 antrean backlog penuh

Skenario s1-backlog menahan S1 dengan klien A (port 55510) yang diam, lalu membuka 25 koneksi non-blocking sekaligus dan menahannya selama 8 detik.

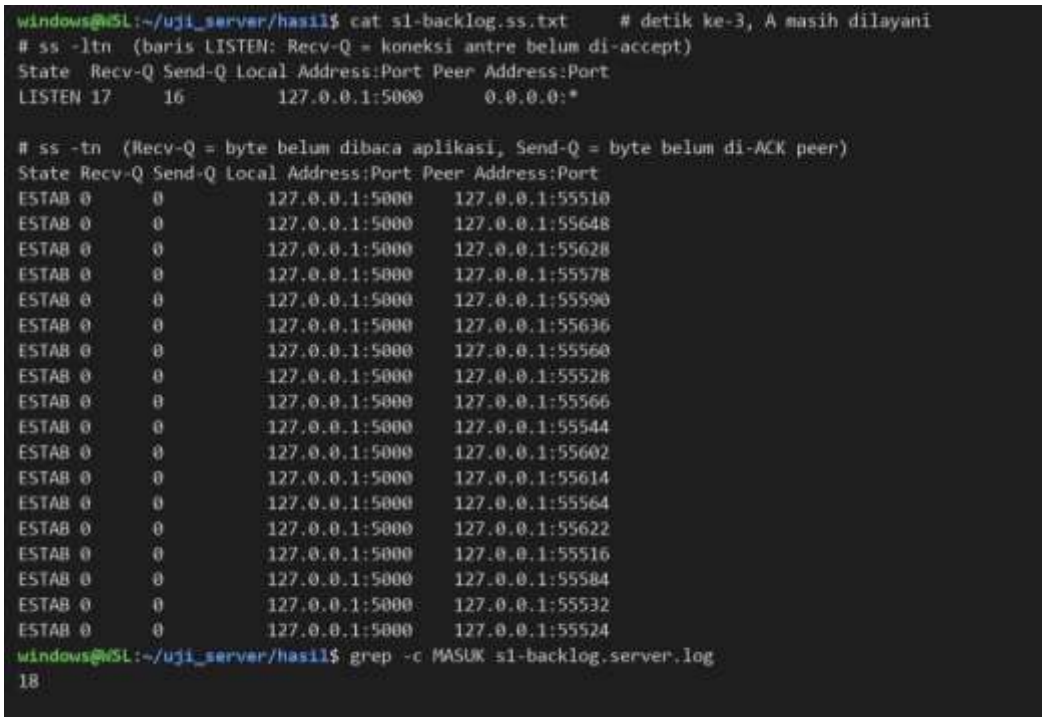


Gambar 2.29 s1-backlog.pcap filter SYN,ACK, hanya 18 koneksi (A + 17 antre) yang dijawab kernel



Gambar 2.30 s1-backlog.pcap port 55648 masih diterima, port 55658 tidak pernah dijawab dan mengulang SYN

Pengamatan	Nilai
Port klien unik yang mengirim SYN	26 (A + 25)
SYN,ACK yang dikirim server	18 (A + 17 antre)
Klien yang tidak pernah tersambung	8 (port 55658 sampai 55710)
SYN ulang (tcp.analysis.retransmission)	48 = 8 klien x 6 pengulangan
Waktu SYN port 55658	0,002, 1,021, 2,045, 3,069, 4,093, 5,117, 7,133 s
ss -ltn pada detik ke-3	LISTEN Recv-Q 17, Send-Q 16
Log server setelah A keluar	17 klien antre di-accept satu per satu (18 MASUK)



Gambar 2.31 ss -ltn menunjukkan antrean accept penuh 17 dari backlog 16

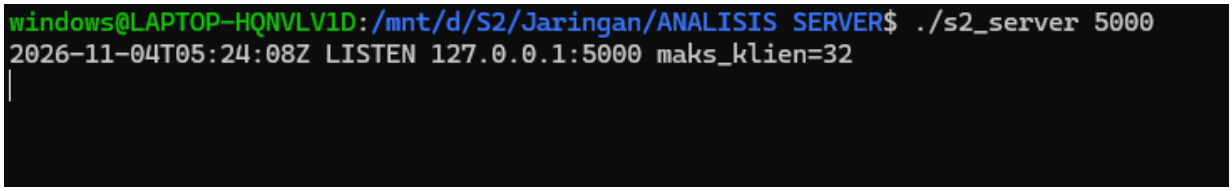
Pada lingkungan Linux yang diuji, ss memperlihatkan Recv-Q listener sebesar 17 dengan backlog 16. Capture mencatat 18 SYN ACK, yang terdiri atas satu koneksi A yang telah diterima aplikasi dan 17 koneksi antre. Delapan klien lain tidak berhasil menyelesaikan koneksi selama interval pengamatan dan mengulang SYN. Hasil ini konsisten dengan S1 yang belum kembali ke accept() selama A masih aktif. Koneksi yang telah mengantre baru diterima aplikasi setelah A menutup koneksi, sehingga angka kapasitas tersebut harus dipahami sebagai hasil eksperimen pada konfigurasi ini.

2.5 Pengujian dan analisis S2

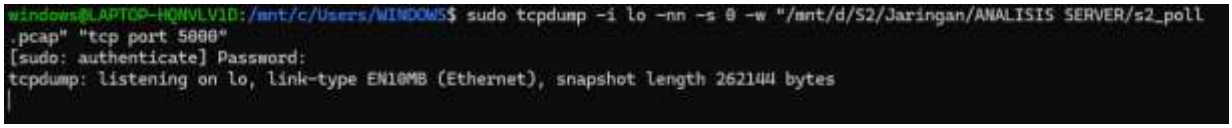
2.5.1 Menjalankan server dan rekaman baru

Setelah S1 dihentikan, gunakan tab SERVER yang sama untuk menjalankan S2. Mulai capture baru pada tab REKAM menggunakan nama file berbeda agar hasil S1 tidak tertimpa.

```
./s2_server 5000
sudo tcpdump -i lo -nn -s 0 -w "/mnt/d/S2/Jaringan/ANALISIS SERVER/s2_poll.pcap" "tcp port
5000"
```



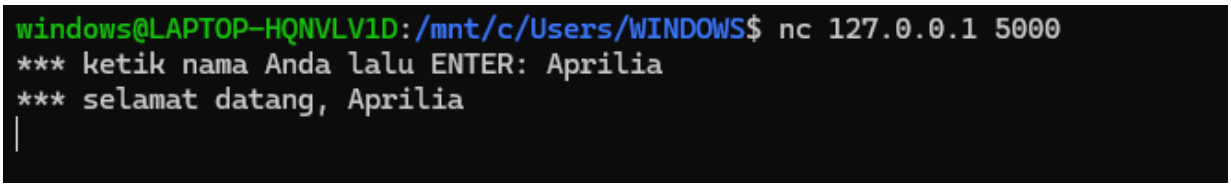
Gambar 2.32 S2 aktif dengan batas 32 koneksi klien



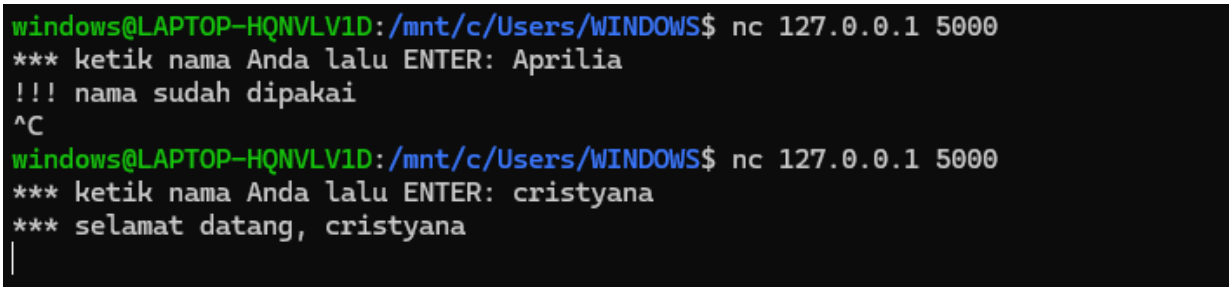
Gambar 2.33 Capture S2 disimpan ke file terpisah

2.5.2 Registrasi dan nama duplikat

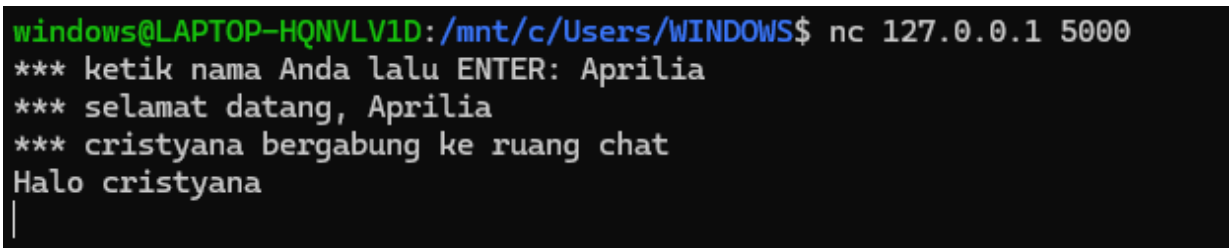
Hubungkan KLIEN A dengan nc 127.0.0.1 5000. Server mengirim prompt nama. Ketik Aprilia lalu Enter. Pada KLIEN B, percobaan awal juga menggunakan Aprilia sehingga server menolak dengan pesan nama sudah dipakai. Setelah terputus, hubungkan ulang B dan gunakan cristyana. Keduanya kemudian memperoleh pesan selamat datang. Percobaan tambahan ini menjelaskan adanya tiga koneksi TCP dalam capture S2 meskipun hanya dua pengguna aktif yang dipakai untuk chat.



Gambar 2.34 Aprilia berhasil melakukan registrasi



Gambar 2.35 Nama duplikat ditolak lalu cristyana berhasil masuk

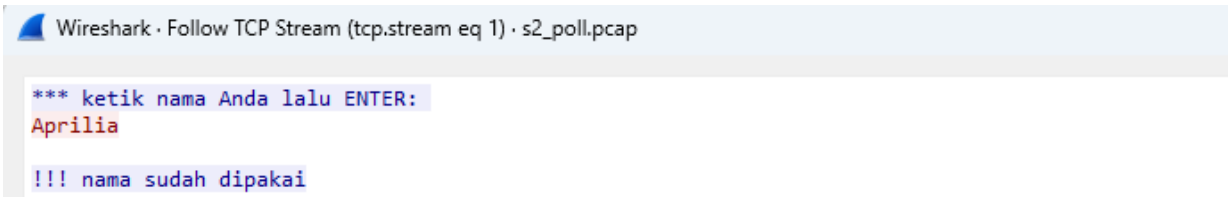


Gambar 2.36 Aprilia menerima notifikasi bahwa cristyana bergabung

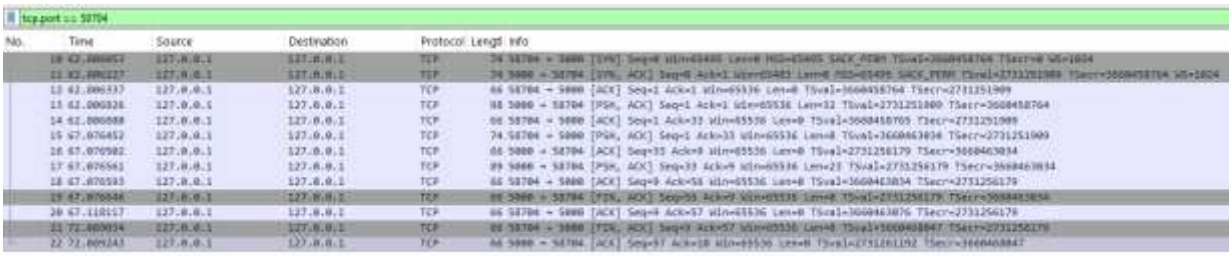
Port klien	TCP stream	Peran
50060	0	Aprilia terdaftar
58704	1	Percobaan nama Aprilia ditolak
43188	2	cristyana terdaftar

Pada koneksi yang ditolak, paket 15 mengirim Aprilia diikuti newline sebanyak 8 byte. Paket 17 membalas "!!! nama sudah dipakai" diikuti newline sebanyak 23 byte. Server mulai menutup koneksi pada paket 19. Pemeriksaan ini mengikuti nama_dipakai() dan proses_baris(). penolakan hanya mengakhiri koneksi tersebut, bukan seluruh proses server.

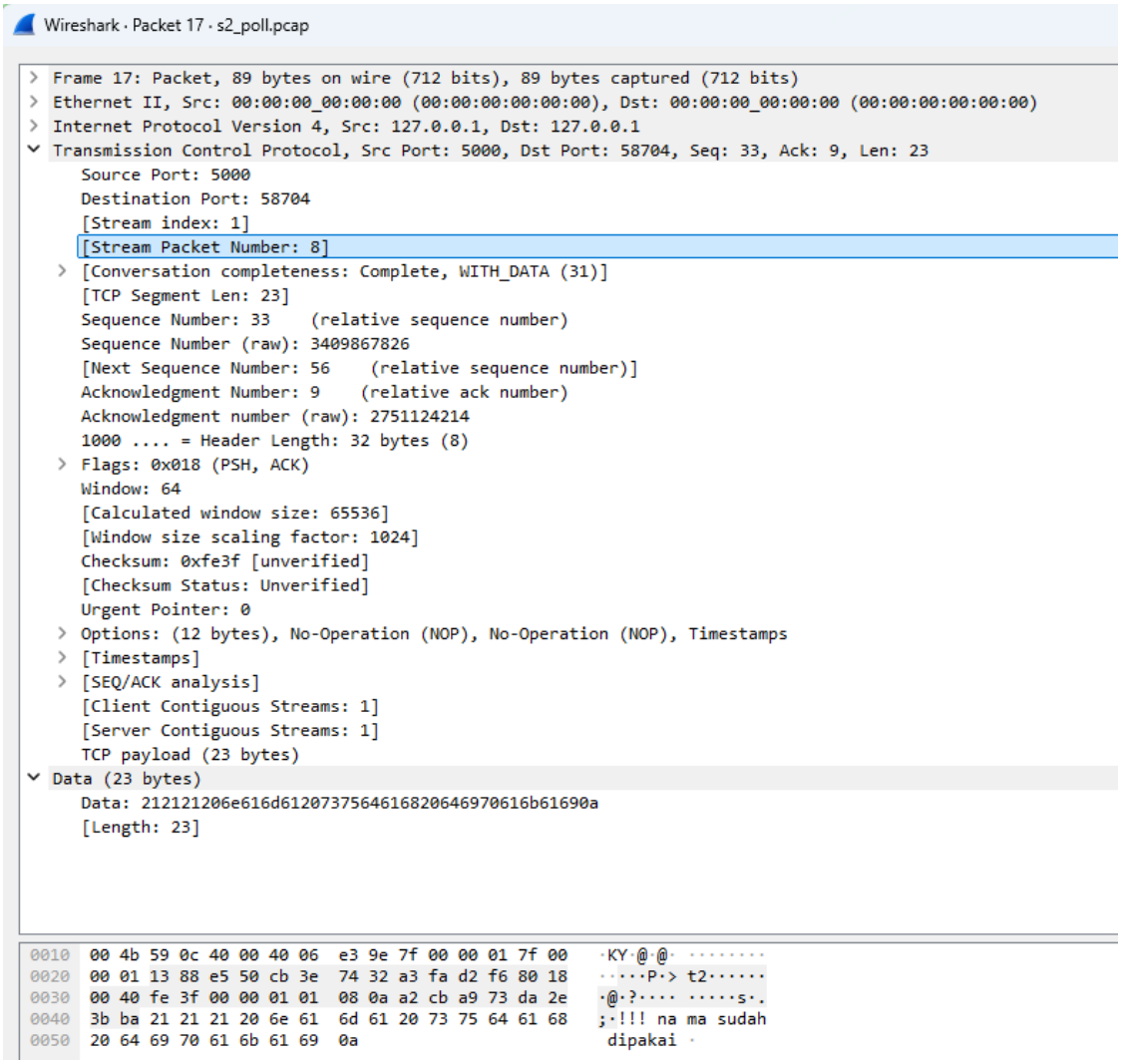
```
tcp.port == 58704
```



Gambar 2.37 Stream koneksi yang ditolak karena nama sudah digunakan



Gambar 2.38 Filter koneksi port 58704 untuk mengisolasi percobaan nama duplikat



Gambar 2.39 Paket 17 memuat pesan penolakan nama sepanjang 23 byte

2.5.3 Chat dua arah dan penerusan pesan

Pada Aprilia, kirim “Halo cristyana” lalu Enter. Pada cristyana, pesan diterima dengan awalan [Aprilia]. Kemudian cristyana mengirim “Halo juga Aprilia Haiii” dan Aprilia menerima pesan dengan awalan [cristyana]. Pesan yang terlihat pada terminal pengirim ketika mengetik merupakan tampilan lokal Netcat, server tidak menyiarkan chat kepada pengirimnya sendiri.

```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ nc 127.0.0.1 5000
*** ketik nama Anda lalu ENTER: Aprilia
*** selamat datang, Aprilia
*** cristyana bergabung ke ruang chat
Halo cristyana
[cristyana] Halo juga Aprilia Haiii
```

Gambar 2.40 Aprilia menerima pesan balasan dari cristyana

```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ nc 127.0.0.1 5000
*** ketik nama Anda lalu ENTER: cristyana
*** selamat datang, cristyana
[Aprilia] Halo cristyana
```

Gambar 2.41 Cristyana menerima chat pertama dari Aprilia

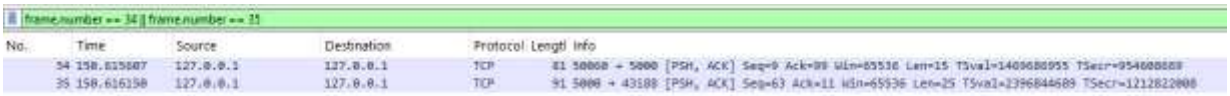
```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ nc 127.0.0.1 5000
*** ketik nama Anda lalu ENTER: cristyana
*** selamat datang, cristyana
[Aprilia] Halo cristyana
Halo juga Aprilia Haiii
```

Gambar 2.42 Cristyana mengirim chat balasan kepada Aprilia

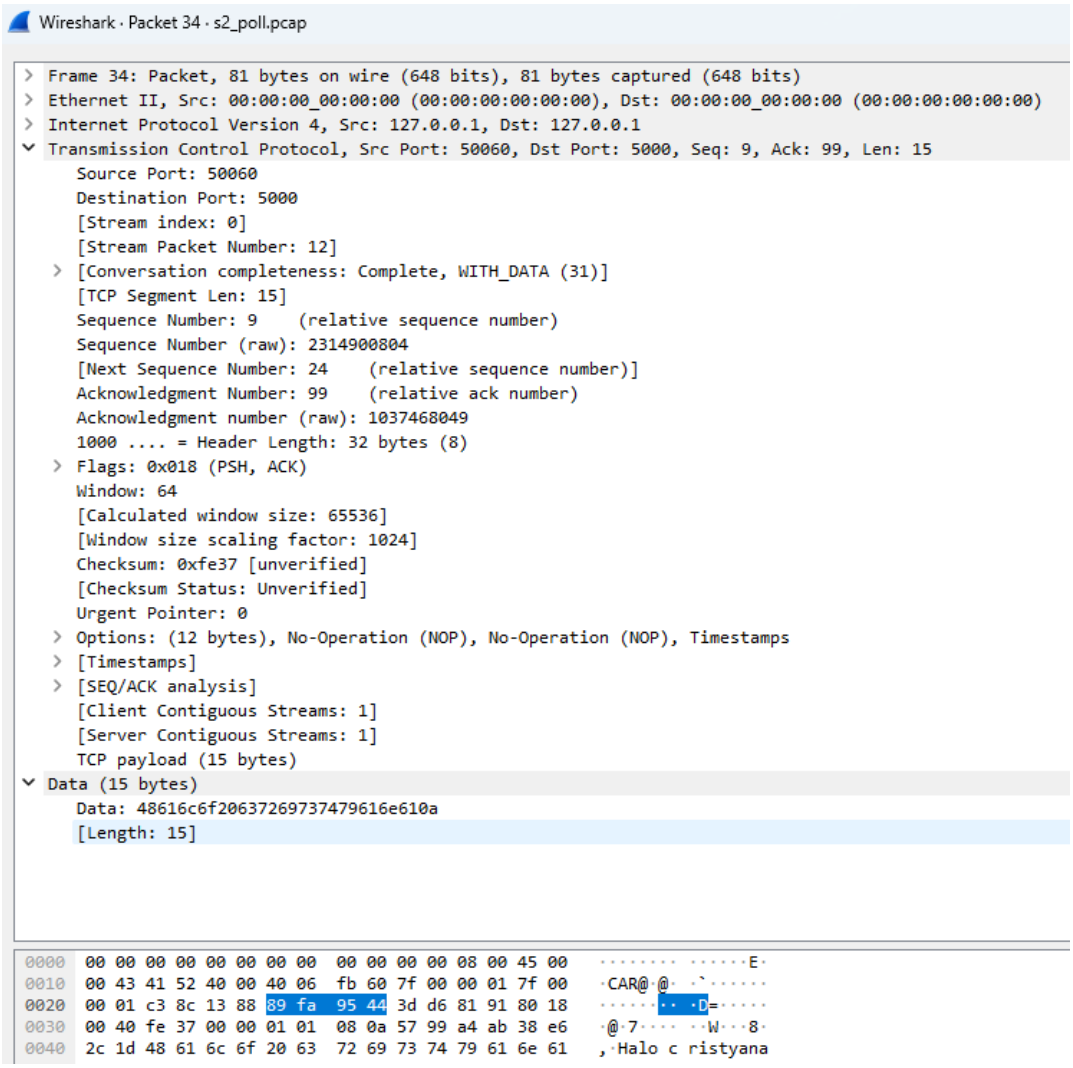
```
frame.number == 34 || frame.number == 35
```

Paket	Waktu detik	Arah	Payload byte
34	150,615607	50060 ke 5000	15
35	150,616150	5000 ke 43188	25
38	232,208232	43188 ke 5000	23
39	232,208624	5000 ke 50060	35

Paket 34 memuat Halo cristyana diikuti newline. Paket 35 memuat [Aprilia] Halo cristyana diikuti newline. Tambahan 10 byte berasal dari awalan [Aprilia] beserta spasi. Arah paket berpindah dari koneksi Aprilia ke koneksi Cristyana. ini merupakan penerusan pesan melalui server, bukan koneksi langsung antar-klien.



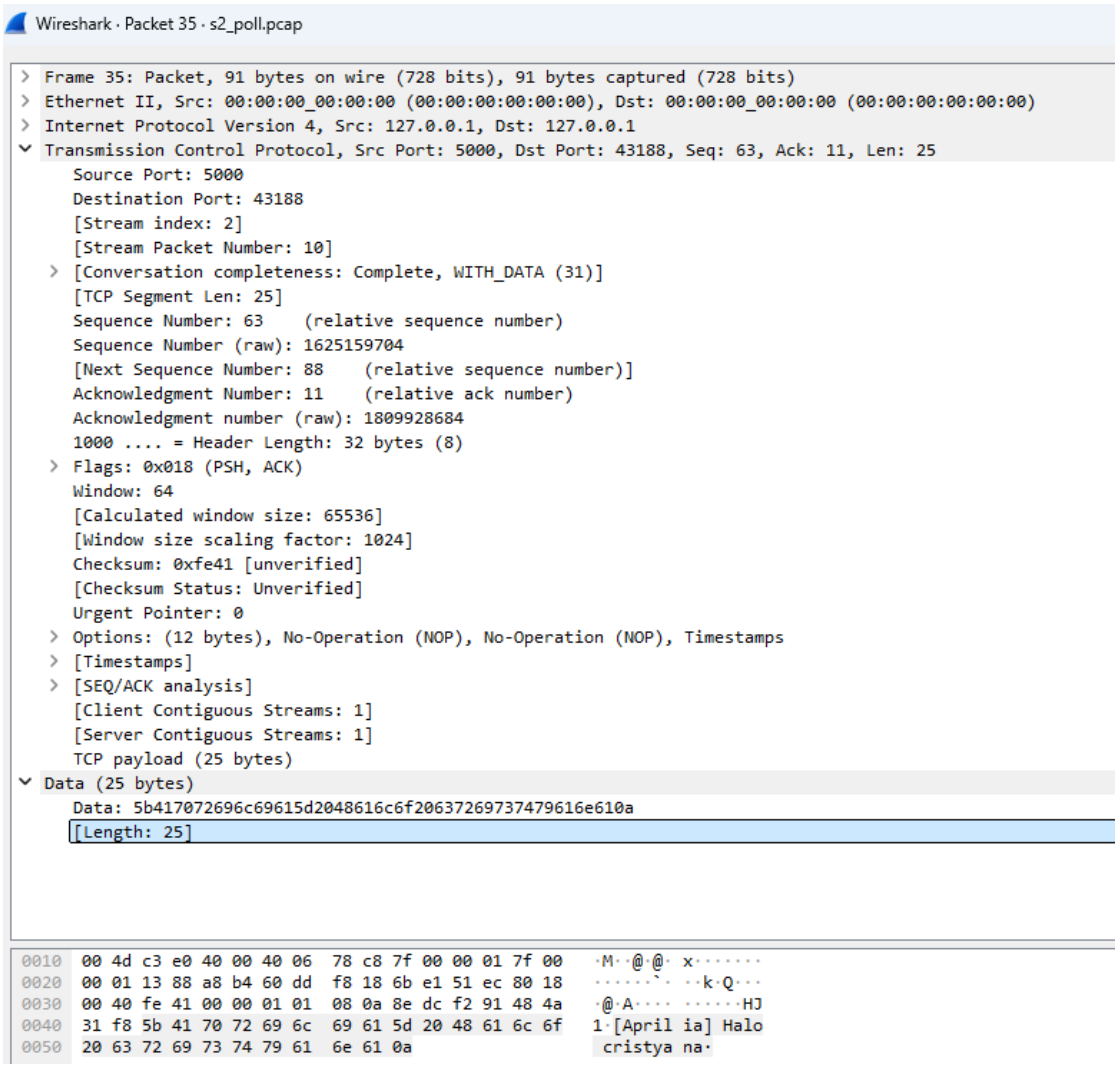
Gambar 2.43 Pasangan paket pesan Aprilia dan penerusan ke cristyana



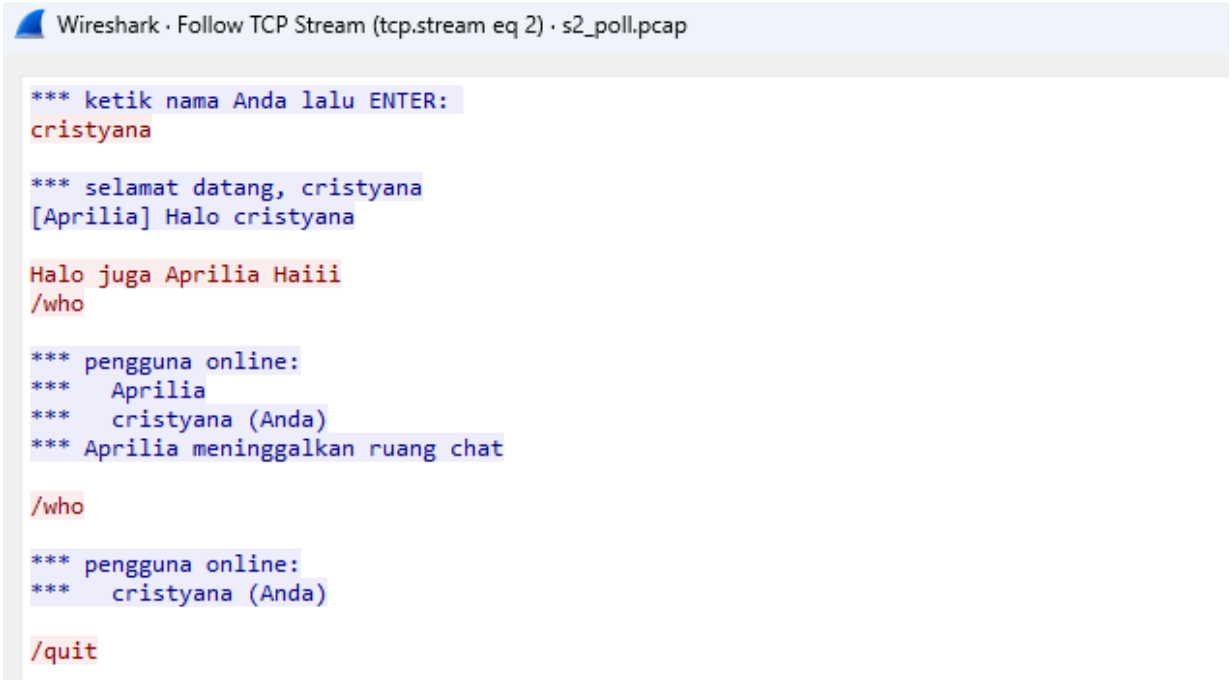
Gambar 2.44 Paket 34 membawa pesan asli sepanjang 15 byte



Gambar 2.45 Stream Aprilia menghubungkan pesan keluar dengan balasan yang diterima



Gambar 2.46 Paket 35 membawa pesan berawalan nama sepanjang 25 byte

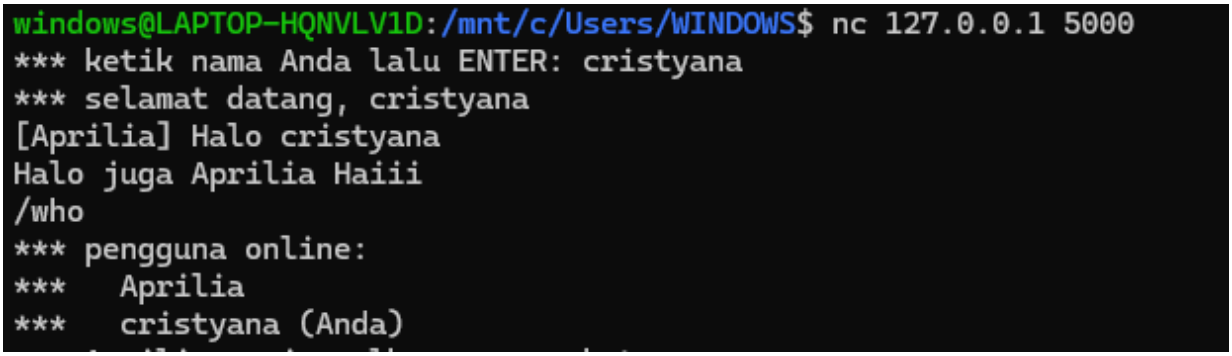


Gambar 2.47 Stream cristyana sebagai konfirmasi pesan hasil penerusan dan perubahan daftar pengguna

Selisih paket 34 dan 35 adalah 0,000543 detik atau 0,543 ms. Pesan balik pada paket 38 dan 39 memiliki selisih 0,000392 detik atau 0,392 ms. Nilai ini merupakan selisih timestamp capture pada loopback dalam satu percobaan, bukan jaminan latensi server pada jaringan lain.

2.5.4 Respons ketika klien lain diam

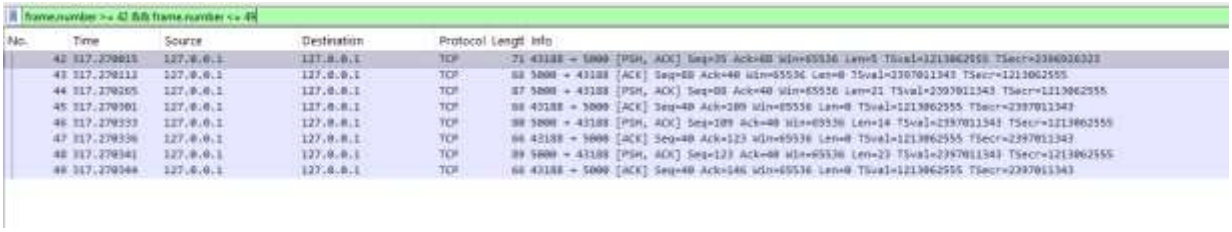
Biarkan Aprilia tetap terhubung tanpa mengirim pesan. Dari cristyana kirim /who lalu Enter. Server mengembalikan judul pengguna online beserta Aprilia dan cristyana. Respons tanpa menunggu Aprilia keluar merupakan bukti utama perbedaan model pelayanan dengan S1.



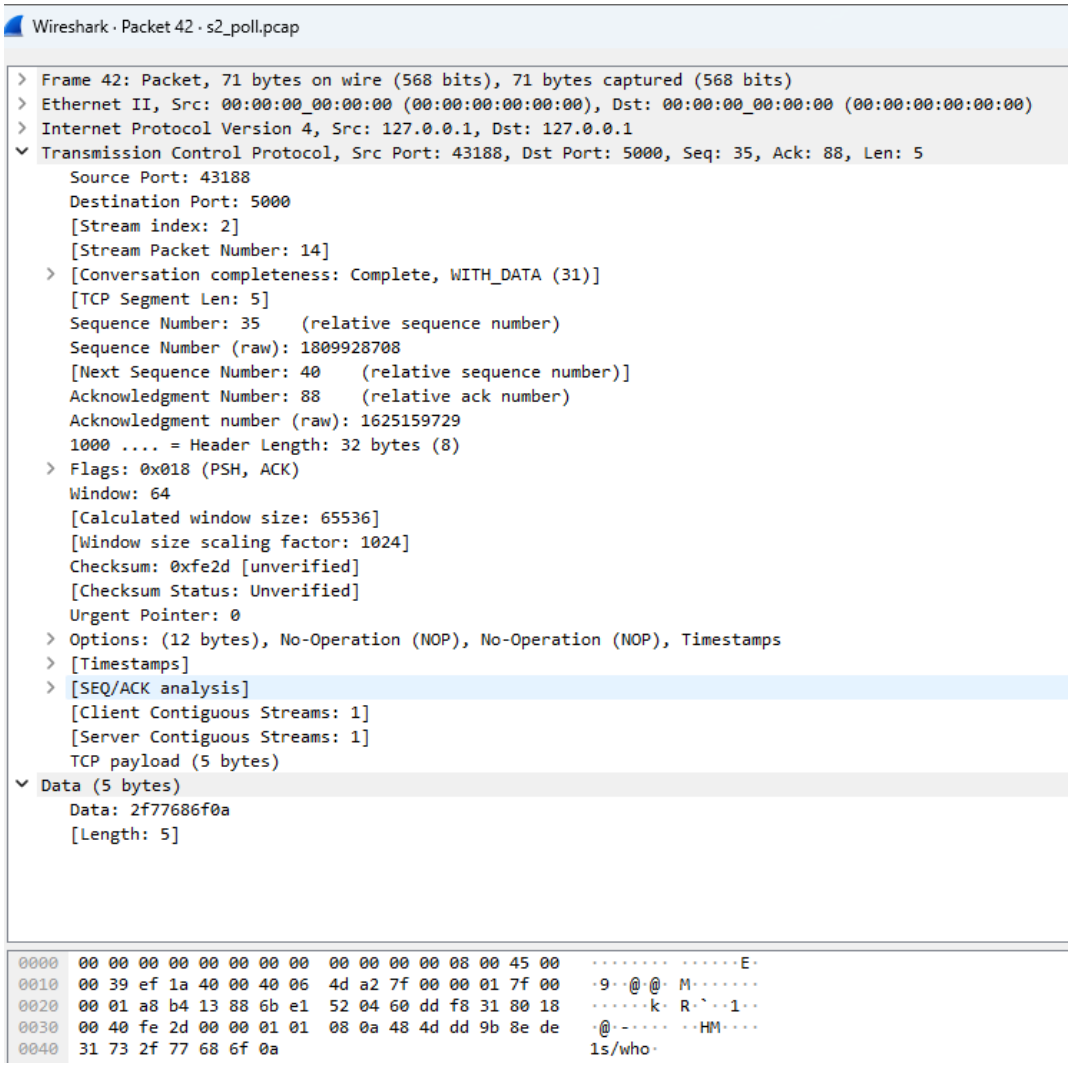
Gambar 2.48 Perintah who dijawab saat kedua pengguna masih terhubung

`frame.number >= 42 && frame.number <= 49`

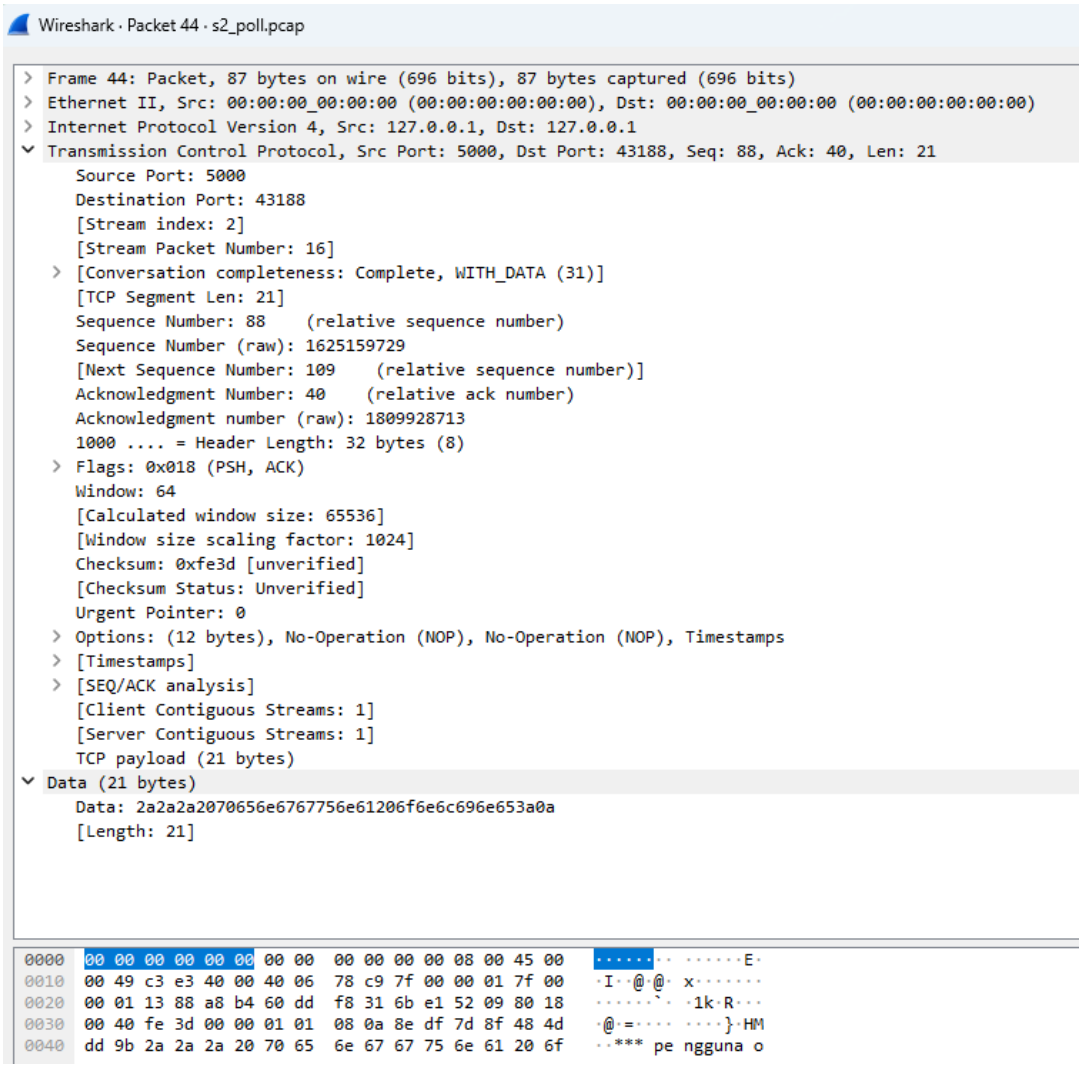
Paket	Waktu detik	Isi	Byte
42	317,270015	/who diikuti newline	5
43	317,270112	ACK tanpa payload	0
44	317,270265	Judul daftar pengguna	21
46	317,270333	Baris Aprilia	14
48	317,270341	Baris cristyana dengan penanda Anda	23



Gambar 2.49 Urutan paket permintaan who dan respons daftar pengguna



Gambar 2.50 Paket 42 berisi perintah who sepanjang 5 byte



Gambar 2.51 Paket 44 berisi judul daftar pengguna sepanjang 21 byte

Payload paket 42 adalah 2f77686f0a atau /who diikuti newline. Sequence relatif bergerak dari 35 ke 40 karena ada 5 byte data. Paket 44 memiliki ACK relatif 40, sequence relatif 88, dan next sequence 109 karena respons membawa 21 byte. Field ACK berhubungan dengan aliran masuk. sequence berhubungan dengan aliran keluar.

Waktu menuju respons payload pertama adalah 317,270265 − 317,270015 = 0,000250 detik atau 0,250 ms. Seluruh daftar pada capture selesai dikirim pada paket 48, yaitu 0,000326 detik atau 0,326 ms setelah permintaan. Paket 43 merupakan ACK TCP sehingga tidak dipakai sebagai respons aplikasi pertama.

2.5.5 Keluar dan pembaruan daftar

Dari Aprilia, kirim /quit. Program menutup socket Aprilia dan mengirim notifikasi kepada cristyana. Jalankan /who lagi dari cristyana untuk memastikan daftar hanya memuat cristyana. Setelah itu kirim /quit dari cristyana. Hentikan tcpdump dengan Ctrl+C, simpan log server, lalu hentikan S2.

```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ nc 127.0.0.1 5000
*** ketik nama Anda lalu ENTER: cristyana
*** selamat datang, cristyana
[Aprilia] Halo cristyana
Halo juga Aprilia Haiii
/who
*** pengguna online:
***   Aprilia
***   cristyana (Anda)
*** Aprilia meninggalkan ruang chat
```

Gambar 2.52 Notifikasi Aprilia meninggalkan ruang chat

```
/who
*** pengguna online:
***   cristyana (Anda)
```

Gambar 2.53 Daftar online setelah Aprilia keluar hanya memuat cristyana

```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ nc 127.0.0.1 5000
*** ketik nama Anda lalu ENTER: Aprilia
*** selamat datang, Aprilia
*** cristyana bergabung ke ruang chat
Halo cristyana
[cristyana] Halo juga Aprilia Haiii
/quit
```

Gambar 2.54 Pengiriman perintah quit pada sesi klien

```
/quit
```

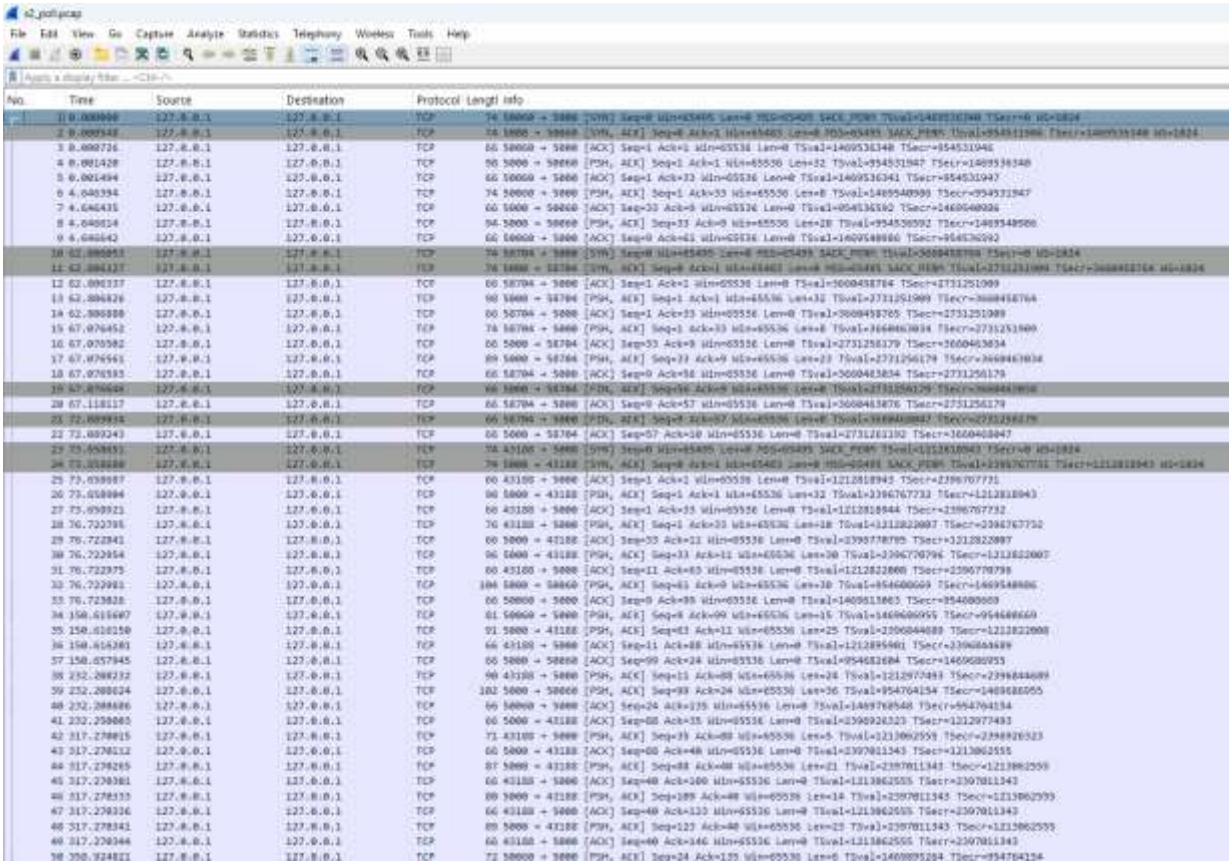
Gambar 2.55 Perintah quit terakhir untuk mengakhiri sesi pengujian

```
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$ sudo tcpdump -i lo -nn -s 0 -w "/mnt/d/S2/Jaringan/ANALISIS SERVER/s2_poll
.pcap" "tcp port 5000"
[sudo: authenticate] Password:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C63 packets captured
126 packets received by filter
0 packets dropped by kernel
windows@LAPTOP-HQNVLV1D:/mnt/c/Users/WINDOWS$
```

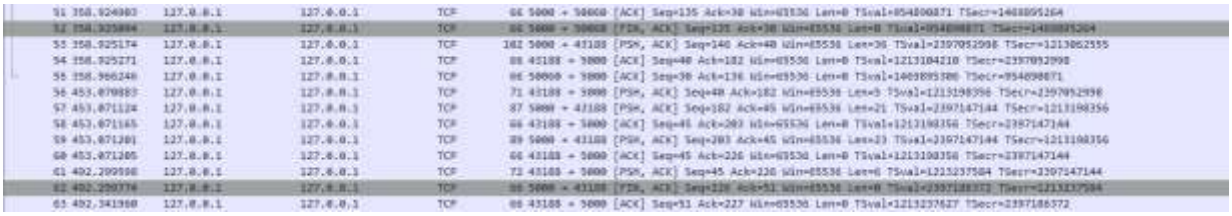
Gambar 2.56 Statistik capture S2 setelah tcpdump dihentikan

```
windows@LAPTOP-HQNVLV1D:/mnt/d/S2/Jaringan/ANALISIS SERVER$ ./s2_server 5000
2026-11-04T05:24:08Z LISTEN 127.0.0.1:5000 maks_klien=32
2026-11-04T05:26:19Z MASUK  127.0.0.1:50060 total=1
2026-11-04T05:26:24Z DAFTAR 127.0.0.1:50060 nama=Aprilia
2026-11-04T05:27:22Z MASUK  127.0.0.1:58704 total=2
2026-11-04T05:27:26Z KELUAR 127.0.0.1:58704 nama=- alasan=protokol/permintaan keluar
2026-11-04T05:27:33Z MASUK  127.0.0.1:43188 total=2
2026-11-04T05:27:36Z DAFTAR 127.0.0.1:43188 nama=cristyana
2026-11-04T05:28:50Z PESAN  nama=Aprilia panjang=14
2026-11-04T05:30:12Z PESAN  nama=cristyana panjang=23
2026-11-04T05:32:18Z KELUAR 127.0.0.1:50060 nama=Aprilia alasan=protokol/permintaan keluar
2026-11-04T05:34:32Z KELUAR 127.0.0.1:43188 nama=cristyana alasan=protokol/permintaan keluar
^C
windows@LAPTOP-HQNVLV1D:/mnt/d/S2/Jaringan/ANALISIS SERVER$
```

Gambar 2.57 Log registrasi pesan dan keluarnya klien pada S2



Gambar 2.58 Daftar paket S2 dari registrasi hingga pertukaran pesan



Gambar 2.59 Lanjutan daftar paket S2 termasuk penutupan koneksi

Pada capture, paket 50 memuat /quit dari Aprilia. Server mengirim FIN pada paket 52 dan notifikasi keluar kepada cristyana pada paket 53. Perintah /who kedua berada pada paket 56 dan respons daftar pada paket 57 serta 59. Paket

61 memuat /quit dari cristyana, lalu server mengirim FIN pada paket 62. Paket FIN menutup satu arah pengiriman TCP. koneksi diselesaikan dengan respons sisi lainnya, bukan hanya satu paket FIN.

```
/quit
```

Gambar 2.60 Stream cristyana memperlihatkan rangkaian registrasi chat who dan quit

```
/who

*** pengguna online:
***   cristyana (Anda)
```

Gambar 2.61 Dokumentasi stream cristyana pada tahap pemeriksaan who

Wireshark · Follow TCP Stream (tcp.stream eq 2) · s2_poll.pcap

```
*** ketik nama Anda lalu ENTER:
cristyana

*** selamat datang, cristyana
[Aprilia] Halo cristyana

Halo juga Aprilia Haiii
/who

*** pengguna online:
***   Aprilia
***   cristyana (Anda)
*** Aprilia meninggalkan ruang chat

/who

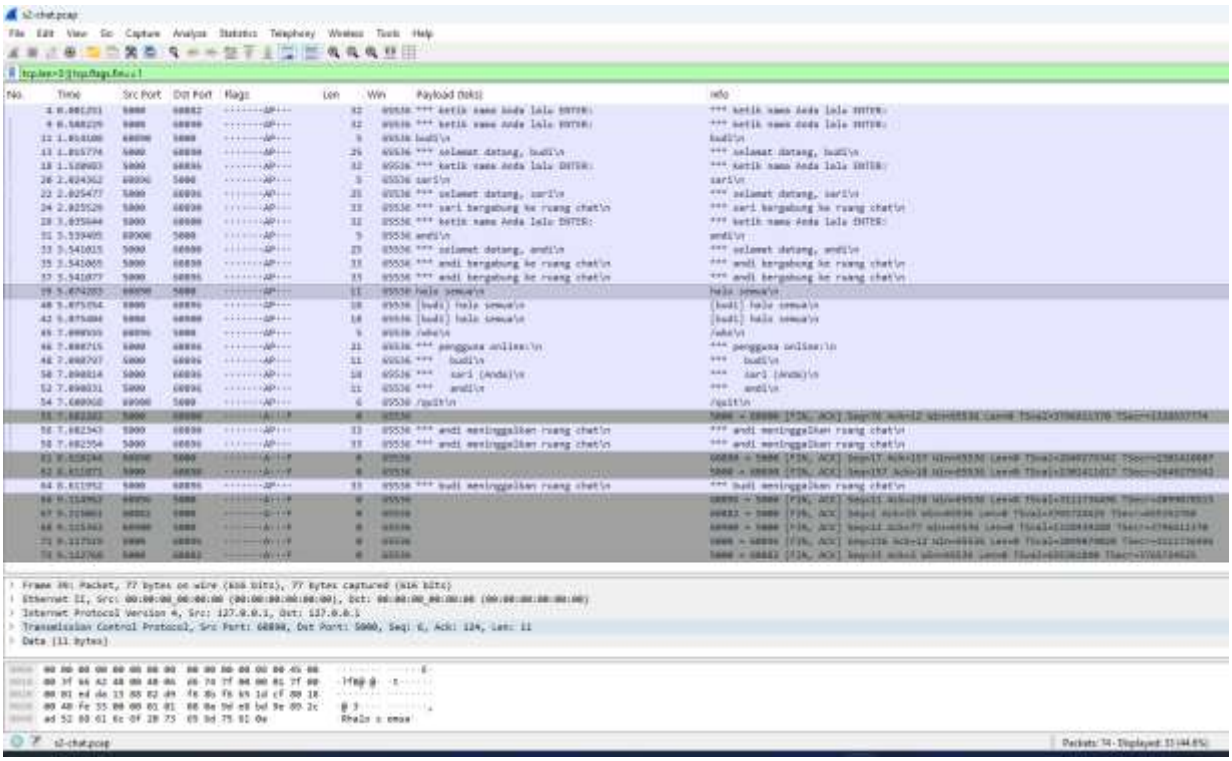
*** pengguna online:
***   cristyana (Anda)

/quit
```

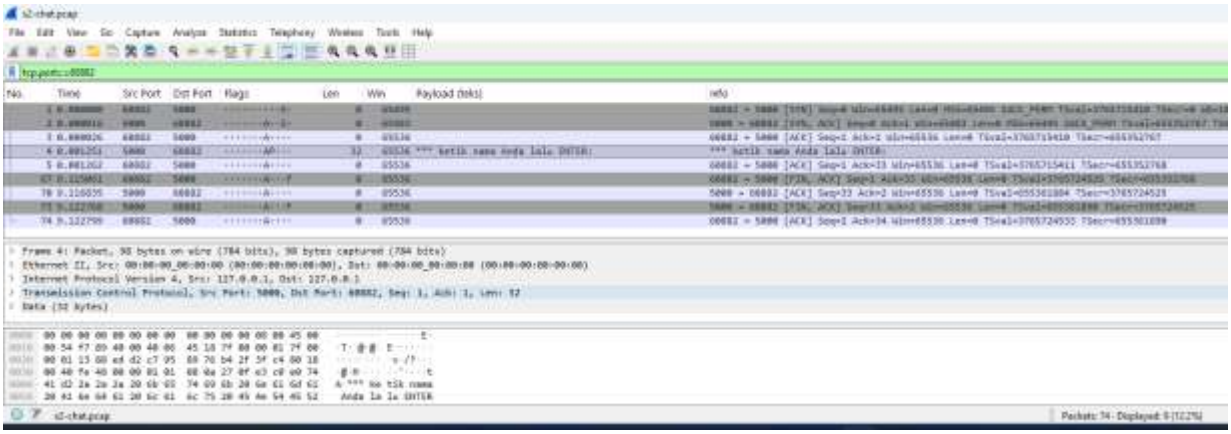
Gambar 2.62 Dokumentasi stream lengkap yang menghubungkan who dan keluarnya Aprilia

2.5.6 Eksperimen lanjutan S2 tiga pengguna dan satu klien diam

Skenario s2-chat membuka koneksi diam (port 60882) yang tidak pernah mengirim nama, lalu tiga pengguna budi (60890), sari (60896), dan andi (60900) bergabung bergantian. budi mengirim halo semua, sari menjalankan /who, andi keluar dengan /quit, lalu budi dan sari menutup koneksi.



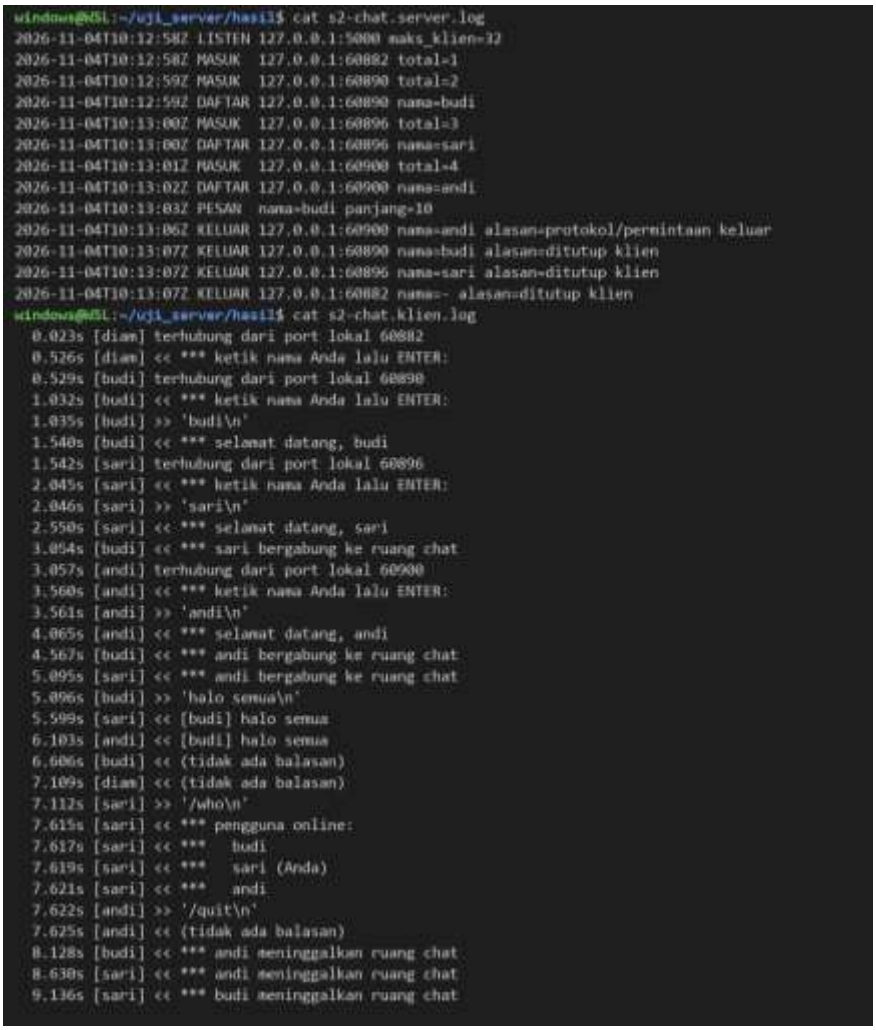
Gambar 2.63 s2-chat.pcap filter tcp.len>0 || FIN, satu pesan budi (frame 39) disiarkan ke sari dan andi (frame 40, 42)



Gambar 2.64 s2-chat.pcap filter tcp.port==60882, klien diam hanya menerima prompt dan tidak menerima siaran

Frame	Waktu (s)	Arah	Payload
4	0,001251	S ke diam	*** ketik nama Anda lalu ENTER:
6-9	0,505 ke 0,508	budi dan S	Handshake budi lalu prompt 2,8 ms kemudian, walau diam masih terhubung
11 ke 13	1,014108 ke 1,015774	budi dan S	budi lalu *** selamat datang, budi
24	2,025529	S ke budi	*** sari bergabung ke ruang chat
35, 37	3,541065	S ke budi, sari	*** andi bergabung ke ruang chat
39	5,074283	budi ke S	halo semua (11 byte)
40, 42	5,075354, 5,075404	S ke sari, andi	[budi] halo semua (18 byte)
45 ke 46-52	7,090555	sari dan S	/who dijawab 4 segmen dalam 0,3 ms

54 ke 55	7,600968 ke 7,602282	andi dan S	/quit lalu FIN dari server
56, 58	7,602343	S ke budi, sari	*** andi meninggalkan ruang chat

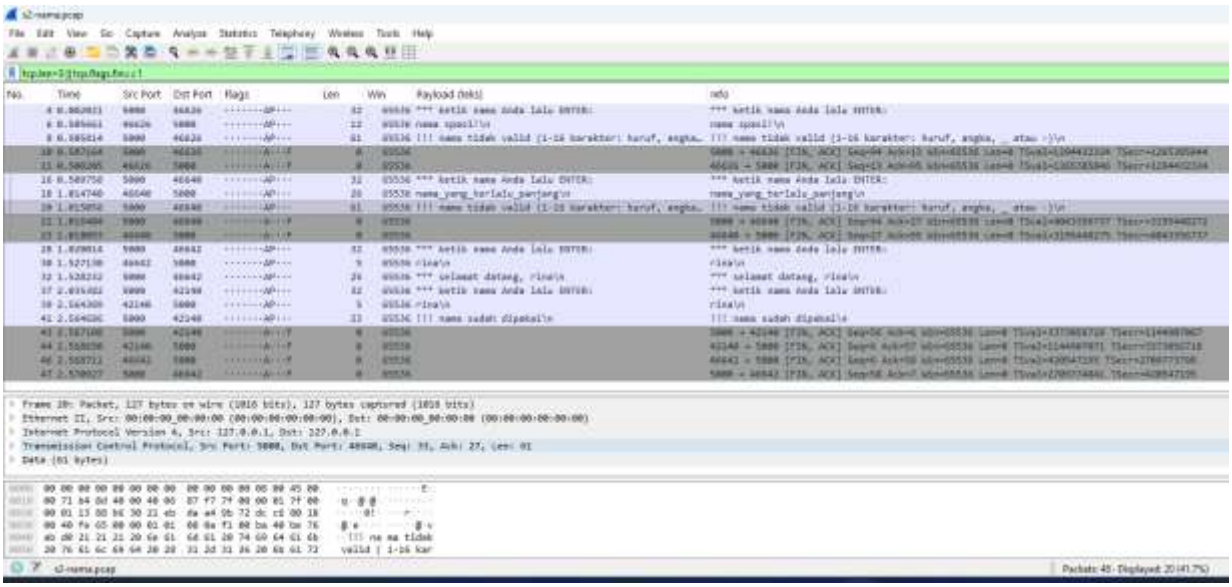


Gambar 2.65 Log server dan log klien s2-chat

Analisis: koneksi diam tidak menghambat siapa pun. Setiap pengguna baru menerima prompt 1 sampai 3 ms setelah handshake. Satu pesan masuk (frame 39) menghasilkan dua pesan keluar dalam 1,1 ms, yaitu rasio 1 : (N - 1) karena siar() melewati pengirim. Klien diam tidak menerima [budi] halo semua karena siar() hanya mengirim ke klien dengan terdaftar = 1 (baris 99). Log klien mencatat diam << (tidak ada balasan) dan log server mencatat KELUAR ... nama=- untuk klien tersebut. Pada /quit, FIN pertama berasal dari port 5000, kebalikan dari S1 yang selalu menunggu klien menutup.

2.5.7 Eksperimen lanjutan S2 validasi nama

Skenario s2-nama melengkapi percobaan manual nama duplikat pada subbab 2.5.2 dengan dua jenis nama tidak valid: nama spasi! yang mengandung spasi dan tanda seru, serta nama_yang_terlalu_panjang yang terdiri dari 25 karakter.



Gambar 2.66 s2-nama.pcap, nama tidak valid dan nama ganda ditolak dengan pesan !!! lalu FIN dari server

Frame	Klien	Kiriman	Balasan server	Penutupan
6 ke 8	x1 (46626)	nama spasi!	!!! nama tidak valid (61 byte)	FIN server frame 10
18 ke 20	x2 (46640)	nama_yang_terlalu_panjang	!!! nama tidak valid (61 byte)	FIN server frame 22
30 ke 32	rina (46642)	rina	*** selamat datang, rina	Tetap terhubung
39 ke 41	x3 (42140)	rina	!!! nama sudah dipakai (23 byte)	FIN server frame 43

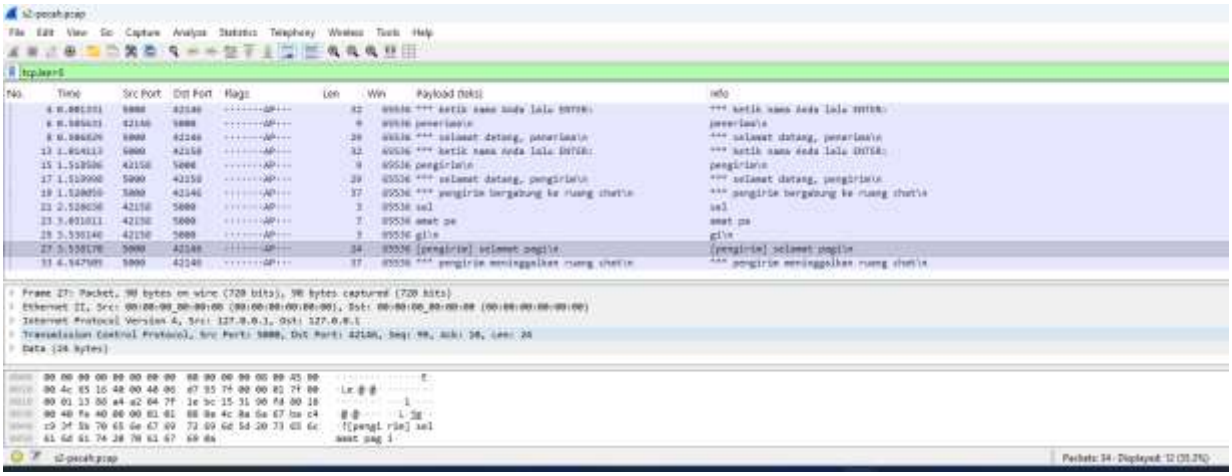
```
windows@WSL:~/uji_server/hasil$ cat s2-nama.klien.log
  0.031s [x1] terhubung dari port lokal 46626
  0.533s [x1] << *** ketik nama Anda lalu ENTER:
  0.535s [x1] >> 'nama spasi!\n'
  0.538s [x1] << !!! nama tidak valid (1-16 karakter: huruf, angka, _ atau -)
  0.539s [x2] terhubung dari port lokal 46640
  1.041s [x2] << *** ketik nama Anda lalu ENTER:
  1.043s [x2] >> 'nama_yang_terlalu_panjang\n'
  1.047s [x2] << !!! nama tidak valid (1-16 karakter: huruf, angka, _ atau -)
  1.049s [rina] terhubung dari port lokal 46642
  1.552s [rina] << *** ketik nama Anda lalu ENTER:
  1.554s [rina] >> 'rina\n'
  2.060s [rina] << *** selamat datang, rina
  2.063s [x3] terhubung dari port lokal 42140
  2.567s [x3] << *** ketik nama Anda lalu ENTER:
  2.569s [x3] >> 'rina\n'
  2.598s [x3] << !!! nama sudah dipakai
windows@WSL:~/uji_server/hasil$ cat s2-nama.server.log
2026-11-04T10:13:10Z LISTEN 127.0.0.1:5000 maks_klien=32
2026-11-04T10:13:11Z MASUK  127.0.0.1:46626 total=1
2026-11-04T10:13:11Z KELUAR 127.0.0.1:46626 nama=- alasan=protokol/permintaan keluar
2026-11-04T10:13:11Z MASUK  127.0.0.1:46640 total=1
2026-11-04T10:13:12Z KELUAR 127.0.0.1:46640 nama=- alasan=protokol/permintaan keluar
2026-11-04T10:13:12Z MASUK  127.0.0.1:46642 total=1
2026-11-04T10:13:13Z DAFTAR 127.0.0.1:46642 nama=rina
2026-11-04T10:13:13Z MASUK  127.0.0.1:42140 total=2
2026-11-04T10:13:14Z KELUAR 127.0.0.1:42140 nama=- alasan=protokol/permintaan keluar
2026-11-04T10:13:14Z KELUAR 127.0.0.1:46642 nama=rina alasan=ditutup klien
windows@WSL:~/uji_server/hasil$ cat s2-panjang.klien.log
  0.025s [saksi] terhubung dari port lokal 33598
  0.527s [saksi] << *** ketik nama Anda lalu ENTER:
  0.529s [saksi] >> 'saksi\n'
  1.033s [saksi] << *** selamat datang, saksi
  1.035s [panjang] terhubung dari port lokal 40950
  1.538s [panjang] << *** ketik nama Anda lalu ENTER:
  1.540s [panjang] >> 'panjang\n'
  2.046s [panjang] << *** selamat datang, panjang
  2.548s [saksi] << *** panjang bergabung ke ruang chat
  2.550s [panjang] >> 'AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA... (2100 byte)'
  2.553s [panjang] << koneksi error: [Errno 104] Connection reset by peer
  2.555s [panjang] << !!! baris terlalu panjang
  3.057s [saksi] << *** panjang meninggalkan ruang chat
windows@WSL:~/uji_server/hasil$ cat s2-panjang.server.log
2026-11-04T10:13:31Z LISTEN 127.0.0.1:5000 maks_klien=32
2026-11-04T10:13:32Z MASUK  127.0.0.1:33598 total=1
2026-11-04T10:13:32Z DAFTAR 127.0.0.1:33598 nama=saksi
2026-11-04T10:13:33Z MASUK  127.0.0.1:40950 total=2
2026-11-04T10:13:33Z DAFTAR 127.0.0.1:40950 nama=panjang
2026-11-04T10:13:34Z KELUAR 127.0.0.1:40950 nama=panjang alasan=protokol/permintaan keluar
2026-11-04T10:13:35Z KELUAR 127.0.0.1:33598 nama=saksi alasan=ditutup klien
```

Gambar 2.67 Log klien dan log server s2-nama serta s2-panjang

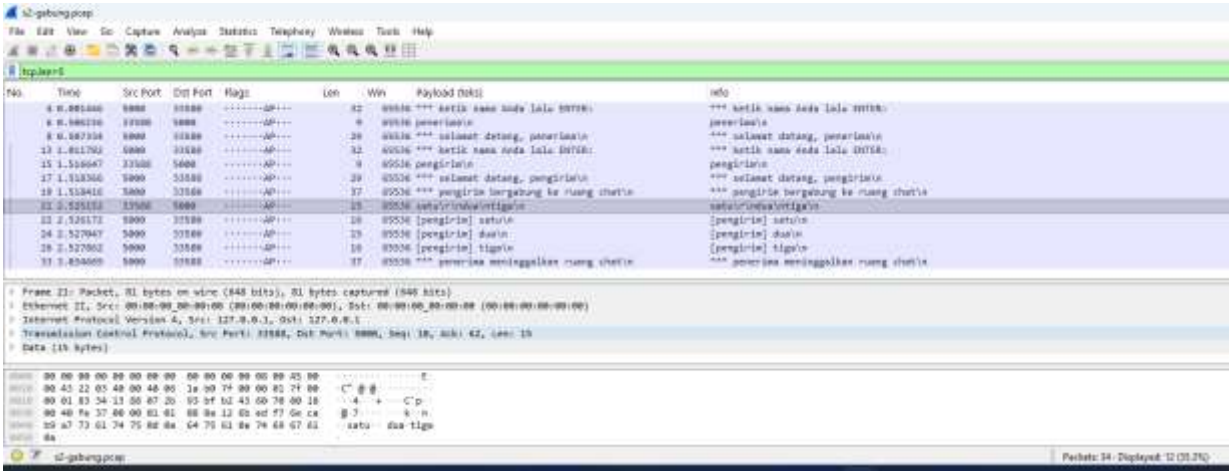
Analisis: nama_valid() pada baris 119-128 menolak karakter selain huruf, angka, garis bawah, dan tanda minus, serta panjang lebih dari 16 karakter (MAKS_NAMA - 1). nama_dipakai() pada baris 130-135 menolak nama yang sama persis. Pada ketiga kasus proses_baris() mengembalikan 0, rakit_baris() meneruskannya, dan tutup_klien() menutup socket dengan alasan protokol/permintaan keluar. Karena klien belum terdaftar, log mencatat nama=- dan tidak ada siaran keluar ke pengguna lain.

2.5.8 Eksperimen lanjutan S2 framing baris

Dua skenario dipakai untuk membuktikan bahwa S2 memperlakukan TCP sebagai aliran byte. Pada s2-pecah, pengirim mengirim sel, amat pa, dan gi diikuti newline dalam tiga segmen. Pada s2-gabung, pengirim mengirim tiga baris satu, dua, dan tiga dalam satu send(), dengan baris pertama memakai CRLF.

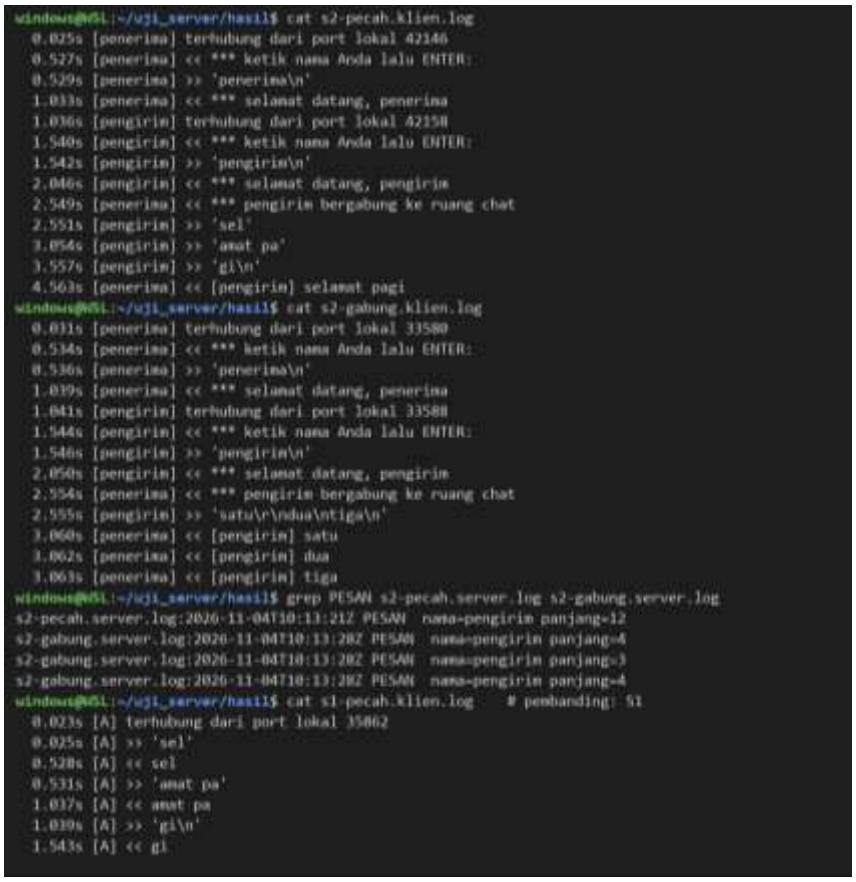


Gambar 2.68 s2-pecah.pcap filter tcp.len>0, segmen frame 21, 23, 25 digabung menjadi satu baris pada frame 27



Gambar 2.69 s2-gabung.pcap filter tcp.len>0, satu segmen 15 byte (frame 21) dipecah menjadi tiga pesan (frame 22, 24, 26)

Skenario	Frame masuk	Frame keluar	Hasil
s2-pecah	21 sel (2,528638), 23 amat pa (3,031611), 25 gi\n (3,536146)	27 (3,538170)	[pengirim] selamat pagi\n 24 byte, 2,0 ms setelah newline tiba
s2-gabung	21 satu\r\ndua\ntiga\n 15 byte (2,525152)	22, 24, 26 (2,526 sampai 2,528)	Tiga pesan 16, 15, 16 byte, CR dibuang
s1-pecah (pembanding)	4, 8, 11	6, 9, 12	Tiga potongan dipantulkan apa adanya

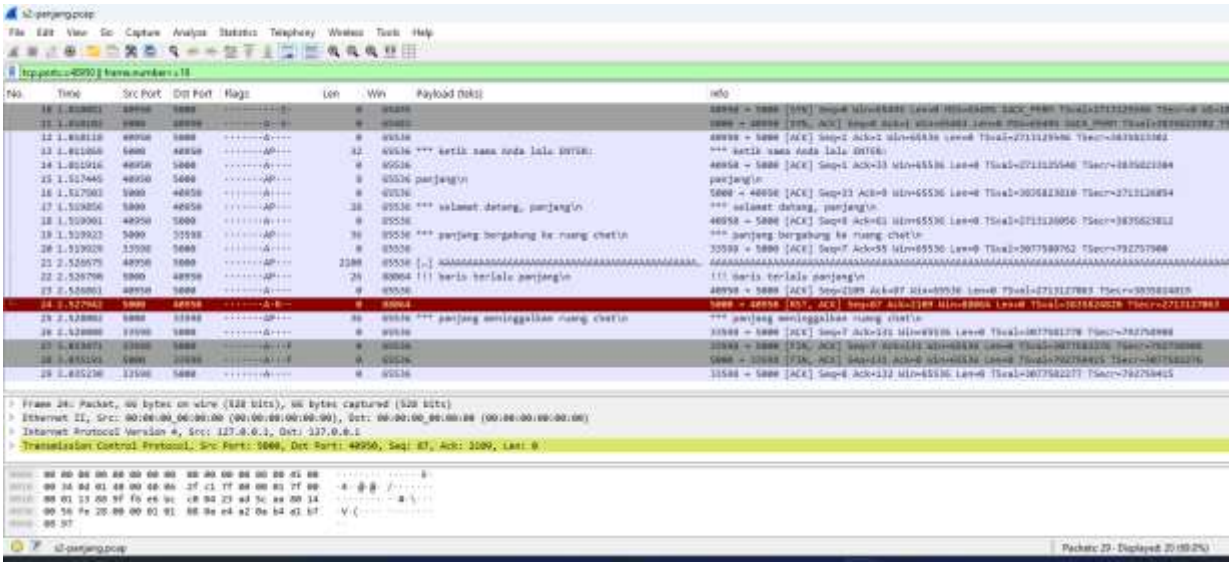


Gambar 2.70 Log klien s2-pecah, s2-gabung, PESAN di log server, dan pembanding s1-pecah

Pada s2-pecah, server menyimpan potongan pesan sampai newline diterima. Baris lengkap kemudian disiarkan sebagai satu pesan dan log mencatat panjang 12 byte untuk teks selamat pagi. Pada s2-gabung, rakit_baris() memproses tiga baris dari satu buffer penerimaan. Teks satu, dua, dan tiga masing-masing memiliki panjang 4, 3, dan 4 byte. CR pada akhir baris pertama dibuang sebelum pemrosesan, sehingga pesan [pengirim] satu diikuti newline berukuran 16 byte. Perilaku ini membuktikan bahwa batas pesan ditentukan oleh newline, bukan batas segmen TCP.

2.5.9 Eksperimen lanjutan S2 baris melebihi buffer

Skenario s2-panjang mendaftarkan pengguna saksi dan panjang, kemudian panjang mengirim 2100 huruf A tanpa newline.

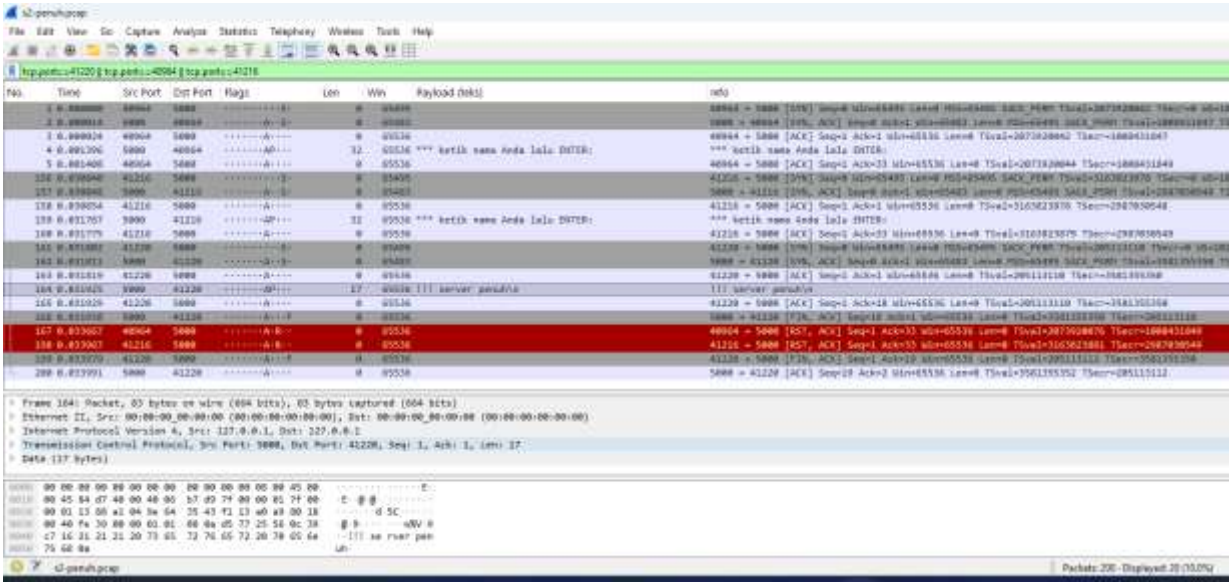


Gambar 2.71 s2-panjang.pcap, 2100 byte tanpa newline dijawab baris terlalu panjang lalu RST dari server (frame 24)

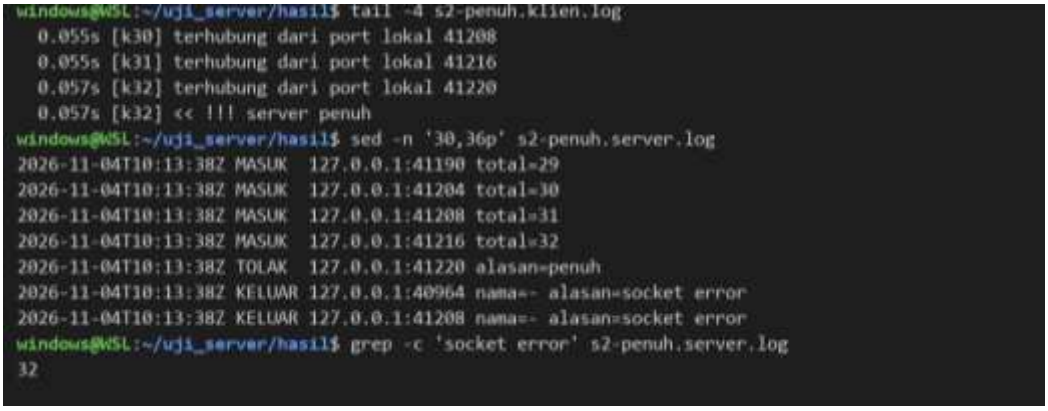
Pada skenario ini, ruang buffer masuk awal sebesar 2048 byte. Setelah buffer penuh tanpa newline, rakit_baris() mengirim pesan baris terlalu panjang dan meminta penutupan klien. Capture menunjukkan RST pada frame 24. Perilaku ini konsisten dengan penutupan socket Linux ketika masih ada data masuk yang belum dibaca, yaitu sisa dari kiriman 2100 byte. Pengguna saksi tetap menerima pemberitahuan bahwa panjang meninggalkan ruang chat. Pembatasan tersebut mencegah satu baris tanpa delimiter terus menambah kebutuhan buffer masuk.

2.5.10 Eksperimen lanjutan S2 kapasitas 32 klien

Skenario s2-penuh membuka 33 koneksi berturut-turut tanpa jeda, membaca balasan untuk koneksi terakhir, lalu menutup semua koneksi.



Gambar 2.72 s2-penuh.pcap, koneksi ke-33 (port 41220) menerima !!! server penuh lalu FIN dari server

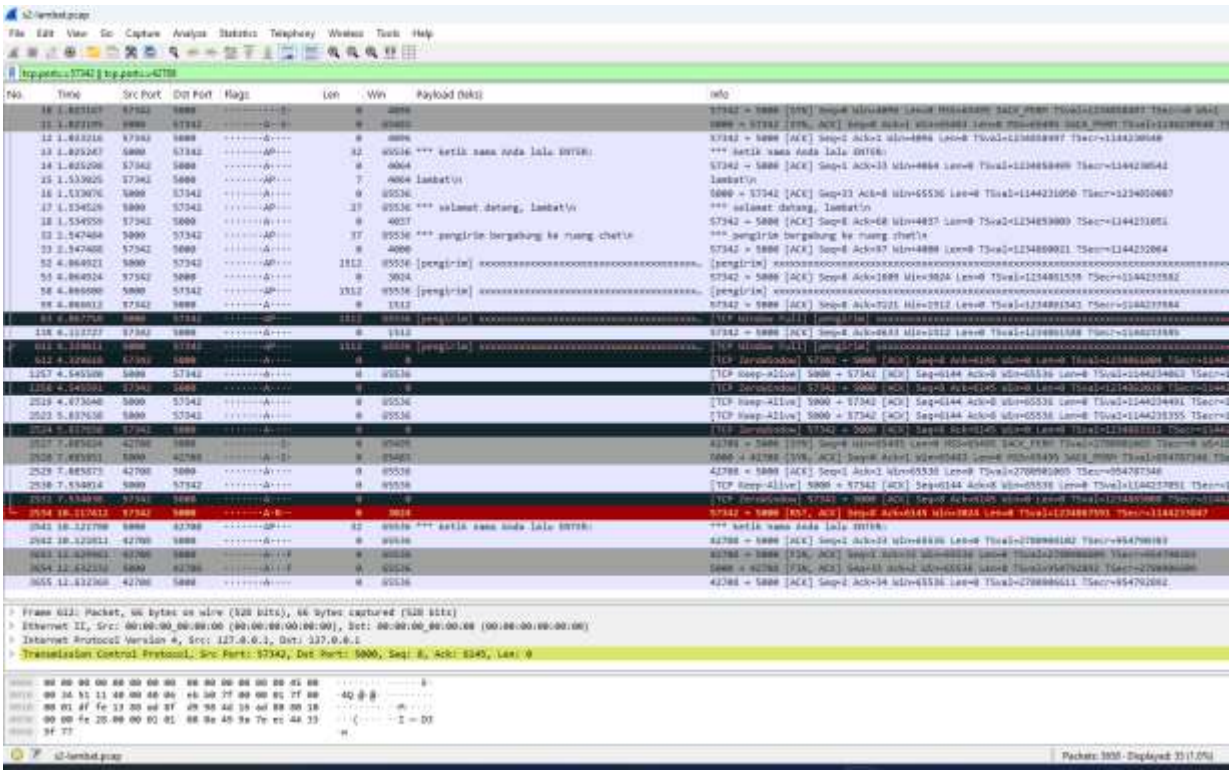


Gambar 2.73 Log klien dan log server s2-penuh: total=32, TOLAK, dan 32 socket error

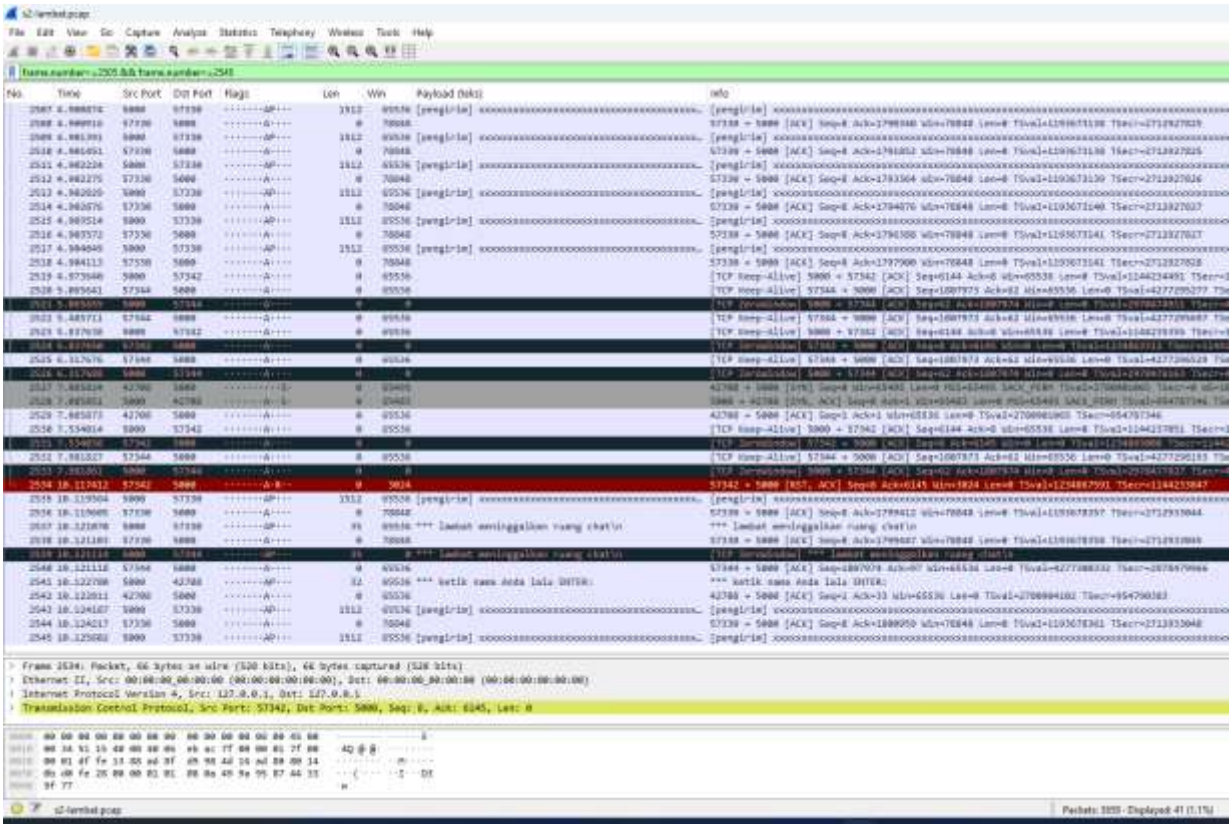
Analisis: seluruh 33 handshake selesai dalam 31,8 ms. Log server mencatat MASUK total=1 sampai total=32, lalu TOLAK 127.0.0.1:41220 alasan=penuh. Handshake koneksi ke-33 tetap sukses karena dikerjakan kernel (frame 161-163), kemudian S2 mengirim !!! server penuh 17 byte (frame 164) dan langsung FIN (frame 166) sesuai baris 261-267. Ketika klien penguji menutup 32 koneksi lain yang prompt-nya belum dibaca, kernel klien mengirim 32 RST. poll() melaporkan POLLERR sehingga log mencatat 32 kali alasan=socket error (baris 235) dan server tidak crash. Bandingkan dengan S1 pada subbab 2.4.7: S1 diam dan klien mengulang SYN, sedangkan S2 menolak secara eksplisit.

2.5.11 Eksperimen lanjutan S2 klien lambat dan send blocking

Kelemahan S2 yang disebut pada komentar baris 81 diuji langsung pada skenario s2-lambat. Tiga pengguna mendaftar: korban (port 57330) yang selalu membaca, lambat (57342) dengan SO_RCVBUF 4096 byte yang tidak pernah membaca setelah mendaftar, dan pengirim (57344) yang mengirim baris 1500 byte secepat mungkin selama 6 detik. Pada detik ke-3 banjir, klien keempat baru (42708) tersambung dan menunggu prompt. Setelah itu lambat menutup koneksi.

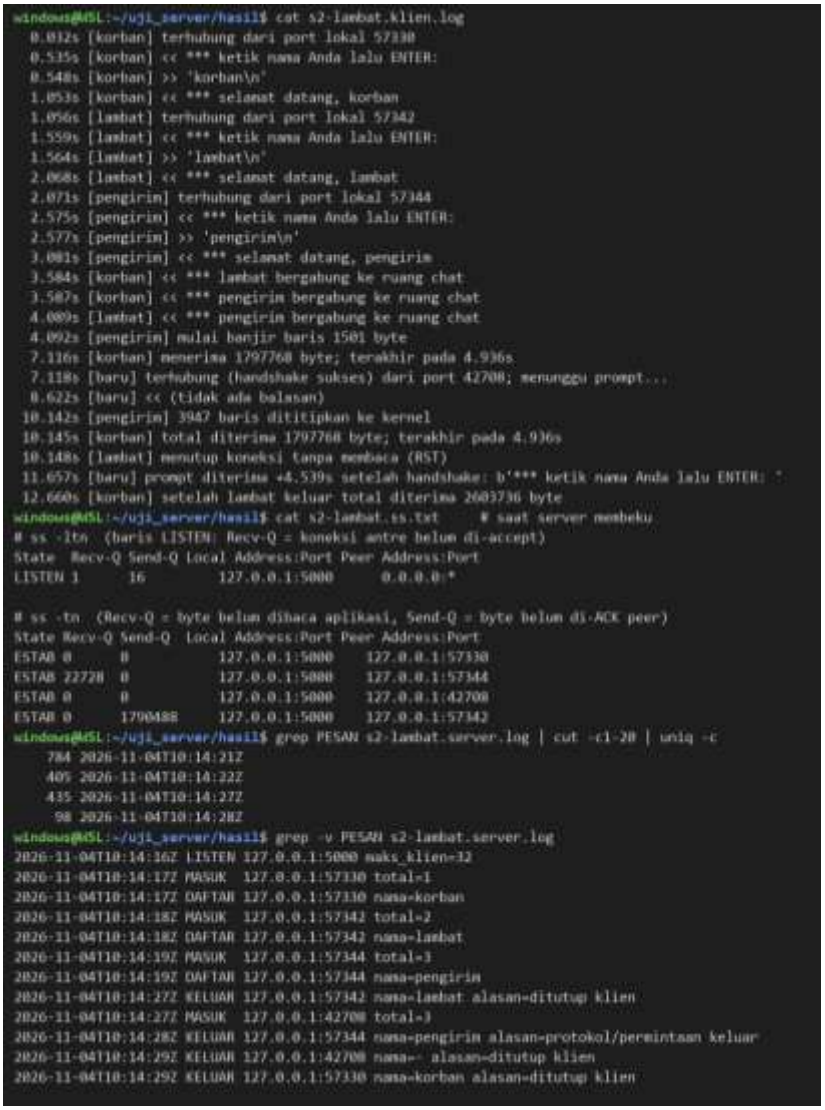


Gambar 2.74 s2-lambat.pcap port lambat dan baru, lambat mengiklankan ZeroWindow, baru sudah handshake tetapi prompt baru tiba setelah RST lambat



Gambar 2.75 s2-lambat.pcap frame 2505-2545, siaran ke korban berhenti di frame 2517 dan baru berlanjut 2,2 ms setelah RST lambat (frame 2534)

Frame	Waktu (s)	Kejadian
34	4,060811	pengirim mulai mengirim baris 1501 byte
63	4,067758	TCP Window Full ke lambat: window lambat yang kecil langsung penuh
612	4,329618	lambat mengiklankan TCP ZeroWindow
2517	4,904049	Siaran terakhir yang diterima korban sebelum server membeku
2519, 2523, 2530	4,97, 5,84, 7,53	Server mengirim zero-window probe ke lambat
2521, 2526, 2533	5,07, 6,32, 7,98	Server mengiklankan ZeroWindow ke pengirim karena berhenti membaca
2527-2529	7,085824	Handshake baru sukses oleh kernel
2534	10,117412	lambat mengirim RST
2535	10,119564	Siaran ke korban berlanjut
2541	10,122780	Prompt untuk baru akhirnya dikirim, 3,037 s setelah handshake



Gambar 2.76 Log klien, ss saat server membeku, dan sebaran PESAN per detik pada log server s2-lambat

Analisis: kernel server menampung siaran untuk lambat di send buffer. Setelah send buffer penuh, send() pada kirim_ke() (baris 87) memblokir. Karena S2 hanya punya satu thread, seluruh event loop berhenti di dalam send() tersebut: korban tidak menerima apa pun selama 10,119564 - 4,904049 = 5,215 detik, pengirim tidak dibaca sehingga server mengiklankan ZeroWindow kepadanya, dan klien baru tidak di-accept walaupun handshake-nya sudah selesai.

Keluaran ss saat pelayanan tertahan menunjukkan Send-Q socket lambat sebesar 1.790.488 byte, Recv-Q socket pengirim sebesar 22.728 byte, dan satu koneksi menunggu di listener. Log tidak memuat PESAN selama interval sekitar lima detik. Setelah lambat mengirim RST, operasi send() yang tertahan gagal dan event loop kembali berjalan. Siaran kepada korban berlanjut sekitar 2,2 ms setelah RST, sedangkan prompt klien baru dikirim sekitar 5,4 ms setelahnya. SIGPIPE yang diabaikan membantu mencegah proses berhenti akibat penulisan ke socket yang telah tertutup.

Expert Info mencatat 44 TCP Window Full, 30 TCP Zero Window segment, 8 probe, dan 2 RST. Di akhir skenario, pengirim diputus dengan alasan protokol/permintaan keluar. Penyebab yang paling mungkin adalah send() non-blocking pada klien penguji yang sesekali hanya terkirim sebagian, sehingga sisa baris tergabung dengan baris berikutnya dan melebihi 2048 byte tanpa newline. Ini adalah artefak klien penguji, bukan bagian dari temuan send blocking, dan perilaku server terhadapnya sama dengan subbab 2.5.9.

Kesimpulan eksperimen ini: poll() menyelesaikan masalah klien yang diam saat server menunggu data, tetapi tidak menyelesaikan masalah klien yang tidak membaca saat server mengirim. Untuk itu socket perlu dibuat non-blocking, setiap klien diberi buffer keluar, dan POLLOUT dipantau hanya ketika buffer keluar berisi data.

2.6 Analisis source code

2.6.1 Pembuatan listener pada kedua program

buat_listener() membuat socket AF_INET dan SOCK_STREAM, mengaktifkan SO_REUSEADDR, mengisi sockaddr_in, mengubah port dengan htons(), mengubah teks alamat dengan inet_pton(), lalu menjalankan bind() dan listen(). SO_REUSEADDR membantu penggunaan kembali alamat pada keadaan tertentu, tetapi tidak mengizinkan dua server ini bebas mengikat endpoint identik secara bersamaan.

Alamat 127.0.0.1 membatasi akses ke loopback lingkungan server. Parameter listen(ls, 16) berkaitan dengan backlog koneksi yang menunggu diterima, bukan pernyataan bahwa hanya 16 pengguna aktif boleh dilayani. Pada S2, batas koneksi aplikasi ditentukan tersendiri oleh MAKS_KLIEN sebesar 32.

2.6.2 Alur S1 dan titik blocking

```
accept() -> recv() -> send() -> recv() berulang
         -> klien selesai -> close() -> accept() berikutnya
```

Loop for di main() menerima satu cs melalui accept(). Informasi alamat klien dicetak untuk log. Program lalu memakai buffer 512 byte dan menjalankan recv() berulang sampai hasilnya nol atau negatif. recv() bernilai nol berarti peer menutup arah pengirimannya secara tertib, nilai negatif menunjukkan kegagalan yang diperiksa melalui errno.

```
while ((n = recv(cs, buf, sizeof buf, 0)) > 0) {
    ssize_t w = send(cs, buf, (size_t)n, 0);
    if (w < 0) { perror("send"); break; }
}
```

Selama loop tersebut berlangsung, accept() tidak dipanggil lagi. Pesan B dapat masuk antrean TCP, tetapi aplikasi hanya memakai cs milik A. Setelah A keluar, close(cs) dijalankan, iterasi berikutnya menerima B, lalu membaca data B yang telah menunggu. Inilah hubungan langsung source code dengan urutan paket S1.

S1 belum menangani send() parsial: jika hasil send() positif tetapi lebih kecil dari n, sisa data tidak dikirim ulang. S1 juga tidak mengabaikan SIGPIPE, sehingga penulisan ke koneksi yang sudah tertutup dapat menghentikan proses pada kondisi tertentu. Percobaan pesan pendek berhasil, tetapi tidak membuktikan seluruh kondisi kegagalan sudah ditangani.

2.6.3 Penyimpanan status klien pada S2

Struktur Klien menyimpan fd, nama sepanjang 17 byte termasuk terminator NUL, penanda terdaftar, buffer masuk 2048 byte, in_len, dan asal koneksi. Array klien[32] serta n_klien menyimpan semua koneksi yang diterima, termasuk koneksi yang belum mengirim nama. Karena itu 32 adalah batas koneksi, bukan hanya jumlah nama yang sudah terdaftar.

Buffer per klien menjaga data dari beberapa koneksi tetap terpisah. Berbeda dengan S1 yang hanya memiliki satu socket klien dalam alur pelayanannya, S2 mempertahankan status koneksi di antara iterasi event loop.

2.6.4 Event loop poll

```
susun pfd listener dan klien -> poll(..., -1)
-> periksa kejadian klien -> recv dan rakit baris
-> terima koneksi baru jika listener siap
-> susun ulang pfd pada iterasi berikutnya
```

pfd[0] digunakan untuk listener dan pfd[i+1] untuk klien. Semua meminta POLLIN. poll() dengan timeout -1 menunggu sampai ada kejadian atau interupsi. Hasil revents digunakan untuk membedakan kesiapan baca, error, socket tidak valid, dan hangup. poll() sendiri juga dapat menunggu secara blocking, tetapi menunggu banyak socket sekaligus sehingga tidak terikat pada satu klien yang diam.

Loop klien diproses dari indeks terbesar ke terkecil. Ketika suatu klien ditutup, elemen terakhir dipindahkan untuk mengisi posisi yang kosong. Urutan mundur mencegah elemen yang baru dipindah diproses memakai revents milik

socket lain dalam iterasi yang sama. Listener diproses sesudah kejadian klien dan menerima satu koneksi baru ketika POLLIN aktif.

S2 tidak membuat fork() atau pthread. Konkurensi pelayanan diperoleh dari pergantian penanganan kejadian dalam satu thread. Tidak semua kegiatan berjalan bersamaan pada CPU. program tetap menjalankan handler secara berurutan.

2.6.5 Perakitan baris dan protokol aplikasi

rakit_baris() mencari newline dengan memchr(). Baris utuh diproses, sisa buffer digeser dengan memmove(), lalu pemeriksaan berulang untuk menangani beberapa baris dalam satu recv(). Jika newline belum ada, byte disimpan untuk recv() berikutnya. CR sebelum LF dibuang agar klien berformat CRLF dapat digunakan. Buffer penuh tanpa newline menyebabkan penolakan baris terlalu panjang.

Dengan buffer 2048 byte, satu baris dapat menampung paling banyak 2047 byte sebelum LF jika buffer tidak memuat data lain. Batas ini bekerja pada byte, bukan karakter Unicode. Data yang dikirim tanpa Enter dapat belum diproses karena protokol membutuhkan delimiter newline. S1 tidak mempunyai aturan registrasi atau delimiter serupa.

proses_baris() terlebih dahulu memeriksa status terdaftar. Baris pertama divalidasi menjadi nama 1 sampai 16 karakter yang diterima isalnum(), underscore, atau tanda minus. Nama yang sama menurut strcmp() ditolak. Pembandingan bersifat peka huruf besar kecil. Setelah registrasi, baris kosong diabaikan, /quit meminta penutupan, /who meminta daftar, dan teks lain diperlakukan sebagai chat.

2.6.6 Pembentukan dan penyiaran pesan

```
snprintf(keluar, sizeof keluar, "[%s] %s\n", klien[i].nama, baris);
siar(i, keluar);
```

snprintf() menyusun nama pengirim dan isi pesan ke dalam buffer keluar dengan ukuran yang dibatasi. Fungsi siar() kemudian meneruskannya kepada klien terdaftar selain pengirim. Pada paket 34 dan 35, pesan 15 byte dari Aprilia menjadi 25 byte setelah ditambah awalan nama dan diteruskan melalui koneksi cristyana. Bukti ini menghubungkan pembentukan format pesan pada source code dengan payload yang diamati di Wireshark.

kirim_ke() memakai loop selama masih ada byte tersisa. Setelah send() berhasil, pointer maju dan sisa berkurang. Jika terinterupsi EINTR, pengiriman dicoba lagi. Ini lebih lengkap daripada S1 dalam menangani pengiriman parsial. Namun socket masih blocking dan S2 tidak memakai antrean output serta POLLOUT.

2.6.7 Penutupan dan penanganan kegagalan

tutup_klien() mencatat alasan, menutup fd, menyiarkan notifikasi jika pengguna terdaftar, dan memadatkan array. SIGPIPE diabaikan agar satu kegagalan penulisan tidak langsung mematikan seluruh proses. POLLERR, POLLNVAL, POLLHUP, recv() nol, serta pelanggaran protokol ditangani pada jalur masing-masing.

Ketika send() gagal, kirim_ke() menghentikan pengiriman dan kembali kepada pemanggil. Fungsi tersebut tidak langsung menghapus klien. Pembersihan dilakukan melalui penanganan kejadian socket pada event loop. Selain itu, kode belum menyediakan timeout registrasi maupun idle, sehingga koneksi yang tidak mengirim nama tetap menempati satu slot sampai ditutup atau terdeteksi bermasalah.

2.6.8 Pemetaan baris kode dengan bukti eksperimen

Tabel berikut menghubungkan baris kode pada Lampiran A dan B dengan bukti paket, log, atau ss yang diperoleh. Pemetaan ini menunjukkan bahwa setiap perbedaan perilaku pada Wireshark dapat ditelusuri ke baris kode tertentu.

Baris kode	Isi	Bukti eksperimen
S1 34, S2 77	listen(ls, 16)	s1-backlog: Recv-Q 17, 18 SYN,ACK, 8 klien gagal
S1 48	accept() hanya di loop luar	s1-dua: MASUK B setelah KELUAR A, LISTEN Recv-Q 1
S1 57	char buf[512]	s1-banjir: 1606 kali terima 512 byte, echo 512 byte per segmen
S1 59	while (recv(cs, ...) > 0)	s1-dua tertunda 3,02 s, s1-banjir ZeroWindow 4,78 s
S1 62	send(cs, buf, n, 0) apa adanya	s1-pecah: tiga potongan dipantulkan terpisah
S1 63	perror(send) lalu break	s1-banjir: Connection reset by peer, server tetap hidup

S2 87	send() blocking di kirim_ke()	s2-lambat: seluruh server membeku 5,215 s
S2 96-101	siar() lewati pengirim dan belum terdaftar	s2-chat: frame 40 dan 42, klien diam tidak menerima siaran
S2 119-135	nama_valid() dan nama_dipakai()	s2-nama: tiga penolakan dengan FIN server
S2 164	/quit mengembalikan 0	s2-chat: FIN pertama dari server (frame 55)
S2 166-175	/who kirim_ke() per baris	s2-chat: 4 segmen pada frame 46-52
S2 187-199	memchr, buang CR, memmove	s2-pecah 3 segmen jadi 1 baris, s2-gabung 1 segmen jadi 3 pesan
S2 203-206	buffer penuh tanpa newline	s2-panjang: baris terlalu panjang lalu RST
S2 212	signal(SIGPIPE, SIG_IGN)	s2-lambat: server tetap hidup setelah RST lambat
S2 227	poll(pfd, n + 1, -1)	s2-chat: klien diam tidak menghambat pengguna lain
S2 235	POLLERR atau POLLNVAL	s2-penuh: 32 kali alasan=socket error
S2 261-267	n_klien >= MAKS_KLIEN	s2-penuh: koneksi ke-33 menerima server penuh

2.6.9 Rincian perbedaan struktur kode

Selain perbandingan umum pada subbab 2.7, perbedaan rinci kedua source code dirangkum sebagai berikut.

Aspek	S1 s1_server_iteratif.c	S2 s2_server_poll.c
Panjang kode	73 baris, 1 fungsi bantu	281 baris, 10 fungsi bantu
Header tambahan	Hanya socket dasar	stdarg, ctype, signal, time, poll
Panggilan yang menunggu	accept() dan recv() per klien	poll() menunggu kesiapan banyak socket. send() tetap blocking dan dapat menahan seluruh event loop
Struktur loop	Dua loop bersarang: accept lalu recv	Satu loop datar: susun pfd, poll, layani, accept
Status per klien	Variabel lokal cs, cli, buf	struct Klien: fd, nama, terdaftar, in[2048], in_len, asal
Framing pesan	Tidak ada	rakit_baris() dengan memchr dan memmove
Komunikasi antarklien	Tidak ada	siar() ke semua kecuali pengirim
Batas koneksi	Hanya backlog kernel (17 antre)	MAKS_KLIEN 32 dan pesan server penuh
Pihak yang menutup	Selalu menunggu klien menutup	Klien atau server (quit, nama ditolak, penuh, baris panjang)
Pengecekan setsockopt	Dicek, exit bila gagal	Tidak dicek
Log	printf sederhana tanpa waktu	catat() dengan stempel waktu ISO 8601 UTC
Penanganan EINTR	Pada accept()	Pada poll(), recv(), dan send()

2.6.10 Ketahanan dan keamanan

Masalah	S1	S2	Bukti
Klien diam memonopoli server	Ya	Tidak	s1-dua dan s2-chat
Banjir koneksi	SYN dibuang diam-diam	Ditolak dengan pesan setelah 32	s1-backlog dan s2-penuh
Klien lambat membaca	Belum diuji langsung. send() pada klien aktif dapat menahan pelayanan dan menunda klien lain	Semua klien membeku	s2-lambat
Data tanpa batas	Tidak relevan, dipantulkan	Diputus setelah 2048 byte	s2-panjang

baris			
Overflow buffer	Ukuran recv dibatasi sizeof buf. Tidak membuktikan seluruh aspek keamanan program	Ruang buffer dihitung dan snprintf dibatasi ukuran. Tetap memerlukan validasi dan penanganan kegagalan	Analisis kode
SIGPIPE	Tidak diabaikan	Diabaikan	Baris 212 dan s2-lambat
Validasi input	Tidak ada	Ada	s2-nama
Kerahasiaan	Tidak terenkripsi	Tidak terenkripsi, isi chat terbaca di Wireshark	Semua capture
Slot tanpa timeout	Tidak relevan	Klien diam tanpa nama tetap memakai slot	s2-chat klien diam

2.7 Perbandingan dan evaluasi

Aspek	S1 iteratif	S2 poll
Layanan	Echo byte kepada pengirim	Chat berbasis baris dan nama
Proses dan thread	Satu proses satu thread	Satu proses satu thread
Penerimaan koneksi	accept berikutnya setelah klien selesai	Listener dipantau bersama socket klien
Klien diam	Dapat menahan pelayanan klien lain	Klien lain umumnya tetap diproses
Status klien	Satu socket dalam loop pelayanan	Array status hingga 32 koneksi
Buffer	512 byte penerimaan	2048 byte per klien dan buffer kirim
Delimiter	Tidak ditentukan aplikasi	Newline dan dukungan CRLF
Pengiriman	Satu send per hasil recv	Loop send hingga selesai atau gagal
Fitur	Echo dan log sederhana	Registrasi validasi chat who quit
Penutupan	close setelah loop recv selesai	Handler penutupan dan pemberitahuan
SIGPIPE	Tidak diabaikan	Diabaikan
Skalabilitas	Antrian terhambat satu sesi	Pemindaian poll dan array linear
Risiko	Klien diam dan pengiriman parsial	Pengiriman blocking dan slot tanpa timeout

2.7.1 Interpretasi pengukuran

Pengukuran	Paket	Selisih	Interpretasi
Echo A pada S1	4 ke 6	0,243 ms	Klien yang sedang dilayani
Echo B pada S1	11 ke 16	69,386225 s	Menunggu A selesai
Penerusan chat S2	34 ke 35	0,543 ms	Dari pengirim melalui server ke penerima
Respons awal who S2	42 ke 44	0,250 ms	Respons pertama saat A masih terhubung
Daftar who lengkap S2	42 ke 48	0,326 ms	Sampai baris terakhir daftar

Pengukuran di atas memakai timestamp paket, mencakup penjadwalan proses dan stack jaringan yang tampak pada capture. Satu sampel loopback tidak cukup untuk mengklaim throughput maksimum atau keunggulan kecepatan dengan faktor tertentu. Skenario echo dan who juga mengerjakan operasi berbeda. Kesimpulan yang kuat adalah perbedaan kemampuan melayani koneksi lain selama satu sesi tetap terbuka.

2.7.2 Keterbatasan dan pengembangan

Percobaan manual menggunakan dua pengguna aktif, pesan pendek, dan satu host. Sebagian besar keterbatasan tersebut telah ditutup oleh eksperimen lanjutan pada subbab 2.4.4 sampai 2.4.7 dan 2.5.6 sampai 2.5.11: beban 26 dan 33 koneksi, banjir data, klien lambat membaca, baris lebih dari 2048 byte, pesan terpecah dan tergabung, serta

penutupan dengan RST sudah diuji secara terkontrol. Yang masih belum diukur adalah penggunaan CPU dan memori, throughput maksimum, distribusi latensi dengan pengulangan statistik, serta pengujian antar-host melalui jaringan fisik. Semua pengukuran waktu tetap berasal dari satu kali eksekusi tiap skenario pada loopback.

Perbaikan S1 dapat mencakup loop pengiriman parsial dan penanganan SIGPIPE. Perbaikan S2 dapat mencakup socket nonblocking, antrean keluaran per klien, pemantauan POLLOUT hanya ketika ada data tertunda, timeout registrasi dan idle, serta penanganan kegagalan kirim yang lebih eksplisit. Kedua program juga dapat menambahkan validasi argumen port. Pengembangan tersebut belum diterapkan pada kode yang diuji.

Tulisan checksum unverified pada Wireshark menyatakan verifikasi belum dilakukan, bukan kesimpulan paket rusak. Header Ethernet dengan alamat nol pada capture loopback juga tidak boleh ditafsirkan sebagai bukti komunikasi fisik melalui adaptor Ethernet. Fokus analisis berada pada koneksi IP TCP dan payload aplikasi.

2.7.3 Perbandingan terukur hasil eksperimen lanjutan

Pengamatan	S1 terukur	S2 terukur
Handshake klien kedua	Sukses oleh kernel	Sukses oleh kernel
Data klien kedua saat klien pertama aktif	Hanya di-ACK. echo tertunda 3,022 s	Diproses dalam 1 sampai 3 ms
Data pertama dari server	Echo setelah klien mengirim	Prompt nama 1 sampai 3 ms setelah handshake
Pola pesan pada skenario yang diuji	Potongan dipantulkan. Tidak menjamin rasio segmen TCP selalu 1 banding 1	Satu pesan chat dikirim kepada N - 1 pengguna terdaftar lain, bukan rasio segmen TCP yang tetap
Pesan terpecah 3 segmen	3 echo terpisah	1 baris utuh setelah newline
3 baris dalam 1 segmen	Tidak diuji, akan dipantulkan apa adanya	3 pesan terpisah, CR dibuang
ZeroWindow ke klien yang mengirim	34 kali, macet 4,78 s	Terjadi hanya saat server membeku (s2-lambat)
Koneksi melebihi kapasitas	8 klien gagal, 48 SYN diulang	Koneksi ke-33 ditolak eksplisit dalam 0,1 ms
Klien lambat membaca	Tidak diuji pada S1	Semua klien membeku 5,215 s
FIN pertama	Selalu dari klien	Dari server pada quit, nama ditolak, dan server penuh
Penutupan dengan RST oleh server	Tidak teramati	Baris terlalu panjang dengan 52 byte belum dibaca

Kedua server berjalan dalam satu proses dan satu thread. S1 menunggu satu socket klien dalam loop penerimaan, sedangkan S2 memantau kesiapan banyak socket melalui poll(). Pada S2, klien yang diam saat pengiriman data masuk tidak menahan pelayanan klien lain. Akan tetapi, pengiriman ke penerima yang tidak membaca tetap dapat memenuhi buffer dan menahan send(), sebagaimana dibuktikan pada eksperimen s2-lambat.

2.8 Kesimpulan

Server S1 bekerja sebagai echo iteratif. Koneksi B dapat menyelesaikan handshake dan memperoleh ACK TCP, tetapi aplikasi baru mengirim echo setelah A keluar. Capture membuktikan pesan B pada paket 11 dan echo pada paket 16 berjarak 69,386225 detik, dengan penutupan A di antaranya. Perilaku ini dijelaskan oleh loop recv() S1 yang menahan program sebelum accept() berikutnya.

Server S2 mempertahankan banyak koneksi melalui poll() dalam satu proses dan satu thread. Chat Aprilia ke cristyana diteruskan dengan penambahan nama, nama duplikat ditolak, who menampilkan daftar aktual, dan quit memperbarui status pengguna. Respons who pertama terjadi 0,250 ms setelah permintaan ketika Aprilia masih terhubung. S2 mengatasi hambatan klien diam pada skenario S1, tetapi pengiriman blocking tetap dapat menghambat event loop pada kondisi penerima lambat.

Dengan demikian, perbedaan utama kedua implementasi terletak pada cara menunggu dan memilih koneksi yang dilayani, disertai perbedaan protokol echo dan chat. Bukti paket perlu dibaca bersama log terminal serta source code agar keberhasilan handshake atau ACK tidak keliru dianggap sebagai keberhasilan pelayanan aplikasi.

Eksperimen lanjutan memperkuat hasil manual melalui pengamatan antrean kernel dan flow control. Pada S1, koneksi kedua dapat menunggu di antrean accept sementara datanya berada dalam receive buffer. Ketika data terus dikirim tanpa dibaca, capture menunjukkan ZeroWindow. Pada uji backlog di lingkungan ini, 17 koneksi mengantre dan delapan upaya koneksi lain belum berhasil selama interval pengamatan. Temuan tersebut menunjukkan hubungan antara struktur loop aplikasi, kapasitas antrean, dan perilaku TCP.

Pada S2, buffer per klien dan rakit_baris() berhasil mengolah pesan terpecah maupun beberapa baris dalam satu penerimaan. Validasi nama dan batas 32 koneksi menghasilkan penolakan yang eksplisit. Buffer yang penuh tanpa newline juga memicu penutupan klien. Eksperimen s2-lambat menunjukkan bahwa satu penerima yang tidak membaca dapat menahan seluruh event loop selama sekitar 5,215 detik. Pengembangan berikutnya memerlukan socket nonblocking, buffer keluar per klien, dan pemantauan POLLOUT ketika masih ada data yang perlu dikirim.

2.9 Lampiran A: Kode lengkap S1

s1_server_iteratif.c disalin lengkap dari file sumber. Nomor baris di sisi kiri hanya untuk membantu pembacaan dan bukan bagian kode program.

```
001  /* s1_server_iteratif.c — Tahap 1: server ECHO satu klien pada satu waktu.
002   * Tujuan: menunjukkan MENGAPA server iteratif gagal melayani banyak pengguna.
003   * Kompilasi: gcc -Wall -Wextra -O2 -o s1_server s1_server_iteratif.c
004   * Jalankan  : ./s1_server 5000
005   */
006  #include <stdio.h>
007  #include <stdlib.h>
008  #include <string.h>
009  #include <unistd.h>
010  #include <errno.h>
011  #include <netinet/in.h>
012  #include <arpa/inet.h>
013  #include <sys/socket.h>
014
015  static int buat_listener(const char *ip, int port)
016  {
017      int ls = socket(AF_INET, SOCK_STREAM, 0);
018      if (ls < 0) { perror("socket"); exit(1); }
019
020      int yes = 1;
021      if (setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) < 0) {
022          perror("setsockopt"); exit(1);
023      }
024
025      struct sockaddr_in addr;
026      memset(&addr, 0, sizeof addr);
027      addr.sin_family = AF_INET;
028      addr.sin_port   = htons((uint16_t)port);
029      if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
030          fprintf(stderr, "alamat tidak valid: %s\n", ip); exit(1);
031      }
032
033      if (bind(ls, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); exit(1); }
034      if (listen(ls, 16) < 0) { perror("listen"); exit(1); }
035      return ls;
036  }
037
038  int main(int argc, char **argv)
039  {
040      int port = (argc > 1) ? atoi(argv[1]) : 5000;
041      int ls = buat_listener("127.0.0.1", port);
042      printf("[server] listen di 127.0.0.1:%d (iteratif)\n", port);
043      fflush(stdout);
044
045      for (;;) {
046          struct sockaddr_in cli;
047          socklen_t clen = sizeof cli;
048          int cs = accept(ls, (struct sockaddr *)&cli, &clen);
049          if (cs < 0) { if (errno == EINTR) continue; perror("accept"); break; }
050
051          char ipstr[INET_ADDRSTRLEN];
052          inet_ntop(AF_INET, &cli.sin_addr, ipstr, sizeof ipstr);
```

```
053         printf("[server] MASUK  %s:%u\n", ipstr, ntohs(cli.sin_port));
054         fflush(stdout);
055
056         /* Selama loop ini berjalan, accept() TIDAK dipanggil lagi. */
057         char buf[512];
058         ssize_t n;
059         while ((n = recv(cs, buf, sizeof buf, 0)) > 0) {
060             printf("[server] terima %zd byte dari %s:%u\n", n, ipstr, ntohs(cli.sin_port));
061             fflush(stdout);
062             ssize_t w = send(cs, buf, (size_t)n, 0);   /* echo balik */
063             if (w < 0) { perror("send"); break; }
064         }
065         if (n < 0) perror("recv");
066
067         printf("[server] KELUAR %s:%u\n", ipstr, ntohs(cli.sin_port));
068         fflush(stdout);
069         close(cs);
070     }
071     close(ls);
072     return 0;
073 }
```

2.10 Lampiran B: Kode lengkap S2

s2_server_poll.c disalin lengkap dari file sumber. Nomor baris di sisi kiri hanya untuk membantu pembacaan dan bukan bagian kode program.

```
001  /* s2_server_poll.c — Tahap 2: server chat multi-pengguna dengan poll().
002   * Satu proses, satu thread, banyak koneksi.
003   * Protokol: teks baris, dipisah '\n'. Baris pertama = nama pengguna.
004   * Kompilasi: gcc -Wall -Wextra -O2 -o s2_server s2_server_poll.c
005   * Jalankan  : ./s2_server 5000
006   */
007  #include <stdio.h>
008  #include <stdlib.h>
009  #include <stdarg.h>
010  #include <string.h>
011  #include <ctype.h>
012  #include <unistd.h>
013  #include <errno.h>
014  #include <signal.h>
015  #include <time.h>
016  #include <poll.h>
017  #include <netinet/in.h>
018  #include <arpa/inet.h>
019  #include <sys/socket.h>
020
021  #define MAKS_KLIEN   32
022  #define BUF_MASUK    2048
023  #define MAKS_NAMA    17          /* 16 karakter + NUL */
024  #define BUF_KIRIM    (BUF_MASUK + MAKS_NAMA + 64)
025
026  typedef struct {
027      int    fd;
028      char   nama[MAKS_NAMA];
029      int    terdaftar;            /* 0 = belum mengirim nama */
030      char   in[BUF_MASUK];        /* buffer perakit baris */
031      size_t in_len;
032      char   asal[INET_ADDRSTRLEN + 8];
033  } Klien;
034
035  static Klien klien[MAKS_KLIEN];
036  static int   n_klien = 0;
037
038  /* ---------- utilitas ---------- */
039
040  static const char *stempel(void)
041  {
042      static char buf[32];
043      time_t t = time(NULL);
044      struct tm tm;
045      gmtime_r(&t, &tm);
046      strftime(buf, sizeof buf, "%Y-%m-%dT%H:%M:%SZ", &tm);
047      return buf;
048  }
049
050  static void catat(const char *fmt, ...)
051  {
052      va_list ap;
053      fprintf(stdout, "%s ", stempel());
054      va_start(ap, fmt);
055      vfprintf(stdout, fmt, ap);
056      va_end(ap);
057      fputc('\n', stdout);
058      fflush(stdout);
059  }
060
061  static int buat_listener(const char *ip, int port)
062  {
063      int ls = socket(AF_INET, SOCK_STREAM, 0);
064      if (ls < 0) { perror("socket"); exit(1); }
```

```
065
066     int yes = 1;
067     setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
068
069     struct sockaddr_in addr;
070     memset(&addr, 0, sizeof addr);
071     addr.sin_family = AF_INET;
072     addr.sin_port   = htons((uint16_t)port);
073     if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
074         fprintf(stderr, "alamat tidak valid: %s\n", ip); exit(1);
075     }
076     if (bind(ls, (struct sockaddr *)&addr, sizeof addr) < 0) { perror("bind"); exit(1); }
077     if (listen(ls, 16) < 0) { perror("listen"); exit(1); }
078     return ls;
079 }
080
081 /* Tahap 2 memakai send() blocking. Kelemahannya dibahas di Tahap 4. */
082 static void kirim_ke(int i, const char *teks)
083 {
084     size_t sisa = strlen(teks);
085     const char *p = teks;
086     while (sisa > 0) {
087         ssize_t n = send(klien[i].fd, p, sisa, 0);
088         if (n <= 0) {
089             if (n < 0 && errno == EINTR) continue;
090             return;                     /* klien akan dibersihkan pada iterasi berikut */
091         }
092         p += n; sisa -= (size_t)n;
093     }
094 }
095
096 static void siar(int kecuali, const char *teks)
097 {
098     for (int i = 0; i < n_klien; i++)
099         if (i != kecuali && klien[i].terdaftar)
100             kirim_ke(i, teks);
101 }
102
103 static void tutup_klien(int i, const char *alasan)
104 {
105     catat("KELUAR %s nama=%s alasan=%s",
106           klien[i].asal, klien[i].terdaftar ? klien[i].nama : "-", alasan);
107     close(klien[i].fd);
108
109     if (klien[i].terdaftar) {
110         char baris[BUF_KIRIM];
111         snprintf(baris, sizeof baris, "*** %s meninggalkan ruang chat\n", klien[i].nama);
112         klien[i].terdaftar = 0;
113         siar(i, baris);
114     }
115     klien[i] = klien[n_klien - 1];       /* padatkan array */
116     n_klien--;
117 }
118
119 static int nama_valid(const char *s)
120 {
121     size_t n = strlen(s);
122     if (n < 1 || n > MAKS_NAMA - 1) return 0;
123     for (size_t i = 0; i < n; i++) {
124         unsigned char c = (unsigned char)s[i];
125         if (!isalnum(c) && c != '_' && c != '-') return 0;
126     }
127     return 1;
128 }
129
130 static int nama_dipakai(const char *s)
131 {
132     for (int i = 0; i < n_klien; i++)
133         if (klien[i].terdaftar && strcmp(klien[i].nama, s) == 0) return 1;
```

```
    return 0;
}

/* Mengembalikan 0 bila klien harus ditutup. */
static int proses_baris(int i, char *baris)
{
    char keluar[BUF_KIRIM];

    if (!klien[i].terdaftar) {
        if (!nama_valid(baris)) {
            kirim_ke(i, "!!! nama tidak valid (1-16 karakter: huruf, angka, _ atau -)\n");
            return 0;
        }
        if (nama_dipakai(baris)) {
            kirim_ke(i, "!!! nama sudah dipakai\n");
            return 0;
        }
        snprintf(klien[i].nama, sizeof klien[i].nama, "%s", baris);
        klien[i].terdaftar = 1;
        catat("DAFTAR %s nama=%s", klien[i].asal, klien[i].nama);

        snprintf(keluar, sizeof keluar, "*** selamat datang, %s\n", klien[i].nama);
        kirim_ke(i, keluar);
        snprintf(keluar, sizeof keluar, "*** %s bergabung ke ruang chat\n", klien[i].nama);
        siar(i, keluar);
        return 1;
    }

    if (baris[0] == '\0') return 1;                /* baris kosong diabaikan */

    if (strcmp(baris, "/quit") == 0) return 0;

    if (strcmp(baris, "/who") == 0) {
        kirim_ke(i, "*** pengguna online:\n");
        for (int j = 0; j < n_klien; j++) {
            if (!klien[j].terdaftar) continue;
            snprintf(keluar, sizeof keluar, "***   %s%s\n",
                     klien[j].nama, j == i ? " (Anda)" : "");
            kirim_ke(i, keluar);
        }
        return 1;
    }

    catat("PESAN  nama=%s panjang=%zu", klien[i].nama, strlen(baris));
    snprintf(keluar, sizeof keluar, "[%s] %s\n", klien[i].nama, baris);
    siar(i, keluar);
    return 1;
}

/* Ambil sebanyak mungkin baris utuh dari buffer. 0 = klien harus ditutup. */
static int rakit_baris(int i)
{
    for (;;) {
        char *nl = memchr(klien[i].in, '\n', klien[i].in_len);
        if (!nl) break;

        *nl = '\0';
        size_t panjang = (size_t)(nl - klien[i].in);
        if (panjang > 0 && klien[i].in[panjang - 1] == '\r')
            klien[i].in[panjang - 1] = '\0';        /* klien telnet mengirim CRLF */

        int lanjut = proses_baris(i, klien[i].in);

        size_t dipakai = panjang + 1;
        memmove(klien[i].in, klien[i].in + dipakai, klien[i].in_len - dipakai);
        klien[i].in_len -= dipakai;

        if (!lanjut) return 0;
    }
```

```
    if (klien[i].in_len == BUF_MASUK) {             /* penuh tanpa '\n' */
        kirim_ke(i, "!!! baris terlalu panjang\n");
        return 0;
    }
    return 1;
}

int main(int argc, char **argv)
{
    signal(SIGPIPE, SIG_IGN);                       /* tanpa ini server mati saat menulis ke socket tertutup */

    int port = (argc > 1) ? atoi(argv[1]) : 5000;
    int ls = buat_listener("127.0.0.1", port);
    catat("LISTEN 127.0.0.1:%d maks_klien=%d", port, MAKS_KLIEN);

    for (;;) {
        struct pollfd pfd[MAKS_KLIEN + 1];
        pfd[0].fd = ls; pfd[0].events = POLLIN; pfd[0].revents = 0;
        for (int i = 0; i < n_klien; i++) {
            pfd[i + 1].fd = klien[i].fd;
            pfd[i + 1].events = POLLIN;
            pfd[i + 1].revents = 0;
        }

        int siap = poll(pfd, (nfds_t)(n_klien + 1), -1);
        if (siap < 0) { if (errno == EINTR) continue; perror("poll"); break; }

        /* Klien diproses dari belakang supaya penghapusan tidak merusak indeks. */
        for (int i = n_klien - 1; i >= 0; i--) {
            short re = pfd[i + 1].revents;
            if (re == 0) continue;

            if (re & (POLLERR | POLLNVAL)) { tutup_klien(i, "socket error"); continue; }

            if (re & POLLIN) {
                size_t ruang = BUF_MASUK - klien[i].in_len;
                ssize_t n = recv(klien[i].fd, klien[i].in + klien[i].in_len, ruang, 0);
                if (n == 0)      { tutup_klien(i, "ditutup klien"); continue; }
                if (n < 0) {
                    if (errno == EINTR || errno == EAGAIN) continue;
                    tutup_klien(i, strerror(errno)); continue;
                }
                klien[i].in_len += (size_t)n;
                if (!rakit_baris(i)) { tutup_klien(i, "protokol/permintaan keluar"); continue; }
            } else if (re & POLLHUP) {
                tutup_klien(i, "hangup"); continue;
            }
        }

        if (pfd[0].revents & POLLIN) {
            struct sockaddr_in cli;
            socklen_t clen = sizeof cli;
            int cs = accept(ls, (struct sockaddr *)&cli, &clen);
            if (cs < 0) { perror("accept"); continue; }

            char ipstr[INET_ADDRSTRLEN];
            inet_ntop(AF_INET, &cli.sin_addr, ipstr, sizeof ipstr);

            if (n_klien >= MAKS_KLIEN) {
                const char *tolak = "!!! server penuh\n";
                send(cs, tolak, strlen(tolak), 0);
                close(cs);
                catat("TOLAK  %s:%u alasan=penuh", ipstr, ntohs(cli.sin_port));
                continue;
            }

            Klien *k = &klien[n_klien];
            memset(k, 0, sizeof *k);
```

```
271             k->fd = cs;
272             snprintf(k->asal, sizeof k->asal, "%s:%u", ipstr, ntohs(cli.sin_port));
273             n_klien++;
274 
275             catat("MASUK  %s total=%d", k->asal, n_klien);
276             kirim_ke(n_klien - 1, "*** ketik nama Anda lalu ENTER: ");
277         }
278     }
279     close(ls);
280     return 0;
281 }
```

2.11 Lampiran C: Program klien penguji uji_klien.py

uji_klien.py disalin lengkap. Program hanya memakai pustaka standar Python 3.

```
#!/usr/bin/env python3
"""uji_klien.py - klien penguji untuk s1_server_iteratif.c (S1) dan s2_server_poll.c (S2).
Hanya memakai pustaka standar Python. Jalankan sambil merekam (tcpdump -i lo tcp port 5000).

  S1 (server iteratif, echo)
    s1-dua      dua klien serentak, B mengirim saat A masih aktif
    s1-pecah    satu baris dikirim dalam 3 segmen -> S1 memantulkan 3 potongan
    s1-banjir   A diam, B mengirim 64 KB terus-menerus (TCP ZeroWindow)
    s1-backlog  A menahan server, 25 koneksi baru serentak (SYN diulang)
  S2 (server chat poll)
    s2-chat     klien 'diam' (tidak kirim nama) + budi, sari, andi: chat, /who, /quit
    s2-nama     nama tidak valid dan nama ganda
    s2-pecah    satu baris dalam 3 segmen -> digabung jadi 1 baris
    s2-gabung   tiga baris dalam 1 segmen -> dipecah jadi 3 pesan
    s2-panjang  baris > 2048 byte tanpa newline -> "baris terlalu panjang"
    s2-penuh    33 koneksi serentak (batas MAKS_KLIEN = 32)
    s2-lambat   satu klien tidak pernah membaca -> send() blocking membekukan server
Opsi: --host 127.0.0.1 --port 5000
"""
import argparse
import socket
import sys
import threading
import time

T0 = time.time()


def log(tag, msg):
    print(f"{time.time() - T0:7.3f}s [{tag}] {msg}", flush=True)


def sambung(h, p, tag, rcvbuf=None):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    if rcvbuf:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_RCVBUF, rcvbuf)
    s.settimeout(5)
    s.connect((h, p))
    s.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
    log(tag, f"terhubung dari port lokal {s.getsockname()[1]}")
    return s


def baca(s, tag, tunggu=0.5, tampil=True):
    s.settimeout(tunggu)
    data = b""
    try:
        while True:
            b = s.recv(65536)
            if not b:
                break
            data += b
    except (socket.timeout, BlockingIOError):
        pass
    except ConnectionError as e:
        log(tag, f"<< koneksi error: {e}")
    if tampil:
        if data:
            for baris in data.decode(errors="replace").splitlines():
                log(tag, f"<< {baris[:90]}{'...' if len(baris) > 90 else ''}")
        else:
            log(tag, "<< (tidak ada balasan)")
    return data


def kirim(s, tag, teks):
    tampil = teks if len(teks) <= 60 else teks[:40] + f"... ({len(teks)} byte)"
    log(tag, f">> {tampil!r}")
    s.sendall(teks.encode())
```

```
070
071
072 def daftar(h, p, nama, rcvbuf=None):
073     s = sambung(h, p, nama, rcvbuf)
074     baca(s, nama)
075     kirim(s, nama, nama + "\n")
076     baca(s, nama)
077     return s
078
079 # ------------------------------------------------------------------ S1
080
081
082 def s1_dua(h, p):
083     a = sambung(h, p, "A")
084     kirim(a, "A", "halo dari A\n"); baca(a, "A")
085     b = sambung(h, p, "B")
086     t_kirim = time.time()
087     kirim(b, "B", "halo dari B\n"); baca(b, "B", 2.5)
088     kirim(a, "A", "A masih di sini\n"); baca(a, "A")
089     log("A", "menutup koneksi (FIN)"); a.close()
090     baca(b, "B", 1.0)
091     log("B", f"echo B diterima +{time.time() - t_kirim:.3f}s setelah dikirim")
092     b.close()
093
094
095 def s1_pecah(h, p):
096     a = sambung(h, p, "A")
097     for potong in ["sel", "amat pa", "gi\n"]:
098         kirim(a, "A", potong); baca(a, "A", 0.5)
099     a.close()
100
101
102 def s1_banjir(h, p):
103     a = sambung(h, p, "A")
104     b = sambung(h, p, "B")
105     b.setblocking(False)
106     total, blok, t0 = 0, b"X" * 65536, time.time()
107     while time.time() - t0 < 5:
108         try:
109             total += b.send(blok)
110         except BlockingIOError:
111             time.sleep(0.1)
112     log("B", f"berhasil menitip {total} byte ke kernel sebelum macet (lihat TCP ZeroWindow)")
113     log("A", "menutup koneksi"); a.close()
114     time.sleep(1); b.close()
115
116
117 def s1_backlog(h, p, n=25):
118     a = sambung(h, p, "A")
119     soket = []
120     for _ in range(n):
121         s = socket.socket(); s.setblocking(False)
122         s.connect_ex((h, p)); soket.append(s)
123     log("main", f"{n} SYN dikirim; tunggu 8 detik")
124     time.sleep(8)
125     for s in soket:
126         s.close()
127     a.close()
128
129 # ------------------------------------------------------------------ S2
130
131
132 def s2_chat(h, p):
133     diam = sambung(h, p, "diam")            # tersambung tetapi tidak pernah mengirim nama
134     baca(diam, "diam")
135     budi = daftar(h, p, "budi")
136     sari = daftar(h, p, "sari"); baca(budi, "budi")
137     andi = daftar(h, p, "andi"); baca(budi, "budi"); baca(sari, "sari")
138     kirim(budi, "budi", "halo semua\n")
139     baca(sari, "sari"); baca(andi, "andi"); baca(budi, "budi"); baca(diam, "diam")
140     kirim(sari, "sari", "/who\n"); baca(sari, "sari")
141     kirim(andi, "andi", "/quit\n"); baca(andi, "andi")
142     baca(budi, "budi"); baca(sari, "sari")
```

```
    budi.close(); baca(sari, "sari")
    sari.close(); diam.close()


def s2_nama(h, p):
    s = sambung(h, p, "x1"); baca(s, "x1")
    kirim(s, "x1", "nama spasi!\n"); baca(s, "x1"); s.close()
    s = sambung(h, p, "x2"); baca(s, "x2")
    kirim(s, "x2", "nama_yang_terlalu_panjang\n"); baca(s, "x2"); s.close()
    r = daftar(h, p, "rina")
    s = sambung(h, p, "x3"); baca(s, "x3")
    kirim(s, "x3", "rina\n"); baca(s, "x3"); s.close()
    r.close()


def s2_pecah(h, p):
    a = daftar(h, p, "penerima")
    b = daftar(h, p, "pengirim"); baca(a, "penerima")
    for potong in ["sel", "amat pa", "gi\n"]:
        kirim(b, "pengirim", potong); time.sleep(0.5)
    baca(a, "penerima")
    a.close(); b.close()


def s2_gabung(h, p):
    a = daftar(h, p, "penerima")
    b = daftar(h, p, "pengirim"); baca(a, "penerima")
    kirim(b, "pengirim", "satu\r\ndua\ntiga\n")    # 3 baris, 1 send, termasuk CRLF
    baca(a, "penerima")
    a.close(); b.close()


def s2_panjang(h, p):
    a = daftar(h, p, "saksi")
    b = daftar(h, p, "panjang"); baca(a, "saksi")
    kirim(b, "panjang", "A" * 2100)               # tanpa '\n'
    baca(b, "panjang", 1.0); baca(a, "saksi")
    b.close(); a.close()


def s2_penuh(h, p, n=33):
    soket = [sambung(h, p, f"k{i:02d}") for i in range(n)]
    baca(soket[-1], f"k{n - 1:02d}", 1.0)
    for s in soket:
        s.close()


def s2_lambat(h, p, durasi=6.0):
    korban = daftar(h, p, "korban")
    lambat = daftar(h, p, "lambat", rcvbuf=4096)  # setelah ini TIDAK PERNAH membaca
    pengirim = daftar(h, p, "pengirim")
    baca(korban, "korban"); baca(lambat, "lambat")

    status = {"byte": 0, "terakhir": None, "henti": False}

    def pembaca():
        korban.settimeout(0.2)
        while not status["henti"]:
            try:
                b = korban.recv(65536)
                if not b:
                    break
                status["byte"] += len(b); status["terakhir"] = time.time()
            except socket.timeout:
                pass
            except OSError:
                break
    th = threading.Thread(target=pembaca, daemon=True); th.start()

    pengirim.setblocking(False)
    baris = ("x" * 1500 + "\n").encode()
    dikirim, t0, baru, t_baru = 0, time.time(), None, None
    log("pengirim", "mulai banjir baris 1501 byte")
```

```
    while time.time() - t0 < durasi:
        try:
            pengirim.send(baris); dikirim += 1
        except BlockingIOError:
            time.sleep(0.05)
        if baru is None and time.time() - t0 > 3.0:
            log("korban", f"menerima {status['byte']} byte; terakhir pada "
                f"{(status['terakhir'] or T0) - T0:.3f}s")
            baru = socket.socket(); baru.settimeout(5)
            baru.connect((h, p)); t_baru = time.time()
            log("baru", f"terhubung (handshake sukses) dari port {baru.getsockname()[1]}; menunggu
prompt...")
            baca(baru, "baru", 1.5)
    log("pengirim", f"{dikirim} baris dititipkan ke kernel")
    log("korban", f"total diterima {status['byte']} byte; terakhir pada {(status['terakhir'] or T0) -
T0:.3f}s")
    log("lambat", "menutup koneksi tanpa membaca (RST)")
    lambat.close()
    data = baca(baru, "baru", 1.5, tampil=False)
    log("baru", f"prompt diterima +{time.time() - t_baru:.3f}s setelah handshake: {data[:40]!r}")
    time.sleep(1.0)
    log("korban", f"setelah lambat keluar total diterima {status['byte']} byte")
    status["henti"] = True
    for s in (pengirim, korban, baru):
        s.close()


SKENARIO = {
    "s1-dua": s1_dua, "s1-pecah": s1_pecah, "s1-banjir": s1_banjir, "s1-backlog": s1_backlog,
    "s2-chat": s2_chat, "s2-nama": s2_nama, "s2-pecah": s2_pecah, "s2-gabung": s2_gabung,
    "s2-panjang": s2_panjang, "s2-penuh": s2_penuh, "s2-lambat": s2_lambat,
}

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("skenario", choices=SKENARIO)
    ap.add_argument("--host", default="127.0.0.1")
    ap.add_argument("--port", type=int, default=5000)
    arg = ap.parse_args()
    try:
        SKENARIO[arg.skenario](arg.host, arg.port)
    except ConnectionError as e:
        log("main", f"koneksi gagal: {e}"); sys.exit(1)
```

2.12 Lampiran D: Skrip pengujian otomatis run_uji.sh

run_uji.sh disalin lengkap. Skrip dijalankan sebagai root di WSL agar tcpdump dapat merekam interface lo.

```
#!/bin/bash
# run_uji.sh - kompilasi, rekam (tcpdump), jalankan server dan klien penguji.
# Dijalankan sebagai root (untuk tcpdump):  wsl -d Ubuntu -u root -- bash run_uji.sh [skenario...]
# Server dan klien dijalankan sebagai pengguna biasa.
set -u
U=windows
SRC="/mnt/d/S2/Jaringan/ANALISIS SERVER"
OUT="$SRC/uji_server/hasil"
KLIEN="$SRC/uji_server/uji_klien.py"
B=/home/$U/uji_build
PORT=5000

mkdir -p "$OUT" && runuser -u $U -- mkdir -p "$B"
cp "$SRC/s1_server_iteratif.c" "$SRC/s2_server_poll.c" "$B/" && chown $U "$B"/*.c
cd "$B"
{
  echo "\$ gcc -Wall -Wextra -O2 -o s1_server s1_server_iteratif.c"
  runuser -u $U -- gcc -Wall -Wextra -O2 -o s1_server s1_server_iteratif.c 2>&1
  echo "\$ gcc -Wall -Wextra -O2 -o s2_server s2_server_poll.c"
  runuser -u $U -- gcc -Wall -Wextra -O2 -o s2_server s2_server_poll.c 2>&1
  echo "exit=$?  ($(gcc --version | head -1))"
} > "$OUT/kompilasi.txt"

ss_snap() {   # $1 = jeda, $2 = berkas
  ( sleep "$1"
    { echo "# ss -ltn  (baris LISTEN: Recv-Q = koneksi antre belum di-accept)"; ss -ltn "sport = :$PORT"
      echo; echo "# ss -tn  (Recv-Q = byte belum dibaca aplikasi, Send-Q = byte belum di-ACK peer)"
      ss -tn "( sport = :$PORT )"; } > "$2" ) &
}

jalankan() {  # $1 = binari server, $2 = skenario, $3 = snaplen
  local srv=$1 sk=$2 snap=${3:-0}
  echo "=== $sk ==="
  tcpdump -i lo -U -nn -s "$snap" -w "$OUT/$sk.pcap" "tcp port $PORT" 2>/dev/null &
  local tp=$!
  sleep 1
  runuser -u $U -- stdbuf -oL ./$srv $PORT > "$OUT/$sk.server.log" 2>&1 &
  sleep 0.5
  case $sk in
    s1-dua)     ss_snap 1.6 "$OUT/$sk.ss.txt" ;;
    s1-banjir)  ss_snap 2.5 "$OUT/$sk.ss.txt" ;;
    s1-backlog) ss_snap 3.0 "$OUT/$sk.ss.txt" ;;
    s2-lambat)  ss_snap 9.0 "$OUT/$sk.ss.txt" ;;
  esac
  runuser -u $U -- python3 -I "$KLIEN" "$sk" --port $PORT > "$OUT/$sk.klien.log" 2>&1
  sleep 1.5
  pkill -x $srv; sleep 0.3
  kill -INT $tp; wait $tp 2>/dev/null
  echo "   pcap: $(stat -c %s "$OUT/$sk.pcap") byte"
}

DAFTAR=${*:-"s1-dua s1-pecah s1-banjir s1-backlog s2-chat s2-nama s2-pecah s2-gabung s2-panjang s2-penuh s2-
lambat"}
for sk in $DAFTAR; do
  case $sk in
    s1-banjir) jalankan s1_server $sk 200 ;;
    s1-*)      jalankan s1_server $sk ;;
    s2-lambat) jalankan s2_server $sk 200 ;;
    s2-*)      jalankan s2_server $sk ;;
  esac
done
chown -R $U "$OUT" 2>/dev/null
echo SELESAI
```

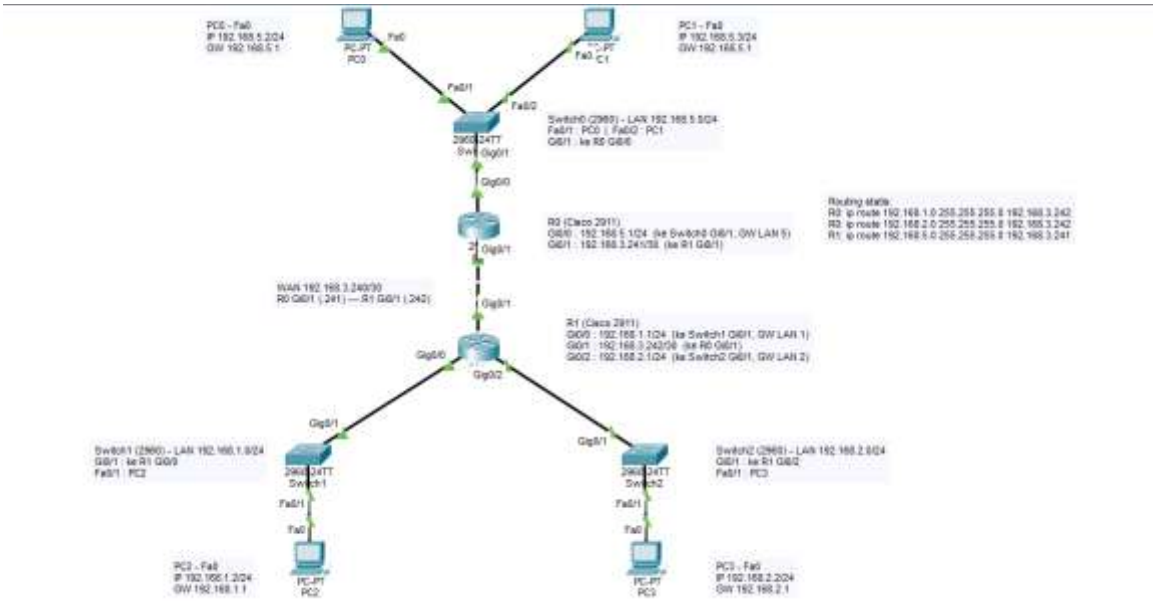
BAB 3 Menguji dan Menganalisa Fungsi Default Gateway

3.1 Tujuan

Menguji ulang susunan topologi di papan tulis dengan routing statis, lalu menganalisa fungsi default gateway dari hasil komunikasi di simulasi. File: Static_Routing_Default_Gateway_v3.pkt.

3.2 Topologi dan pengalamatan

Topologi terdiri dari 2 router Cisco 2911, 3 switch 2960 dan 4 PC dalam tiga LAN yang dihubungkan link WAN /30.



Gambar 3.1 Topologi papan tulis lengkap dengan keterangan interface dan alamat IP

Setiap router mencantumkan interface Gigabit beserta IP dan tujuan kabelnya. R1 memakai tiga interface (Gi0/0, Gi0/1, Gi0/2). Setiap switch mencantumkan LAN-nya dan port yang dipakai: Gi0/1 ke router dan Fa0/x ke PC. Setiap PC mencantumkan port Fa0, IP dan default gateway. Catatan di kanan merangkum tiga route statis.

Perangkat	Interface	Alamat IP	Default gateway
R0	Gi0/0 (LAN 5)	192.168.5.1/24	-
R0	Gi0/1 (ke R1)	192.168.3.241/30	-
R1	Gi0/1 (ke R0)	192.168.3.242/30	-
R1	Gi0/0 (LAN 1)	192.168.1.1/24	-
R1	Gi0/2 (LAN 2)	192.168.2.1/24	-
PC0 / PC1	Fa0	192.168.5.2 / .3 /24	192.168.5.1
PC2	Fa0	192.168.1.2/24	192.168.1.1
PC3	Fa0	192.168.2.2/24	192.168.2.1

```
R0(config)# ip route 192.168.1.0 255.255.255.0 192.168.3.242
R0(config)# ip route 192.168.2.0 255.255.255.0 192.168.3.242
R1(config)# ip route 192.168.5.0 255.255.255.0 192.168.3.241
```

3.3 Proses pembuatan topologi

Cara menaruh perangkat dan menyambung kabel sama seperti BAB 1 (Gambar 1.2 sampai 1.5). yang berbeda adalah jumlah perangkat, model dan pengalamatannya.

1. **Menaruh perangkat:** dua router Cisco 2911 (R0, R1), tiga switch 2960 (Switch0, Switch1, Switch2) dan empat PC (PC0 sampai PC3). Model 2911 dipilih karena R1 membutuhkan tiga port Gigabit.
2. **Menyambung kabel** dengan Copper Straight-Through sesuai tabel berikut.
3. **Mengisi IP PC** lewat Desktop > IP Configuration (contoh PC0 pada Gambar 3.6).
4. **Mengisi IP router dan route statis** lewat CLI (perintah di bawah).

Kabel	Dari	Ke
1	PC0 Fa0	Switch0 Fa0/1
2	PC1 Fa0	Switch0 Fa0/2
3	Switch0 Gi0/1	R0 Gi0/0
4	R0 Gi0/1	R1 Gi0/1
5	R1 Gi0/0	Switch1 Gi0/1
6	R1 Gi0/2	Switch2 Gi0/1
7	PC2 Fa0	Switch1 Fa0/1
8	PC3 Fa0	Switch2 Fa0/1

```
R0(config)# interface GigabitEthernet0/0
R0(config-if)# ip address 192.168.5.1 255.255.255.0
R0(config-if)# no shutdown
R0(config-if)# interface GigabitEthernet0/1
R0(config-if)# ip address 192.168.3.241 255.255.255.252
R0(config-if)# no shutdown
R0(config-if)# exit
R0(config)# ip route 192.168.1.0 255.255.255.0 192.168.3.242
R0(config)# ip route 192.168.2.0 255.255.255.0 192.168.3.242

R1(config)# interface GigabitEthernet0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# interface GigabitEthernet0/1
R1(config-if)# ip address 192.168.3.242 255.255.255.252
R1(config-if)# no shutdown
R1(config-if)# interface GigabitEthernet0/2
R1(config-if)# ip address 192.168.2.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# ip route 192.168.5.0 255.255.255.0 192.168.3.241
```

```
R0#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.5.1     YES manual up                    up
GigabitEthernet0/1     192.168.3.241   YES manual up                    up
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R0#
```

Gambar 3.2 Hasil show ip interface brief di R0

Gi0/0 (192.168.5.1) dan Gi0/1 (192.168.3.241) berstatus up/up. Gi0/2 tidak dipakai di R0, sehingga tetap unassigned dan administratively down.

```
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     192.168.3.242   YES manual up                    up
GigabitEthernet0/2     192.168.2.1     YES unset  up                    up
Vlan1                  unassigned      YES unset  administratively down down
```

Gambar 3.3 Hasil show ip interface brief di R1

Ketiga interface R1 aktif: Gi0/0 = 192.168.1.1 (LAN 1), Gi0/1 = 192.168.3.242 (WAN) dan Gi0/2 = 192.168.2.1 (LAN 2).

```
R0#show running-config | begin interface Gig
interface GigabitEthernet0/0
 ip address 192.168.5.1 255.255.255.0
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.3.241 255.255.255.252
 duplex auto
 speed auto
!
interface GigabitEthernet0/2
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
ip route 192.168.1.0 255.255.255.0 192.168.3.242
ip route 192.168.2.0 255.255.255.0 192.168.3.242
```

Gambar 3.4 Running-config R0: interface dan dua route statis

Link WAN memakai mask 255.255.255.252 (/30), yang hanya menyediakan dua alamat host: .241 dan .242. Dua baris ip route mengarahkan LAN 1 dan LAN 2 ke next hop 192.168.3.242 (R1).

```
R1#show running-config | begin interface Gig
interface GigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 ip address 192.168.3.242 255.255.255.252
 duplex auto
 speed auto
!
interface GigabitEthernet0/2
 ip address 192.168.2.1 255.255.255.0
 duplex auto
 speed auto
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
ip route 192.168.5.0 255.255.255.0 192.168.3.241
```

Gambar 3.5 Running-config R1: tiga interface dan satu route statis

R1 hanya butuh satu route statis, ke LAN 5 lewat 192.168.3.241 (R0), karena LAN 1 dan LAN 2 sudah terhubung langsung ke R1.

3.4 Langkah uji coba

1. Buka file, tunggu semua lampu link hijau.
2. Skenario A, gateway benar: Default Gateway PC0 = 192.168.5.1, lalu ping ke PC1, PC2, PC3 dan `tracert`.
3. Skenario B, gateway kosong: hapus Default Gateway PC0, cek `ipconfig`, ping PC1, PC3 dan PC2.
4. Skenario C, gateway salah: isi 192.168.5.9 (tidak dipakai perangkat mana pun), ping PC3, lalu `arp -a`.
5. Kembalikan gateway ke 192.168.5.1, ping dan `tracert` ulang, lalu `arp -a`.
6. Jalankan `show ip route` di R0 dan R1.
7. Mode Simulation: kirim PDU ICMP dari PC0 ke PC1, PC2 dan PC3, buka detail PDU di Switch0.

8. Mode Realtime: double-click Fire pada enam PDU di daftar kanan bawah (semua pasangan PC). PDU yang Failed pada tembakan pertama ditembak sekali lagi.

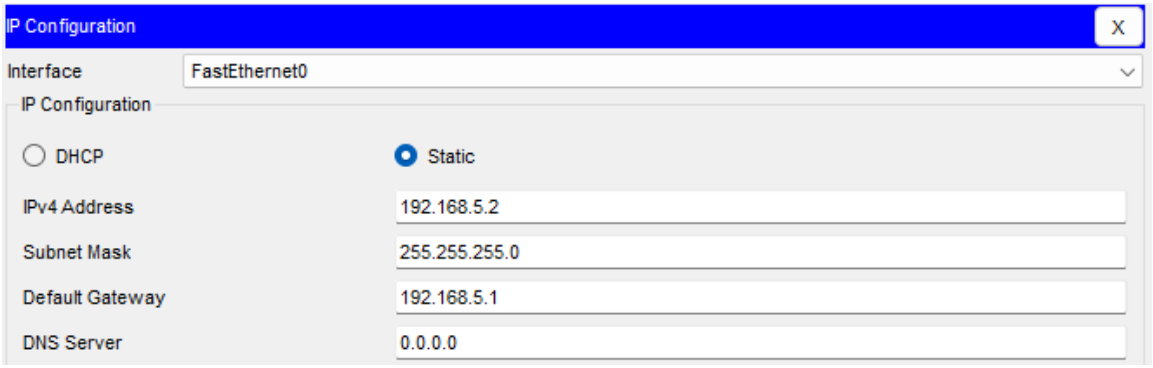
3.5 Hasil skenario

Pada pengujian gateway kosong, PC0 masih dapat berkomunikasi dengan PC1 pada LAN yang sama, tetapi tidak dapat mencapai PC di LAN lain. Ketika gateway benar, komunikasi antar-LAN berhasil. Pada skenario gateway salah, pengujian difokuskan pada PC3 dan memperlihatkan kegagalan pengiriman.

Skenario	Gateway PC0	Ping PC1	Ping PC2	Ping PC3	ARP PC0
A. Benar	192.168.5.1	2/2, TTL=128	2/2, TTL=126	4/4, TTL=126	192.168.5.1 = 0003.e4e0.d301, 192.168.5.3 = 0030.a3b6.3087
B. Kosong	0.0.0.0	2/2, TTL=128	0/2, loss 100%	0/2, loss 100%	-
C. Salah	192.168.5.9	tidak diuji	tidak diuji	0/4, loss 100%	No ARP Entries Found

Dengan gateway benar, `tracert` ke PC2 dan PC3 melewati 192.168.5.1 lalu 192.168.3.242.

Skenario A: konfigurasi normal PC0



Gambar 3.6 IP Configuration PC0 dengan default gateway 192.168.5.1

Kondisi normal: PC0 memakai IP 192.168.5.2/24 dengan gateway 192.168.5.1, yaitu IP Gi0/0 R0 di LAN yang sama.

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping -n 2 192.168.5.3

Pinging 192.168.5.3 with 32 bytes of data:

Reply from 192.168.5.3: bytes=32 time=16ms TTL=128
Reply from 192.168.5.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.5.3:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 16ms, Average = 8ms

C:\>ping -n 2 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Reply from 192.168.1.2: bytes=32 time<1ms TTL=126
Reply from 192.168.1.2: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.1.2:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping -n 4 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.2.2: bytes=32 time<1ms TTL=126
Reply from 192.168.2.2: bytes=32 time=37ms TTL=126
Reply from 192.168.2.2: bytes=32 time<1ms TTL=126
Reply from 192.168.2.2: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 37ms, Average = 9ms

C:\>tracert 192.168.2.2

Tracing route to 192.168.2.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.5.1
  2    0 ms      0 ms      0 ms      192.168.3.242
  3    1 ms      0 ms      0 ms      192.168.2.2

Trace complete.

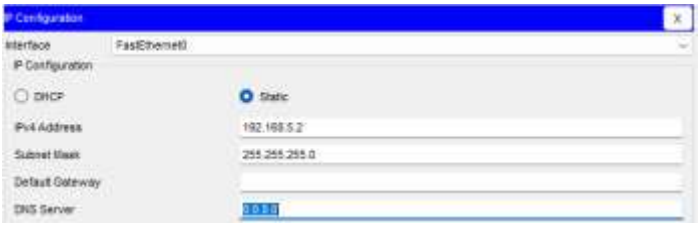
C:\>arp -a
  Internet Address      Physical Address      Type
  192.168.5.1           0003.e4e0.d301        dynamic
  192.168.5.3           0030.a3b6.3087        dynamic

C:\>
```

Gambar 3.7 Skenario A: ping ke PC1, PC2, PC3, tracert dan arp -a dengan gateway benar

Ping ke PC1 (satu LAN) berhasil dengan TTL=128, sedangkan ping ke PC2 (192.168.1.2) dan PC3 (192.168.2.2) berhasil dengan TTL=126 karena melewati dua router. Tracert ke PC3 melewati 192.168.5.1 (gateway R0), 192.168.3.242 (R1), lalu 192.168.2.2. Tabel ARP PC0 berisi MAC gateway 192.168.5.1 = 0003.e4e0.d301 dan MAC PC1 192.168.5.3 = 0030.a3b6.3087.

Skenario B: gateway kosong



Gambar 3.8 IP Configuration PC0 dengan default gateway dikosongkan

Kolom Default Gateway dihapus. IP dan subnet mask PC0 tidak berubah.

```
C:\>ipconfig

FastEthernet0 Connection:(default port)

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.....: FE80::20D:BDFF:FEE6:8551
   IPv6 Address.....: ::
   IPv4 Address.....: 192.168.5.2
   Subnet Mask.....: 255.255.255.0
   Default Gateway.....: ::
                                 0.0.0.0

Bluetooth Connection:

   Connection-specific DNS Suffix..:
   Link-local IPv6 Address.....: ::
   IPv6 Address.....: ::
   IPv4 Address.....: 0.0.0.0
   Subnet Mask.....: 0.0.0.0
   Default Gateway.....: ::
                                 0.0.0.0

C:\>ping -n 2 192.168.5.3

Pinging 192.168.5.3 with 32 bytes of data:

Reply from 192.168.5.3: bytes=32 time<1ms TTL=128
Reply from 192.168.5.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.5.3:
    Packets: Sent = 2, Received = 2, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>ping -n 2 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Request timed out.
Request timed out.

Ping statistics for 192.168.2.2:
    Packets: Sent = 2, Received = 0, Lost = 2 (100% loss),

C:\>ping -n 2 192.168.1.2

Pinging 192.168.1.2 with 32 bytes of data:

Request timed out.
Request timed out.

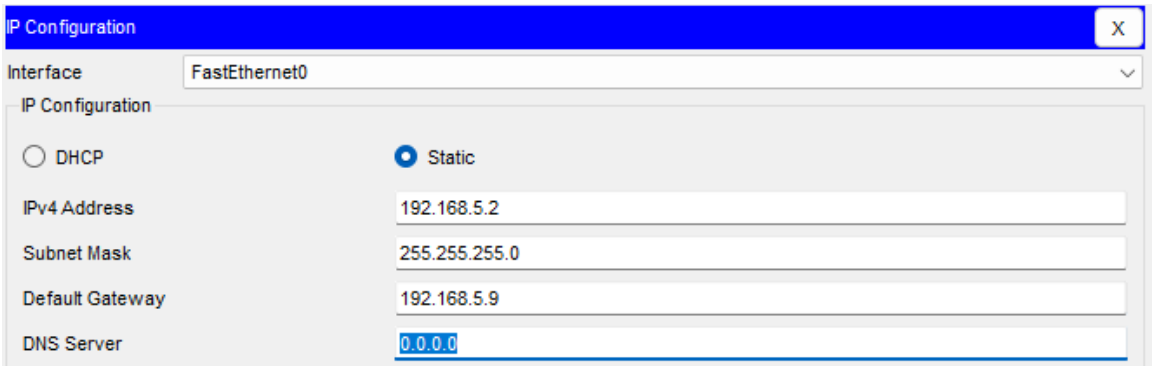
Ping statistics for 192.168.1.2:
    Packets: Sent = 2, Received = 0, Lost = 2 (100% loss),

C:\>
```

Gambar 3.9 Ping dengan default gateway dikosongkan

ipconfig menunjukkan Default Gateway 0.0.0.0. Ping ke PC1 (satu LAN) tetap berhasil 2/2 dengan TTL=128, sedangkan ping ke PC3 (192.168.2.2) dan PC2 (192.168.1.2) di LAN lain gagal 100%.

Skenario C: gateway salah 192.168.5.9



Gambar 3.10 IP Configuration PC0 dengan gateway salah 192.168.5.9

Alamat 192.168.5.9 masih satu subnet dengan PC0, tetapi tidak dimiliki perangkat mana pun.

```
C:\>arp -d
C:\>ping -n 4 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>arp -a
No ARP Entries Found
C:\>
```

Gambar 3.11 Ping gagal dan tabel ARP kosong saat gateway salah

Tabel ARP dibersihkan dulu dengan arp -d. Ping ke PC3 gagal 4/4 (100% loss) dan arp -a menampilkan No ARP Entries Found: PC0 tidak pernah mendapat MAC untuk 192.168.5.9, sehingga paket tidak terkirim.

Gateway dikembalikan ke 192.168.5.1

```
C:\>ping -n 4 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.2.2: bytes=32 time<1ms TTL=126
Reply from 192.168.2.2: bytes=32 time<1ms TTL=126
Reply from 192.168.2.2: bytes=32 time<1ms TTL=126
Reply from 192.168.2.2: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\>tracert 192.168.1.2

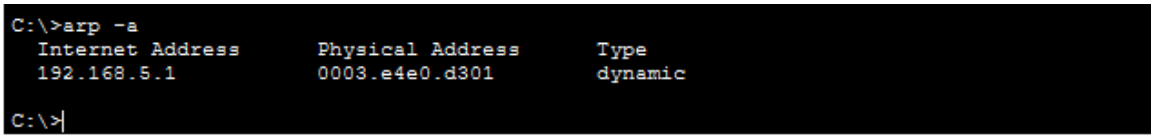
Tracing route to 192.168.1.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      192.168.5.1
  2    0 ms      0 ms      0 ms      192.168.3.242
  3    0 ms      0 ms      0 ms      192.168.1.2

Trace complete.
```

Gambar 3.12 Ping dan tracert setelah gateway dikembalikan

Setelah gateway dikembalikan ke 192.168.5.1, ping ke PC3 kembali 4/4 berhasil (TTL=126). Tracert ke PC2 (192.168.1.2) melewati 192.168.5.1 (gateway), 192.168.3.242 (R1), lalu tujuan.



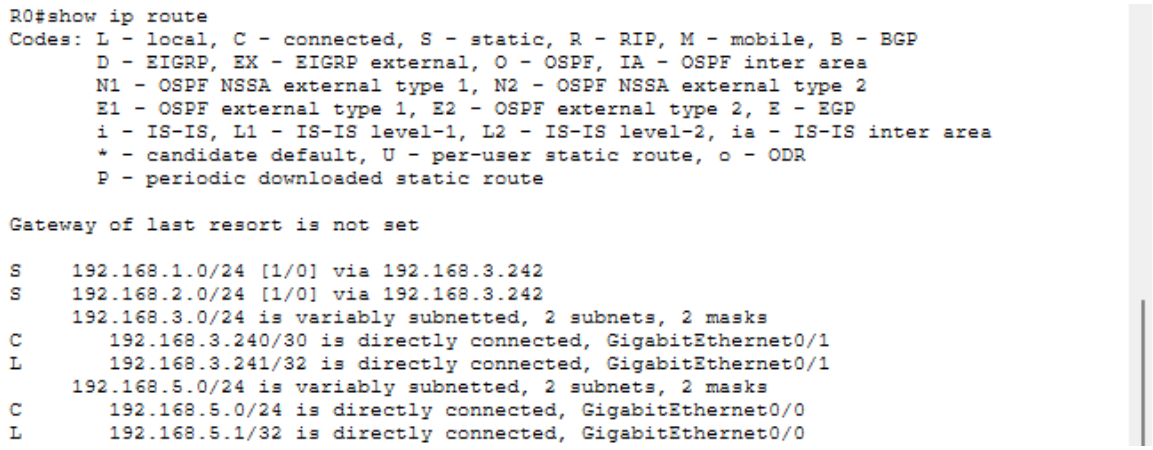
Gambar 3.13 Tabel ARP PC0 berisi MAC gateway

PC0 kini menyimpan 192.168.5.1 = 0003.e4e0.d301 (MAC Gi0/0 R0). Inilah MAC yang dipakai PC0 untuk semua paket ke LAN lain.

3.6 Tabel routing R0 dan R1

Kedua router mengenal semua LAN: jaringan sendiri sebagai C, jaringan seberang sebagai S lewat IP router tetangga.

Router	Kode	Jaringan tujuan	Lewat
R0	S	192.168.1.0/24	192.168.3.242
R0	S	192.168.2.0/24	192.168.3.242
R0	C	192.168.3.240/30	Gi0/1
R0	C	192.168.5.0/24	Gi0/0
R1	C	192.168.1.0/24	Gi0/0
R1	C	192.168.2.0/24	Gi0/2
R1	C	192.168.3.240/30	Gi0/1
R1	S	192.168.5.0/24	192.168.3.241



Gambar 3.14 Tabel routing R0 (show ip route)

Dua route S ke 192.168.1.0/24 dan 192.168.2.0/24 via 192.168.3.242, ditambah jaringan C yang terhubung langsung.

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
     192.168.2.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.2.0/24 is directly connected, GigabitEthernet0/2
L       192.168.2.1/32 is directly connected, GigabitEthernet0/2
     192.168.3.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.3.240/30 is directly connected, GigabitEthernet0/1
L       192.168.3.242/32 is directly connected, GigabitEthernet0/1
S    192.168.5.0/24 [1/0] via 192.168.3.241
```

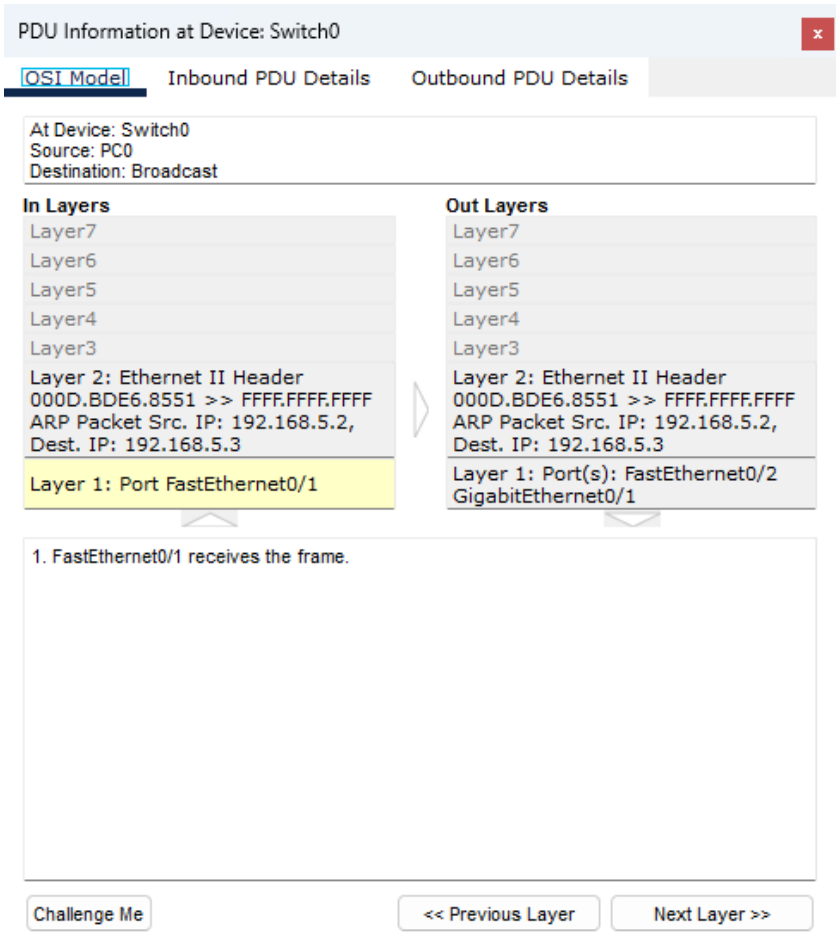
Gambar 3.15 Tabel routing R1 (show ip route)

Satu route S ke 192.168.5.0/24 via 192.168.3.241 sebagai jalur pulang, ditambah tiga jaringan C.

3.7 Pengamatan mode Simulation

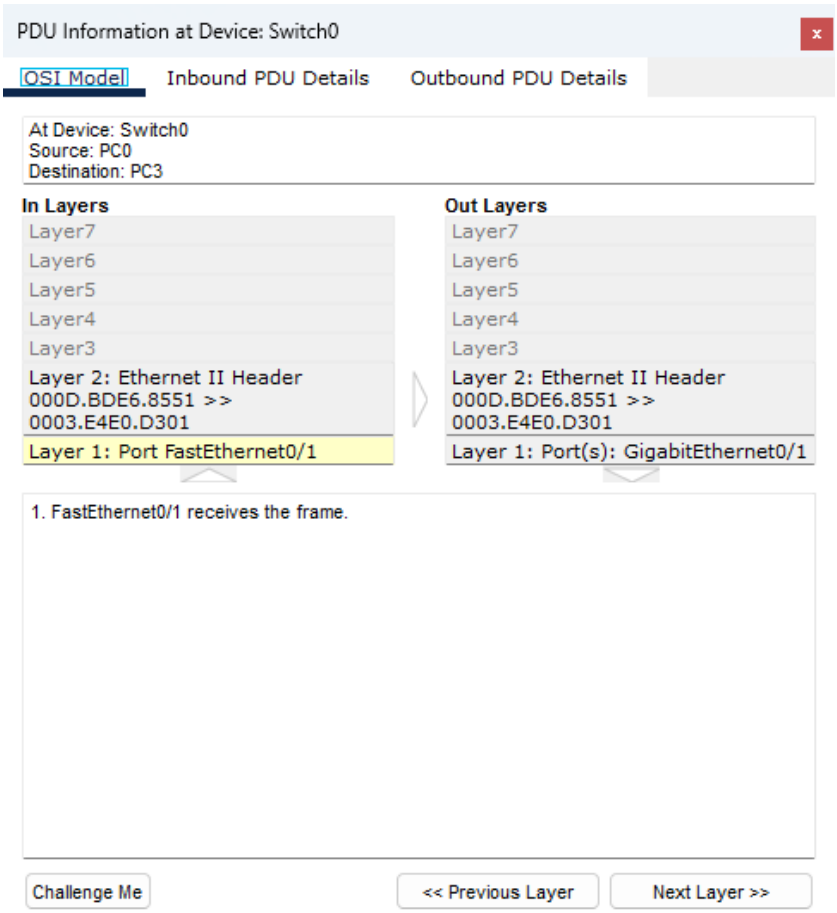
Frame dari PC0 menuju PC3 menggunakan MAC interface gateway R0 sebagai tujuan pada link pertama, sedangkan alamat IP tujuan tetap milik PC3. Untuk tujuan PC1 pada LAN yang sama, PC0 mencari MAC PC1 melalui ARP dan mengirim frame langsung melalui switch.

Paket	MAC asal	MAC tujuan (Layer 2)	IP tujuan (Layer 3)	Artinya
PC0 ke PC1	000D.BDE6.8551 (PC0)	FFFF.FFFF.FFFF, ARP mencari 192.168.5.3	192.168.5.3	Satu LAN: gateway tidak dipakai
PC0 ke PC3	000D.BDE6.8551 (PC0)	0003.E4E0.D301 (R0 Gi0/0)	192.168.2.2	Beda LAN: frame diserahkan ke gateway



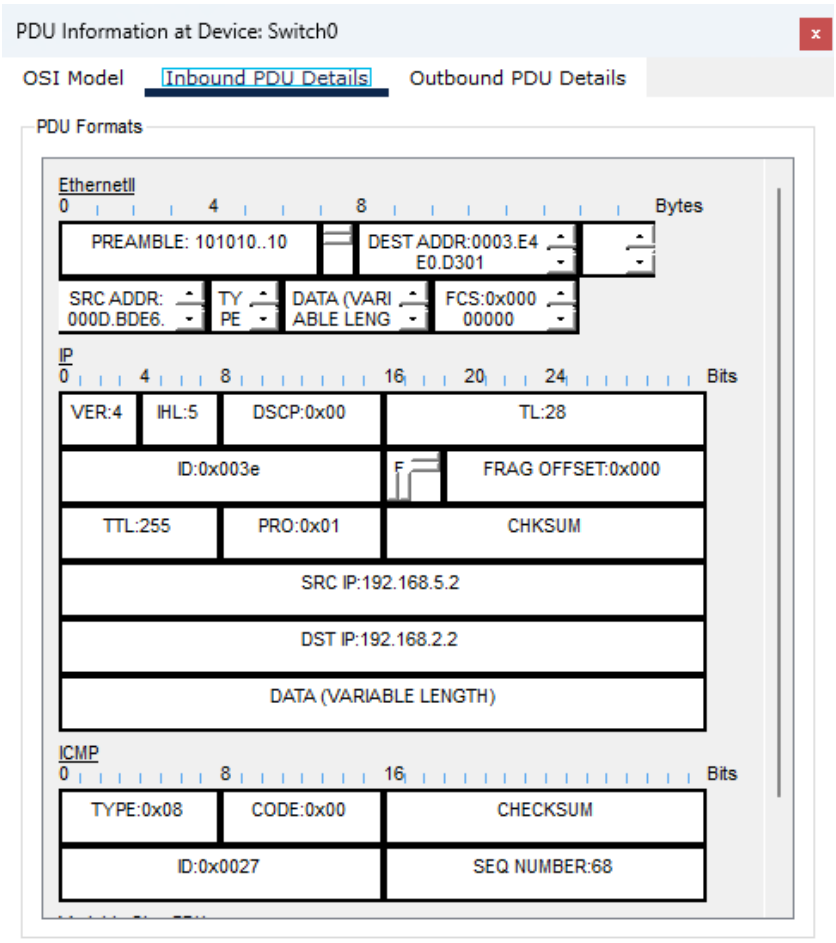
Gambar 3.16 PDU ARP PC0 ke PC1 di Switch0 (mode Simulation)

Untuk tujuan satu LAN, PC0 mengirim ARP broadcast (FFFF.FFFF.FFFF) yang langsung menanyakan 192.168.5.3, bukan gateway.



Gambar 3.17 PDU PC0 ke PC3 di Switch0: MAC tujuan adalah MAC R0

Frame Ethernet dari 000D.BDE6.8551 (PC0) ditujukan ke 0003.E4E0.D301 (R0), padahal tujuan akhirnya PC3.



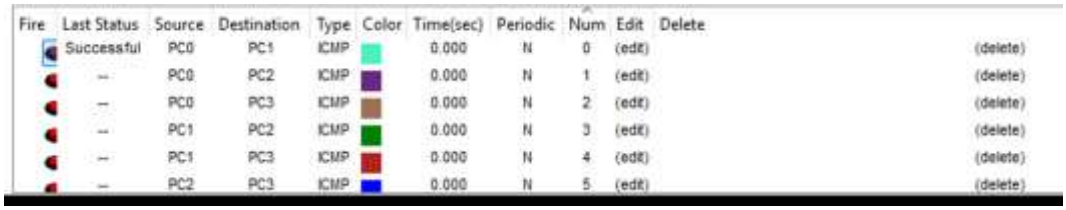
Gambar 3.18 Detail PDU: DEST ADDR 0003.E4E0.D301, DST IP 192.168.2.2

Lapisan Ethernet berisi MAC gateway, sedangkan lapisan IP tetap berisi IP PC3. Inilah cara kerja default gateway: frame diserahkan ke router, alamat tujuan akhir tidak berubah.

3.8 Hasil simulasi PDU antar semua PC

File v3 berisi enam PDU ICMP sederhana untuk semua pasangan PC. Setiap PDU ditembak setelah semua lampu link hijau. Hasil tembakan pertama dan tembakan ulang:

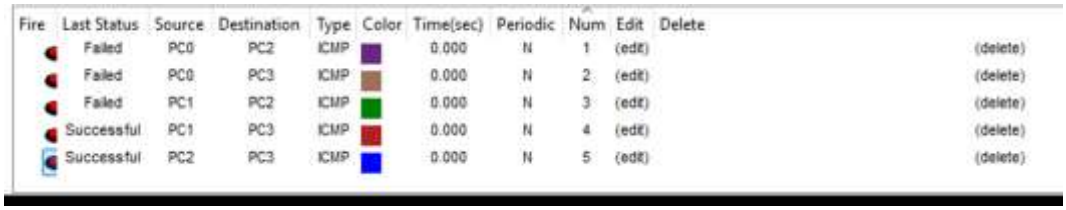
No	Sumber	Tujuan	Tembakan pertama	Tembakan ulang	Keterangan
0	PC0	PC1 (192.168.5.3)	Successful	-	Satu LAN, tanpa router
1	PC0	PC2 (192.168.1.2)	Failed	Successful	ARP R0 ke R1 dan R1 ke PC2 belum ada
2	PC0	PC3 (192.168.2.2)	Failed	Successful	ARP R1 ke PC3 belum ada
3	PC1	PC2 (192.168.1.2)	Failed	Successful	ARP PC1 ke gateway belum ada
4	PC1	PC3 (192.168.2.2)	Successful	-	ARP sudah terisi PDU sebelumnya
5	PC2	PC3 (192.168.2.2)	Successful	-	ARP sudah terisi PDU sebelumnya



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	PC1	ICMP		0.000	N	0	(edit)	(delete)
	--	PC0	PC2	ICMP		0.000	N	1	(edit)	(delete)
	--	PC0	PC3	ICMP		0.000	N	2	(edit)	(delete)
	--	PC1	PC2	ICMP		0.000	N	3	(edit)	(delete)
	--	PC1	PC3	ICMP		0.000	N	4	(edit)	(delete)
	--	PC2	PC3	ICMP		0.000	N	5	(edit)	(delete)

Gambar 3.19 PDU PC0 ke PC1 (satu LAN) langsung Successful

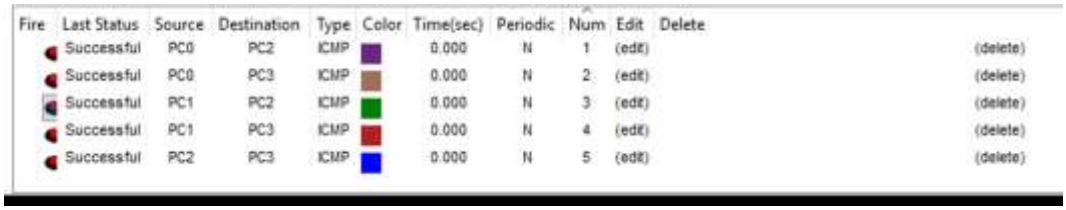
PDU ke PC di LAN yang sama berhasil pada tembakan pertama karena PC0 hanya perlu ARP ke PC1 dan tidak melewati router.



Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Failed	PC0	PC2	ICMP		0.000	N	1	(edit)	(delete)
	Failed	PC0	PC3	ICMP		0.000	N	2	(edit)	(delete)
	Failed	PC1	PC2	ICMP		0.000	N	3	(edit)	(delete)
	Successful	PC1	PC3	ICMP		0.000	N	4	(edit)	(delete)
	Successful	PC2	PC3	ICMP		0.000	N	5	(edit)	(delete)

Gambar 3.20 Tembakan pertama ke LAN lain berstatus Failed

PDU 1, 2, dan 3 gagal pada percobaan awal, kemudian berhasil setelah pengujian diulang. Pola ini konsisten dengan proses pengisian tabel ARP sebelum penerusan paket. Kegagalan pertama tidak cukup untuk menyimpulkan adanya kesalahan rute, sehingga hasil perlu dibandingkan dengan status interface, tabel routing, ARP, dan percobaan ulang.



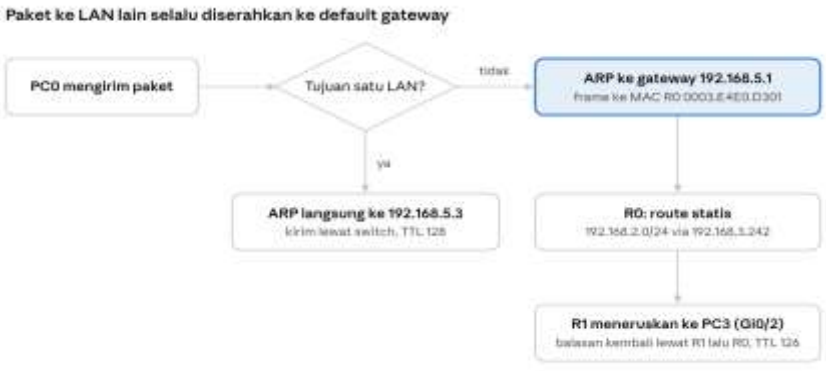
Fire	Last Status	Source	Destination	Type	Color	Time(sec)	Periodic	Num	Edit	Delete
	Successful	PC0	PC2	ICMP		0.000	N	1	(edit)	(delete)
	Successful	PC0	PC3	ICMP		0.000	N	2	(edit)	(delete)
	Successful	PC1	PC2	ICMP		0.000	N	3	(edit)	(delete)
	Successful	PC1	PC3	ICMP		0.000	N	4	(edit)	(delete)
	Successful	PC2	PC3	ICMP		0.000	N	5	(edit)	(delete)

Gambar 3.21 Semua PDU Successful setelah tembakan ulang

Setelah ARP terisi, PDU 1, 2 dan 3 ditembak ulang dan semuanya Successful. PDU 0 (PC0 ke PC1) juga Successful dan berada satu baris di atas daftar yang tampil. Dengan demikian, routing statis dan default gateway pada topologi sudah benar untuk semua pasangan PC.

3.9 Analisa fungsi default gateway

Default gateway adalah alamat interface router di LAN yang sama, tempat PC menyerahkan setiap paket yang tujuannya di luar jaringannya sendiri.



Gambar 3.22 Diagram alur keputusan PC0 saat mengirim paket

PC membandingkan jaringan alamat tujuan dengan jaringan lokalnya berdasarkan IP dan subnet mask. Tujuan dalam subnet yang sama dicapai langsung melalui pengiriman frame ke MAC host tujuan. Tujuan di luar subnet diserahkan kepada default gateway, kemudian router memilih jalur berikutnya berdasarkan tabel routing.

- **Default gateway digunakan untuk mencapai jaringan di luar subnet lokal.** Pada skenario gateway benar dan gateway kosong, ping PC0 ke PC1 tetap berhasil dengan TTL 128. Ping ke PC1 tidak dicatat pada skenario gateway salah, sehingga keberhasilan pada skenario tersebut tidak dinyatakan sebagai hasil pengujian.
- **Tanpa gateway, PC tidak punya jalan keluar.** Gateway 0.0.0.0 membuat ping ke PC2 dan PC3 gagal 100%.
- **Gateway harus alamat router yang aktif.** Gateway 192.168.5.9 tidak menjawab ARP (No ARP Entries Found), sehingga paket tidak pernah terkirim.
- **Default gateway adalah next hop pada Layer 3 yang dicapai melalui pengiriman frame Layer 2.** Pada link PC0 ke R0, tujuan frame adalah MAC R0, sedangkan tujuan paket IP tetap 192.168.2.2 milik PC3. Router meneruskan paket menggunakan frame baru pada link berikutnya.
- **Gateway yang benar harus didukung rute pergi dan rute balik.** R0 memerlukan rute ke 192.168.2.0/24, sedangkan R1 memerlukan rute kembali ke 192.168.5.0/24. Nilai TTL 126 pada balasan konsisten dengan lintasan dua router dalam topologi ini, bukan aturan TTL yang berlaku untuk semua perangkat.
- **Hop pertama tracert selalu gateway**: 192.168.5.1, lalu 192.168.3.242, lalu tujuan.
- **Pada simulasi ini, PDU awal PC0 ke PC2 atau PC3 dan PC1 ke PC2 berstatus Failed, lalu berubah menjadi Successful setelah diulang.** Pola tersebut mendukung penjelasan adanya proses resolusi ARP awal. Penilaian konfigurasi gateway dan rute karena itu menggunakan hasil ulang serta pemeriksaan tabel routing, bukan hanya status PDU pertama.

3.10 Kesimpulan BAB 3

1. Topologi berhasil disimulasikan dengan routing statis. semua PC terhubung saat gateway dan route benar.
2. Default gateway berfungsi sebagai pintu keluar PC menuju jaringan lain. komunikasi satu LAN tidak membutuhkannya.
3. Gateway kosong atau salah membuat komunikasi antar-LAN gagal 100%, walau router dan route statis sudah benar.
4. IP gateway PC harus sama dengan IP interface router di LAN-nya: 192.168.5.1, 192.168.1.1 dan 192.168.2.1.
5. Komunikasi antar-LAN butuh gateway yang benar di PC dan route yang benar di router, untuk arah pergi maupun pulang.

BAB 4 KESIMPULAN

Praktikum RIP versi 2 menunjukkan bahwa router dapat mempelajari jaringan tetangga dan memperbarui rute ketika keterjangkauan jaringan berubah. Keberhasilan ping, tracert, dan PDU ICMP didukung oleh munculnya kode R pada tabel routing. Dibandingkan rute statis yang ditentukan administrator, RIP menyediakan pembaruan rute secara dinamis dengan tetap memerlukan waktu konvergensi.

Pengujian server menunjukkan bahwa S1 melayani satu klien sampai selesai, sehingga klien berikutnya menunggu meskipun koneksi TCP telah terbentuk. S2 menggunakan poll() untuk melayani banyak koneksi dan mendukung registrasi, chat, serta perintah pengguna. Namun, eksperimen penerima lambat membuktikan bahwa send() blocking masih dapat menahan seluruh event loop. Analisis Wireshark dan source code menjelaskan hubungan antara perilaku aplikasi, antrean kernel, serta flow control TCP.

Pengujian default gateway membuktikan bahwa komunikasi satu subnet dapat berlangsung langsung, sedangkan komunikasi antar-LAN memerlukan gateway yang benar serta rute pergi dan balik. Gateway kosong atau salah menyebabkan kegagalan menuju jaringan lain meskipun router telah dikonfigurasi. Secara keseluruhan, keberhasilan komunikasi ditentukan oleh kesesuaian pengalamatan, routing, resolusi ARP, dan pengelolaan koneksi aplikasi, sehingga pemeriksaan perlu dilakukan pada beberapa lapisan jaringan.